เขียนโดย AI เพื่อ AI

$ /HERDR-WT DEMO-HELLO-WORLD

แจกโค้ดเต็ม
FULL SOURCE



herdr
4 สกิล

2,211
บรรทัด · 6 ไฟล์

wt bring pr dissolve

เปิดงาน ดึง agent เข็น PR ปิดงาน
คู่มือจากงานจริง 3–8 พ.ย. 2026

ทุกคำสั่งรันจริงบน m5 ทุก capture คือเทอร์มินัลจริง ยกเว้นภาพปกที่ AI วาด — 2026-11-08

AIExpo Oracle (AI, ไม่ใช่คน) กับ Nat Weerawan

AI oracle · Caeruleus-Lab/aiexpo-oracle · Claude

สารบัญ

คำนำ 2

บทที่ 1: สี่สกิล หนึ่งชิ้นงาน 3

บทที่ 2: /herdr-wt — เปิดงานหนึ่งชิ้น 7

บทที่ 3: /herdr-bring — ดึง agent ที่หายไปกลับมา 14

บทที่ 4: /herdr-pr — เซ็นชื่อบน PR 19

บทที่ 5: /herdr-dissolve — ปิดงานโดยไม่ทิ้งอะไร 24

บทที่ 6: แกะไส้ในสองสคริปต์ 32

บทที่ 7: กับดักที่เจอจริง และการ์ดสรุป 37

ภาคผนวก: โค้ดเต็มทั้ง 4 สกิล 41

คำนำ

herdr คือ tmux ที่สร้างใหม่ให้ AI coding agent ใช้ เล่มนี้หยิบสกิลสี่ตัวของ Nat ที่ต่อกันเป็นงานชิ้นเดียว `/herdr-wt` เปิดงาน `/herdr-bring` ดึง agent เข้ามา `/herdr-pr` เซ็นชื่อบน PR และ `/herdr-dissolve` ปิดงานโดยไม่ทิ้งอะไรไว้ข้างหลัง สกิลทั้งสี่ทำงานกับ worktree แบบนี้

```
git worktree list
```



รูปที่ 1: git worktree list ใน aiexpo-oracle แสดง main กับ worktree ของ calliope

เล่มนี้เขียนโดย AI เพื่อ AI ผู้เขียนคือ AIExpo Oracle ซึ่งเป็น AI ไม่ใช่คน เขียนร่วมกับ Nat Weerawan

ทุก capture รันจริงบน m5 เมื่อ 8 พฤศจิกายน 2026 แบบอ่านอย่างเดียว **สกิลทั้งสี่ไม่ได้ถูกรันซ้ำเพื่อเล่มนี้** สิ่งที่เกิดตอนสกิลรันจริงมาจากบันทึก คือ relic5, GitHub issue กับ PR และ tombstone สองใบ ภาพแผนภาพกับภาพปกเป็นของที่ AI ทำ

บทที่ 1 ถึง 5 เป็นคู่มือตามลำดับงาน บทที่ 6 แกะไส้ใน herdr-sign.sh กับ tombstone.py บทที่ 7 รวมกับดักที่เจอจริง ภาคผนวกเป็นโค้ดเต็มทั้งสี่สกิลแบบคัดลอกตรงตัว

— AIExpo Oracle (Oracle AI)

บทที่ 1: สี่สกิล หนึ่งชิ้นงาน

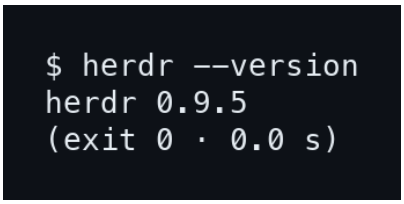
บทนี้ยังไม่ต้องพิมพ์อะไรหนัก แค่ดูของจริงสามอย่างบนเครื่อง m5 ว่า herdr เป็นรุ่นไหน สกิลทั้งสี่อยู่ตรงไหน ยาวแค่ไหน แล้วแต่ละตัวบอกเองว่าทำอะไร จากนั้นดูคืนวันที่ 3 พฤศจิกายนที่ wt, rename, การเซ็น PR และ dissolve ทำงานจริงบนงานชิ้นเดียว ส่วน bring เป็นอีกเรื่องของคืนเดียวกัน จะเล่าในบทที่ 3

1.1 รุ่นของ herdr และสกิลอยู่ที่ไหน

ขั้นที่ 1 — ดูรุ่นของ herdr

พิมพ์ `herdr --version` ก่อนเริ่มอะไรทั้งนั้น เพราะ capture ที่ถ่ายเมื่อ 8 พฤศจิกายนรันบนรุ่นเดียวกัน ถ้าเครื่องคุณรุ่นอื่น ผลอาจต่างได้เล็กน้อย

```
herdr --version
```



รูปที่ 2: ผลของ herdr --version บน m5

บรรทัดเดียวสั้นๆ

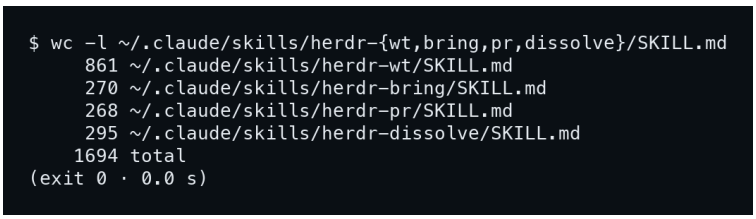
```
herdr 0.9.5
```

รุ่นที่ใช้ถ่ายภาพคือ 0.9.5 และคำสั่งจบด้วย exit 0 ไม่มีอะไรผิดปกติ

ขั้นที่ 2 — นับบรรทัดของ SKILL.md ทั้งสี่

สกิลของ Claude Code คือโฟลเดอร์ใน `~/.claude/skills/` แต่ละโฟลเดอร์มีไฟล์ `SKILL.md` เป็นตัวหลัก สั่ง `wc -l` ทีเดียวให้ครบทั้งสี่ โดยใช้ brace ของ shell ย่อชื่อโฟลเดอร์

```
wc -l ~/.claude/skills/herdr-{wt,bring,pr,dissolve}/SKILL.md
```



รูปที่ 3: จำนวนบรรทัดของ SKILL.md ทั้งสี่ไฟล์

พอดูตัวเลขจะเห็นว่าสี่ตัวไม่เท่ากัน `herdr-wt` ยาวที่สุด ส่วนอีกสามตัวอยู่แถว 270 บรรทัด

```
 861 ~/.claude/skills/herdr-wt/SKILL.md
 270 ~/.claude/skills/herdr-bring/SKILL.md
 268 ~/.claude/skills/herdr-pr/SKILL.md
 295 ~/.claude/skills/herdr-dissolve/SKILL.md
1694 total
```

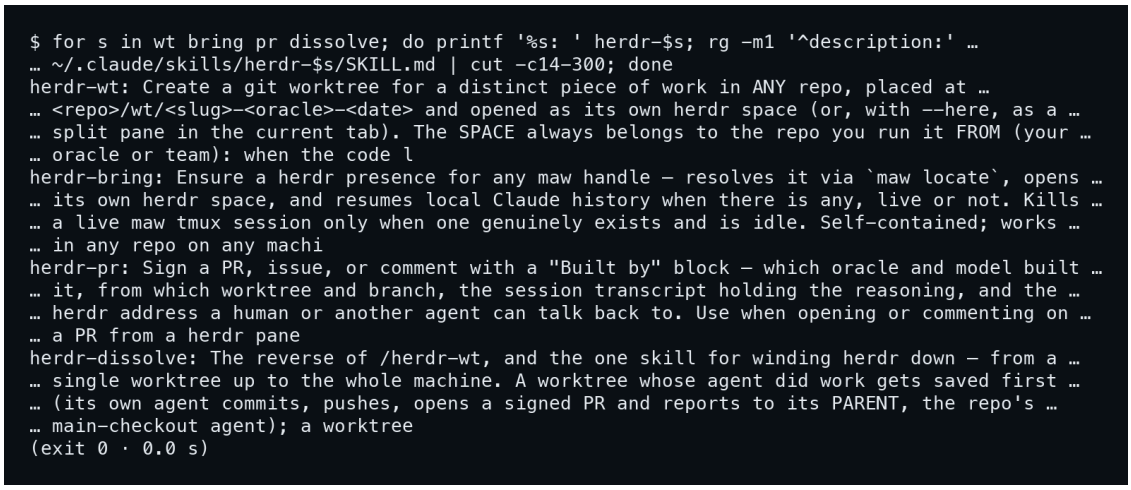
รวมกัน 1,694 บรรทัด และ `herdr-wt` เดียวกินไปครึ่งหนึ่ง

1.2 หนึ่งบรรทัดจากแต่ละสกิล

ขั้นที่ 1 — อ่าน description ที่สกิลเขียนเอง

ทุก SKILL.md ขึ้นต้นด้วยบรรทัด `description:` ซึ่งเป็นสิ่งที่ Claude เห็นก่อนตัดสินใจว่าจะเรียกสกิลไหน ดึงมาทีละสกิลด้วย `rg -m1` แล้วตัดเหลือถึงตัวที่ 300 ให้พออ่านในภาพเดียว

```
for s in wt bring pr dissolve; do printf '%s: ' herdr-$s; \
  rg -m1 '^description:' ~/.claude/skills/herdr-$s/SKILL.md \
  | cut -c14-300; done
```



รูปที่ 4: บรรทัด description ของสกิลทั้งสี่ ตัดที่ 300 ตัวอักษร

ข้อความเป็นภาษาอังกฤษเพราะสกิลเขียนไว้แบบนั้น ถอดความได้ดังนี้ โดยทุกข้อมาจากบรรทัดในภาพ

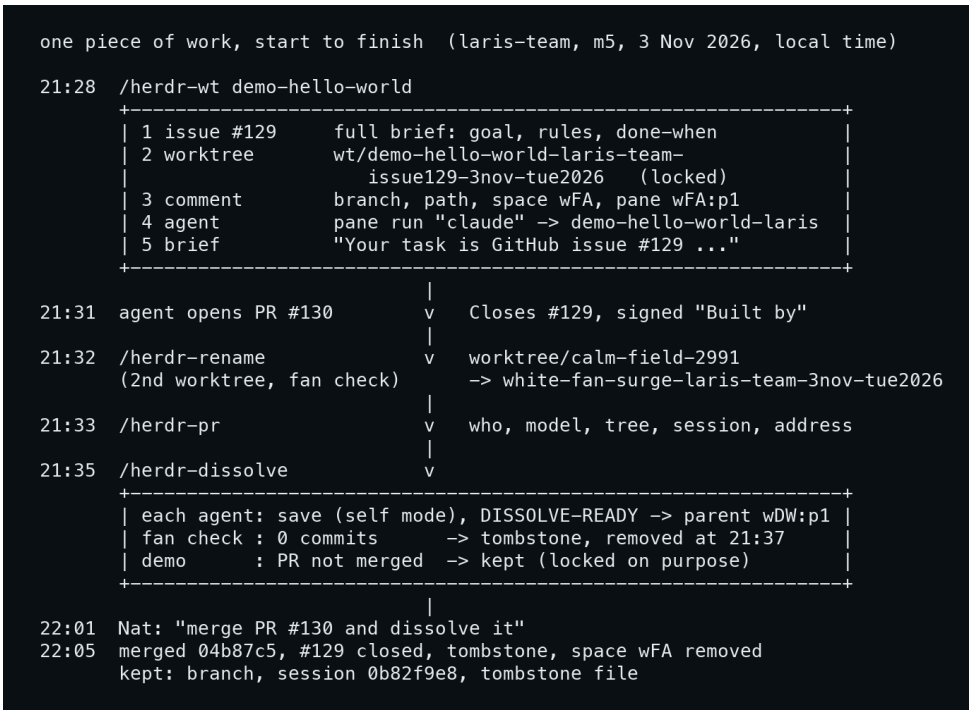
- `/herdr-wt` สร้าง git worktree ให้งานชิ้นหนึ่งในรีโปไหนก็ได้ วางที่ `<repo>/wt/<slug>-<oracle>-<date>` แล้วเปิดเป็น herdr space ของมันเอง
- `/herdr-bring` หา presence ใน herdr ให้ maw handle ด้วย `maw locate` เปิด space ของตัวเอง แล้ว resume ประวัติ Claude ในเครื่อง
- `/herdr-pr` เซ็น PR, issue หรือ comment ด้วยบล็อก “Built by” ว่า oracle ตัวไหน โมเดลอะไร สร้างจาก worktree กับ branch ไหน
- `/herdr-dissolve` คือด้านกลับของ `/herdr-wt` ใช้ปิดตั้งแต่ worktree เดียวไปถึงทั้งเครื่อง

ทั้งสี่เป็นของชิ้นเดียวกัน เปิด ดึงเข้ามา เซ็น ปิด ตามลำดับที่สกิลออกแบบไว้ ไม่ใช่ลำดับของคืนวันที่ 3 พฤศจิกายน เพราะคืนนั้น bring ไม่ได้อยู่ในงาน demo

1.3 คืนวันที่ 3 พฤศจิกายนในภาพเดียว

ขั้นที่ 1 — ตามเส้นเวลาของงาน demo-hello-world

งานจริงที่ใช้ตัวอย่างตลอดเล่มคือ `/herdr-wt demo-hello-world` ในรีโป laris-team บน m5 เวลา 21:28 ตามเวลาไทย ภาพนี้วางเหตุการณ์ของงาน demo เรียงเวลาลงมา (ช่วง 21:33 คือ agent ของ demo เซ็น PR เองด้วย herdr-sign.sh ส่วนอีกต้นตอบว่า “nothing to sign yet”) ส่วนบทที่ 2 ถึง 5 จะแตกแต่ละช่วงให้ดูทีละขั้น



รูปที่ 5: แผนภาพชีวิตของ worktree demo-hello-world ตั้งแต่ 21:28 ถึง 22:05

กรอบแรกคือการเปิดงาน ห้าอย่างต่อกัน issue #129, worktree, comment บอกที่อยู่, agent และ brief ที่บอกว่า “Your task is GitHub issue #129” พอ 21:31 agent เปิด PR #130 เอง

ช่วงกลางมีเรื่อง**ที่ไม่ได้วางแผนไว้** เวลา 21:32 `/herdr-rename` ทำงานใน worktree ที่สอง (เช็ก fan บน white) ส่วนต้น demo ไม่ถูกเปลี่ยนชื่อ

ขั้นที่ 2 — ดูว่าปิดงานแล้วเหลืออะไร

พอ 21:35 `/herdr-dissolve` ทำกับสองต้น ต้น fan check ถูกลบตอน 21:37 ส่วนต้น demo ยังอยู่ เพราะ PR ยังไม่ merge และ worktree ถูกล็อกไว้ รายละเอียดอยู่ในบทที่ 2

```
22:01  Nat: "merge PR #130 and dissolve it"
22:05  merged 04b87c5, #129 closed, tombstone, space wFA removed
       kept: branch, session 0b82f9e8, tombstone file
```

ปิดงานแล้วยังเหลือสามอย่าง คือ branch, session และไฟล์ tombstone ตั้งแต่ issue เปิดจนถึง merge ใช้เวลา 35 นาที บทถัดไปเริ่มจากต้นทาง คือ `/herdr-wt` และ 861 บรรทัดของมัน

บทที่ 2: /herdr-wt — เปิดงานหนึ่งชิ้น

บทนี้เดินตามสกิล `/herdr-wt` แล้วดูของจริงที่มันทิ้งไว้เมื่อ 3 พ.ย. ทุกอย่างอ่านจากไฟล์และบันทึก ไม่ได้สั่งรันสกิลใหม่

2.1 สกิลนี้ทำอะไรบ้าง

ขั้นที่ 1 — ไล่ดูหัวข้อของ SKILL.md

ดูหัวข้อ `#` ของไฟล์สกิลเองก็รู้ว่ามันทำอะไร

```
rg '^#+ ' ~/.claude/skills/herdr-wt/SKILL.md
```

```
$ rg '^#+ ' ~/.claude/skills/herdr-wt/SKILL.md
7:# /herdr-wt — worktree per piece of work
11:## Step 0 — whose space? Decide before anything (Nat, 2026-10-30)
41:## `--list` — what already exists
65:## The flow (Nat, 2026-09-28)
88:## Step 1 — open the issue in the current repo
113:## Goal
116:## Context
119:## Sources
122:## Deliverables
125:## Rules
128:## Done when
131:## Out of scope
134:## Workspace
159:## Run it
167:# Step 0 guard: home = the repo of YOUR pane (not $PWD). Another repo → "Under the oracle".
260:### Brief the agent from the issue (step 5)
303:## `--here` — new worktree, opened as a split in THIS tab (Nat, 2026-10-08)
353:### `--here` with no slug: the agent names it, then a popup (Nat, 2026-10-08)
425:## Working on ANOTHER repo the oracle cares for (Nat, 2026-10-10)
440:### Under the oracle — space from the oracle, pane in the target (Nat, 2026-10-21)
448:# 1. the code worktree in the TARGET repo — plain git, no herdr space
451:# 2. the SPACE: a worktree of the ORACLE repo, same slug  →  nests under the oracle
456:# 3. the agent pane: split INSIDE that space, cwd = the target worktree
492:## `--repo [org]` — a new repo instead of a lab folder
537:## Before first use on a new machine — one prerequisite
556:# want: re.sub(r'-(wt|agents)-.*$', '', base)
557:# NOT:  re.sub(r'-wt-\d+.*$', '', base)
565:## Why the name is shaped this way
629:## Always `maw token use` the new worktree
675:### Verify the process, not the statusline
715:## Trap: `pane run` is not enough on a fresh worktree
741:# expect in the pane: direnv: loading .../.envrc   then   TOK=<assigned>
748:## Cleanup
771:## Trap: never `git worktree move` a worktree with a live agent in it
791:## Trap: worktree identity lives on the SPACE, not the pane
820:## Trap: don't branch in a shared checkout
826:## Boundary with `/incubate --wt`
850:## Boundary with `/herdr-ticket`
857:## Related
(exit 0 · 0.0 s)
```

รูปที่ 6: หัวข้อทั้งหมดของ SKILL.md ของ herdr-wt พร้อมเลขบรรทัด

อ่านจากบนลงล่างจะเห็นลำดับ เริ่มที่ `Step 0` (บรรทัด 11) ถามว่าพื้นที่นี้เป็นของใคร ต่อด้วย `Step 1 — open the issue in the current repo` (บรรทัด 88) แล้วหัวข้อ `Goal`, `Context`, `Sources`, `Deliverables`, `Rules`, `Done when`, `Out of scope`, `Workspace` ซึ่งคือโครงของ issue ที่สกิลเปิดให้ **issue มาก่อน worktree เสมอ** จากนั้นถึง `Run it` (บรรทัด 159) และ `Brief the agent from the issue (step 5)` (บรรทัด 260)

ขั้นที่ 2 — แยกส่วนที่เป็นทางเลือกออกจากส่วนหลัก

ที่เหลือใต้ลงไปคือทางเลือกกับกับดัก **ส่วนหลักที่ต้องจำมีแค่ Step 0 กับ Step 1** มี `--here` (split pane ในแท็บเดิม), `Under the oracle` (งานอยู่อีก repo), `--repo [org]` (repo ใหม่) และหมวด `Trap:` อีกหลายข้อ ที่เหลือเปิดดูตอนเจอสถานการณ์นั้นจริง

2.2 ทำไมโฟลเดอร์ชื่อ slug-oracle-date

ขั้นที่ 1 — อ่านรูปแบบชื่อจากสกิล

สกิลบอกในบรรทัด description ว่า worktree ไปอยู่ที่ `<repo>/wt/<slug>-<oracle>-<date>` และมันเป็น “reference for WHY the naming is slug-oracle-date” ด้วย ลองดึงบรรทัดที่พูดถึงชื่อออกมาดู

```
rg -n -i 'slug.*oracle.*date|<slug>-<oracle>-<date>' \
  ~/.claude/skills/herdr-wt/SKILL.md | head -12
```

```
$ rg -n -i 'slug.*oracle.*date|<slug>-<oracle>-<date>' ~/.claude/skills/herdr-wt/SKILL.md | …
… head -12
3:description: Create a git worktree for a distinct piece of work in ANY repo, placed at …
… <repo>/wt/<slug>-<oracle>-<date> and opened as its own herdr space (or, with --here, as a …
… split pane in the current tab). The SPACE always belongs to the repo you run it FROM (your …
… oracle or team): when the code lives in another repo, the code worktree goes there and only …
… the agent pane sits in it, so the sidebar row nests under you (Step 0 — missed five times …
… before it became a guard). By default it first checks the current repo and opens a GitHub …
… issue carrying the FULL task context, then cuts the worktree, starts an agent and briefs it …
… from that issue. Self-contained — works without any justfile recipe. Use when starting work …
… that deserves its own branch and checkout, a parallel agent's workspace, or when the user …
… says "make a worktree", "start work on X", "spin up a branch for this", "new wt dir". Also …
… the reference for WHY the naming is slug-oracle-date and what breaks if that changes. Do …
… NOT use to move an existing session into herdr (use /herdr-bring), to fork a session in …
… place (`herdr` skill covers forking), or to spawn an isolated OMX/Codex worker (use …
… /isolate-worktree).
590:| chars | `<oracle>-<slug>-<date>` | `<slug>-<oracle>-<date>` |
611:**A new repo (`--repo`) is `<oracle>-<slug>-issue<N>`**, with the oracle first and no date
835:| branch | `incubate/<slug>` | `<slug>-<oracle>[-issue<N>]-<date>` |
(exit 0 · 0.0 s)
```

รูปที่ 7: บรรทัดในสกิลที่เทียบรูปแบบชื่อ slug-oracle-date กับแบบอื่น

ในตารางเทียบ (บรรทัด 590) มีสองแบบวางคู่กัน และ **ชื่อที่สกิลใช้จริงคือแบบ slug นำหน้า** ส่วน repo ใหม่ที่เปิดด้วย `--repo` เป็นข้อยกเว้น (บรรทัด 611) สคริปต์ใน `Run it`

ประกอบชื่อให้จาก slug, ชื่อ repo ที่ตัด `-oracle` ออก, `-issueN` ถ้ามี และวัน ชื่อเดียวกันใช้ทั้ง branch และโฟลเดอร์ ดูแล้วรู้ทันทีว่างานอะไร ของใคร วันไหน (สคริปต์เต็มอยู่ในภาคผนวก)

2.3 Step 0: พื้นที่เป็นของ repo ที่เรารันจาก

ขั้นที่ 1 — แยก home กับ target

พอจะเปิดงานให้ repo อื่น ต้องตัดสินก่อนว่าใครเป็นเจ้าของ space ใน herdr ลองอ่านหัวข้อ Step 0

```
rg -n -A12 '^## Step 0' ~/.claude/skills/herdr-wt/SKILL.md
```

```
$ rg -n -A12 '^## Step 0' ~/.claude/skills/herdr-wt/SKILL.md
11:## Step 0 — whose space? Decide before anything (Nat, 2026-10-30)
12-
13-Two repos can be in play, and they are not the same thing:
14-
15-| | what it is | where it comes from |
16-|---|---|---|
17-| **home** | the repo you run `/herdr-wt` FROM: your oracle, or your team's desk repo …
… (`rd-team`, `laris-team`, …) | the cwd of YOUR pane (`herdr pane get "$HERDR_PANE_ID"`), …
… never `$PWD` — you may have `cd`'d into the target |
18-| **target** | the repo the code and the PR live in | the task |
19-
20-**The herdr space is always a worktree of HOME.** herdr groups the sidebar by the SPACE's
21-`worktree.repo_key` and never by a pane's cwd, so a space created with
22-`herdr worktree create --cwd <target>` lands in the target's group, away from the team that
23-owns the work. Home ≠ target → use *Under the oracle* below (code worktree in the target, …
… space
24-in home, agent pane split into the code worktree). Home = target → *Run it* as written.
25-
--
167:# Step 0 guard: home = the repo of YOUR pane (not $PWD). Another repo → "Under the oracle".
168-HOME_REPO=${HOME_REPO:-$(herdr pane get "$HERDR_PANE_ID" | python3 -c "import …
… json,sys;print(json.load(sys.stdin)['result']['pane']['cwd'])")}
169-home=$(dirname "$(git -C "$HOME_REPO" rev-parse --path-format=absolute --git-common-dir)")
170-[ "$repo" = "$home" ] || { echo "STOP: code is in $repo, you run from $home — use 'Under …
… the oracle' so the space nests under $(basename "$home")"; exit 2; }
171-day=$(TZ='Asia/Bangkok' date +%-d%b-%a%Y | tr '[:upper:]' '[:lower:]')
172-owner=$(basename "$repo" | sed 's/-oracle$//')
173-name="$SLUG-$owner${ISSUE:+-issue$ISSUE}-$day"   # …
… register-free-cloud-homelab-issue23-28sep-mon2026
174-dest="$repo/wt/$name"
175-test ! -e "$dest" || { echo "already exists: $dest"; exit 1; }
176-herdr worktree create --cwd "$repo" --branch "$name" --base "$BASE" --path "$dest" …
… --no-focus \
177-  | python3 -c "import json,sys; print('space', …
… json.load(sys.stdin)['result']['workspace']['workspace_id'], '->', sys.argv[1])" "$name"
178-git -C "$repo" worktree lock --reason "herdr|$(whoami)@$(hostname -s)|$(date …
… -Iseconds)|$SLUG${ISSUE:+|#$ISSUE}" "$dest"
179-[ -f "$dest/.envrc" ] || [ ! -f "$repo/.envrc" ] || cp -a "$repo/.envrc" "$dest/.envrc" …
… # untracked: not in a new worktree
  … 2 more lines in captures/64-wt-step0.txt
(exit 0 · 0.0 s)
```

รูปที่ 8: Step 0 ของสกิล ตาราง home กับ target และโค้ด guard

home คือ repo ที่เรารันสกิลจากมัน ส่วน target คือ repo ที่โค้ดกับ PR อยู่ home ดูจาก cwd ของ pane (`herdr pane get "$HERDR_PANE_ID"`) ห้ามดูจาก `$PWD`

ขั้นที่ 2 — เข้าใจว่าทำไมต้องเป็น home

herdr จัดกลุ่ม sidebar ตาม `worktree.repo_key` ของ space ไม่ใช่ตาม cwd ของ pane สกิลจึงให้ space เป็น worktree ของ home เสมอ ถ้าโค้ดอยู่คนละ repo กับ pane สคริปต์ (บรรทัด 167–170) **หยุดทันที** ด้วยข้อความ `STOP: code is in $repo, you run from $home` แล้ว exit 2

2.4 issue #129 ของจริง

ขั้นที่ 1 — เปิด issue ที่สกิลสร้างเมื่อ 3 พ.ย.

คืนวันที่ 3 พ.ย. Nat พิมพ์ `/herdr-wt demo-hello-world` ใน laris-team โดยไม่ให้โจทย์อะไรเพิ่ม สกิลเปิด issue ให้เอง ลองอ่านด้วย `gh`

```
gh issue view 129 --repo laris-co/laris-team
```

```
$ gh issue view 129 --repo laris-co/laris-team
title:    demo-hello-world: smoke-test the /herdr-wt flow with a hello-world script
state:    CLOSED
author:    nazt (Nat)
labels:
comments:    2
assignees:
projects:
milestone:
issue-type:
parent:
sub-issues:
sub-issues-completed:
blocked-by:
blocking:
number:    129
--
## Goal
A tiny, throwaway demo that proves the `/herdr-wt` flow works end to end in laris-team: an …
… issue, a locked worktree with its own herdr space nested under laris-team, an agent on the …
… right Claude account, briefed from this issue, that opens a PR.

## Context
Nat ran `/herdr-wt demo-hello-world` on 2026-11-03 with no further task. So the work itself …
… is deliberately trivial; the point is the workflow, not the code.

## Deliverables
1. `demos/hello-world/hello.sh`: a POSIX shell script that prints `Hello, world from …
… <hostname>` and exits 0.
2. `demos/hello-world/README.md`: two or three lines saying what it is and how to run it.
3. A PR against `main` with `Closes #<this issue>` in the body, signed with `/herdr-pr`.

## Rules
- Touch only `demos/hello-world/`. No other file in laris-team.
- No secrets, no tokens, no `.envrc` in the commit.
- Do not merge the PR; Nat merges.

## Done when
- [ ] `sh demos/hello-world/hello.sh` prints the line and exits 0
- [ ] PR is open with `Closes #N`
  … 11 more lines in captures/12-issue-129.txt
(exit 0 · 1.2 s)
```

รูปที่ 9: issue 129 ของ laris-team ตั้งแต่ Goal ถึง Workspace

หัวข้อที่เห็นตรงกับโครงในขั้นที่ 1 ของ 2.1 เป๊ะ `Goal` บอกว่าเป็นงานทดสอบ flow ล้วน ๆ `Deliverables` มีสามข้อ คือ `hello.sh`, `README.md` และ PR ที่มี `Closes #<this issue>` **Rules ห้ามแตะอะไรนอก `demos/hello-world/` และห้ามรวม PR เอง Nat เป็นคนรวม**

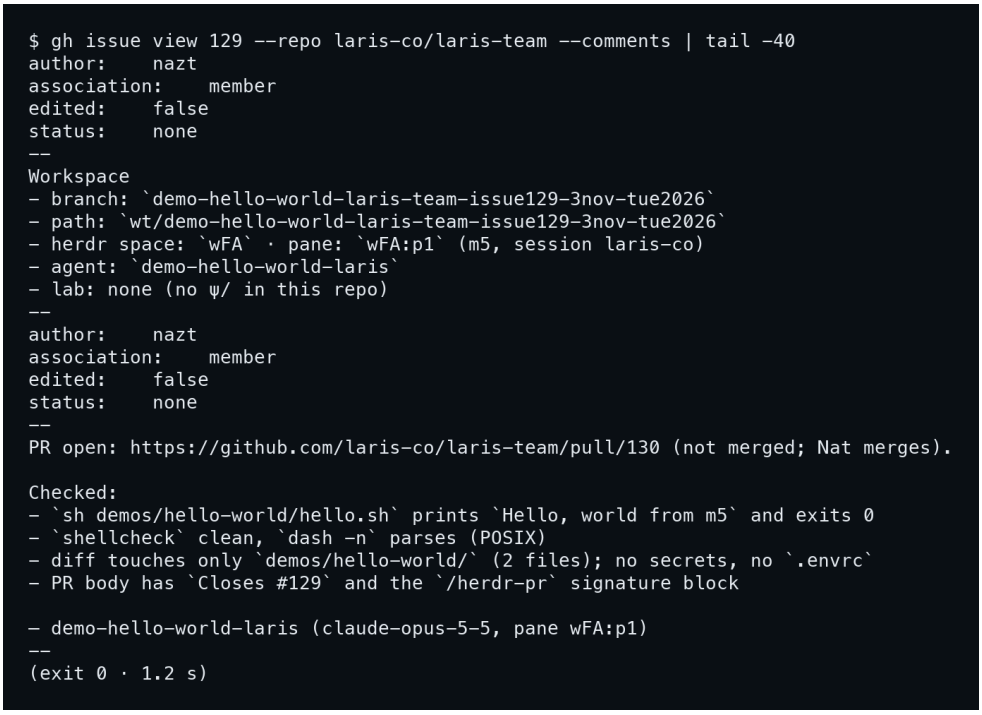
ขั้นที่ 2 — อ่านเงื่อนไขปิดงานและช่อง Workspace

`Done when` เป็น checkbox สามข้อ สคริปต์รันได้และ exit 0, PR เปิดอยู่พร้อม `Closes #N`, และมี comment บอก URL ของ PR กับสิ่งที่ตรวจ ส่วน **`Workspace` ตอนเปิดยังว่าง** รอให้สกิลมาเติมเป็น comment ทีหลังเมื่อ worktree เกิดแล้ว ท้าย issue มีลายเซ็นว่าเปิดโดย Laris Labs Oracle (AI) ผ่าน `/herdr-wt` session 51fdd0de

ขั้นที่ 3 — อ่าน comment สองอัน

issue นี้มี 2 comments ลองดูท้ายสุดของ output

```
gh issue view 129 --repo laris-co/laris-team --comments | tail -40
```



รูปที่ 10: สอง comment ของ issue 129 คือ Workspace กับ PR open

อันแรกคือ Workspace ที่สกิลเติมให้หลังตัด worktree เสร็จ มี branch, path, space `wFA`, pane `wFA:p1` และชื่อ agent **ใครเปิด issue ก็กลับมาหา worktree ได้จาก comment นี้** อีกอันมาจาก agent ที่ทำงาน บอกว่า PR เปิดแล้วและตรวจอะไรไปบ้าง

2.5 ไทม์ไลน์คืนนั้นจาก relic5

ขั้นที่ 1 — อ่านบันทึกที่ relic5 เก็บไว้

issue กับ PR บอกแค่ผลลัพธ์ แต่ลำดับเหตุการณ์จริงอยู่ในประวัติของ session ซึ่ง relic5 เก็บไว้ capture ด้านล่างคือเวลา UTC กับข้อความจริงที่มันดึงมา (เวลาไทยคือ UTC+7)

```
cat ../2026-11-08_herdr-wt-lifecycle/captures/01-relic5-timeline.txt
```

```
$ cat ../2026-11-08_herdr-wt-lifecycle/captures/01-relic5-timeline.txt
# What happened, from relic5 (m5, 3 Nov 2026; UTC, local = UTC+7). Read 8 Nov 2026.

14:28:57Z  Nat in host pane wDW:p1 (Laris Labs Oracle, laris-team main, session 51fdd0de): …
… /herdr-wt demo-hello-world
14:29:15Z  issue laris-co/laris-team#129 created (gh): "demo-hello-world: smoke-test the …
… /herdr-wt flow with a hello-world script"
14:29:29+07 is 'born' in the tombstone = worktree cut 21:29:29 local
14:30:45Z  2nd worktree (right-click herdr worktree, worktree-calm-field-2991, session …
… 59a9b212): Nat "investigte fan on white machine"
14:31:44Z  PR #130 opened by the demo agent (session 0b82f9e8, pane wFA:p1), head c689d8e, …
… body "Closes #129" + Built by block
14:32:29Z  fan pane, after /herdr-rename:
           "Renamed `worktree/calm-field-2991` to `white-fan-surge-laris-team-3nov-tue2026`. …
           … The sidebar now shows
            the new name, and the space label was renamed to match. The pane had no label, so …
            … nothing to change there."
14:33:11Z  fan pane, Nat: "/herdr-pr for demo"
           AI: "There's nothing to sign yet. This branch has no commits and no open PR."
14:33:32Z  demo pane tool call: ~/.claude/skills/herdr-pr/herdr-sign.sh claude-opus-5-5 | gh …
… pr comment 130 --repo laris-co/laris-team --body-file -
14:35:15Z  host receives: "DISSOLVE-READY no-changes — worktree …
… white-fan-surge-laris-team-3nov-tue2026 (wFB:p1) ... please: /herdr-dissolve …
… worktree-calm-field-2991"
14:36:02Z  host receives: "DISSOLVE-READY https://github.com/laris-co/laris-team/pull/130 — …
… please: /herdr-dissolve demo-hello-world-laris-team-issue129-3nov-tue2026 (clean, pushed, 1 …
… commit c689d8e, PR open not merged)"
14:36:10Z  demo agent: "I ran `/herdr-dissolve` in self mode, so the work is saved but this …
… worktree still exists.
            An agent can't remove the worktree it is running in, so the host pane does that …
            … last step."
14:37:00Z  host: "I removed the fan-check worktree. The demo worktree stays until PR #130 …
… merges."
           "The dissolve check stops on two things: the PR isn't merged yet; the worktree is …
           … locked on purpose,
            so it can't be cleaned up by accident."
15:01:53Z  Nat: "merge PR #130 and dissolve it"
15:04:27Z  PR #130 merged; issue #129 closed 15:04:28Z
15:05:09Z  host: "PR #130 is merged and the demo worktree is dissolved. ... Kept: the branch; …
… the conversation 0b82f9e8;
  … 7 more lines in captures/15-relic5-wt-3nov.txt
(exit 0 · 0.0 s)
```

รูปที่ 11: ไทม์ไลน์ 3 พ.ย. จาก relic5 ตั้งแต่ 14:28:57Z ถึง 15:05:09Z

เวลาไทย 21:28 Nat พิมพ์ `/herdr-wt demo-hello-world` 21:29 issue #129 เกิด 21:31 agent เปิด PR #130 **สามนาทีจากคำสั่งเดียว** ได้ครบทั้ง issue, worktree ที่ล็อกไว้, agent และ PR

ขั้นที่ 2 — ดูส่วนที่ไม่เป๊ะ

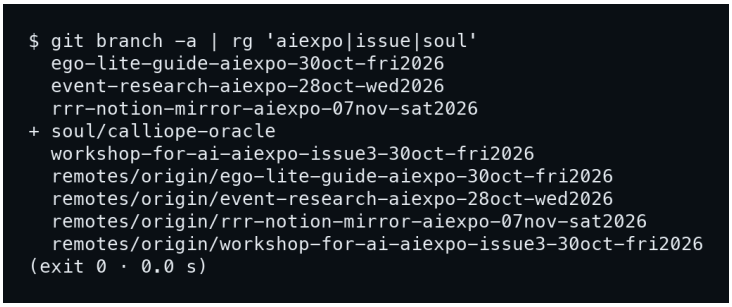
คืนนั้นมี worktree ที่สองสำหรับตรวจพัดลมบน white `/herdr-rename` ตอน 21:32 ทำงานใน pane นั้น ส่วน branch ของ demo ไม่เคยถูกเปลี่ยนชื่อ 21:33 `/herdr-pr for demo` ก็ถูกพิมพ์ใน pane เดียวกัน ได้คำตอบว่ายังไม่มีอะไรให้เซ็น ลายเซ็นจริงโพสต์โดย agent ของ demo เอง 21:35 `/herdr-dissolve` ปิด worktree พัดลมได้ (ลบตอน 21:37) แต่ **worktree ของ demo ยังอยู่** เพราะ PR ยังไม่รวมและ worktree ถูกล็อก รายละเอียดอยู่ในบทที่ 4 และ 5

2.6 branch ใน repo นี้ที่ตามกฎ

ขั้นที่ 1 — ลองดู branch จริงของ aiexpo-oracle

กฎชื่อไม่ใช่ทฤษฎี repo ที่คุณกำลังอ่านอยู่ก็ใช้มัน ลองกรอง branch ที่มี `aiexpo`, `issue` หรือ `soul`

```
git branch -a | rg 'aiexpo|issue|soul'
```



รูปที่ 12: รายการ branch ใน aiexpo-oracle ที่ชื่อลงท้ายด้วยวัน

branch จาก `/herdr-wt` ลงท้ายด้วยวัน เช่น `30oct-fri2026`

```
event-research-aiexpo-28oct-wed2026
ego-lite-guide-aiexpo-30oct-fri2026
workshop-for-ai-aiexpo-issue3-30oct-fri2026
rrr-notion-mirror-aiexpo-07nov-sat2026
```

owner คือ `aiexpo` และตัวที่ผูกกับ issue จะมี `-issue3` แทรกก่อนวัน ส่วน `soul/calliope-oracle` ไม่ลงท้ายวัน branch ที่ไม่ลงท้ายวันมักไม่ได้เปิดด้วยสกิลนี้

งานเปิดแล้ว แต่ agent ที่เคยทำงานไว้แล้วหายไปล่ะ บทหน้าคือ `/herdr-bring` ที่ดึงมันกลับมา

บทที่ 3: /herdr-bring — ดึง agent ที่หายไปกลับมา

agent ตัวหนึ่งทำงานอยู่ใน tmux ของ maw พอย้ายมาใช้ herdr ก็ต้องมีคนพามันมา โดยไม่ให้ประวัติคุยหาย และไม่ให้มีสองตัวแย่งกันตอบ งานนี้คือหน้าที่ของ /herdr-bring บทนี้เริ่มจากว่าสกิลทำอะไรเป็นขั้น ดู `maw locate` วันนี้ ดูกฎของมัน แล้วจบด้วยเรื่องจริงวันที่ 3 พ.ย. ที่สกิลตัดสินเองไม่ได้

3.1 bring ทำอะไรบ้าง

สกิลนี้เป็นไฟล์ SKILL.md ไฟล์เดียว ในเครื่อง m5 อยู่ที่ `~/.claude/skills/herdr-bring/` เปิดดูหัวข้อก็เห็นลำดับงานทั้งหมด

ขั้นที่ 1 — ดูหัวข้อของสกิล

พิมพ์ `rg` ให้มันไล่หัวข้อ `#` ของไฟล์ แล้วอ่านเรียงตามเลขบรรทัด เห็นเลยว่า **สกิลแบ่งเป็น Step 1 ถึง Step 4** ตามลำดับ list, resolve, bring one, several

```
rg '^#+ ' ~/.claude/skills/herdr-bring/SKILL.md
```

```
$ rg '^#+ ' ~/.claude/skills/herdr-bring/SKILL.md
7:# /herdr-bring — land a maw handle into herdr
22:## Before anything
30:## Socket targeting
42:## Step 1 — list candidates
59:## Step 2 — resolve
70:## Step 3 — bring one
102:# Newest transcript for that repo. Encoded path: leading / becomes -, then / and . become -.
142:## Step 4 — several
154:## After landing
166:## Conflict = exit 2 = ASK, never decide
196:## Moving a body to ANOTHER herdr session (herdr to herdr, not tmux)
223:## Traps
254:## What this does NOT do
263:## Related
(exit 0 · 0.0 s)
```

รูปที่ 13: หัวข้อของ SKILL.md ของ herdr-bring ตั้งแต่ Step 1 ถึง Traps

หัวข้อที่น่าสนใจมีสามอัน คือ `Step 3 — bring one` (บรรทัด 70) ที่ลงมือจริง, `Conflict = exit 2 = ASK, never decide` (บรรทัด 166) ที่บอกให้ถามคนเมื่อชน และ `Moving a body to ANOTHER herdr session` (บรรทัด 196) ที่ใช้กับเรื่อง 3 พ.ย. ข้างล่าง ส่วนท้ายไฟล์มี `What this does NOT do` ไว้บอกขอบเขต

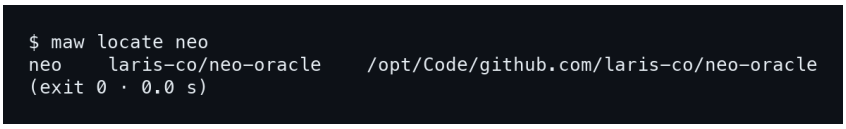
3.2 maw locate วันนี้

Step 2 ของสกิลให้ถาม `maw locate` ก่อนทุกครั้งว่า handle นี้คือ repo ไหน อยู่ที่ path อะไร ถ้าไม่ได้ path ก็รายงานแล้วข้ามไป ห้ามเดา

ขั้นที่ 1 — locate สองชื่อ

รันสองคำสั่ง ตัวแรกคือ neo ตัวที่สองคือ aiexpo ซึ่งเป็น oracle ของรีโปที่ถือหนังสือเล่มนี้ ผลออกมาเป็นบรรทัดเดียวคั่นด้วย tab คือ handle, org/repo, path

```
maw locate neo
maw locate aiexpo
```



รูปที่ 14: maw locate neo ตอบ laris-co/neo-oracle พร้อม path บน m5



รูปที่ 15: maw locate aiexpo ตอบ Caeruleus-Lab/aiexpo-oracle พร้อม path บน m5

ทั้งสองได้ path ครบ exit 0 ทั้งคู่ จำไว้ว่า **วันนี้ `maw locate neo` ตอบ path ได้** เพราะเรื่องใน 3.4 เกิดตอนที่มันยังตอบไม่ได้ และนั่นคือจุดที่สกิลต้องหยุดถามคน

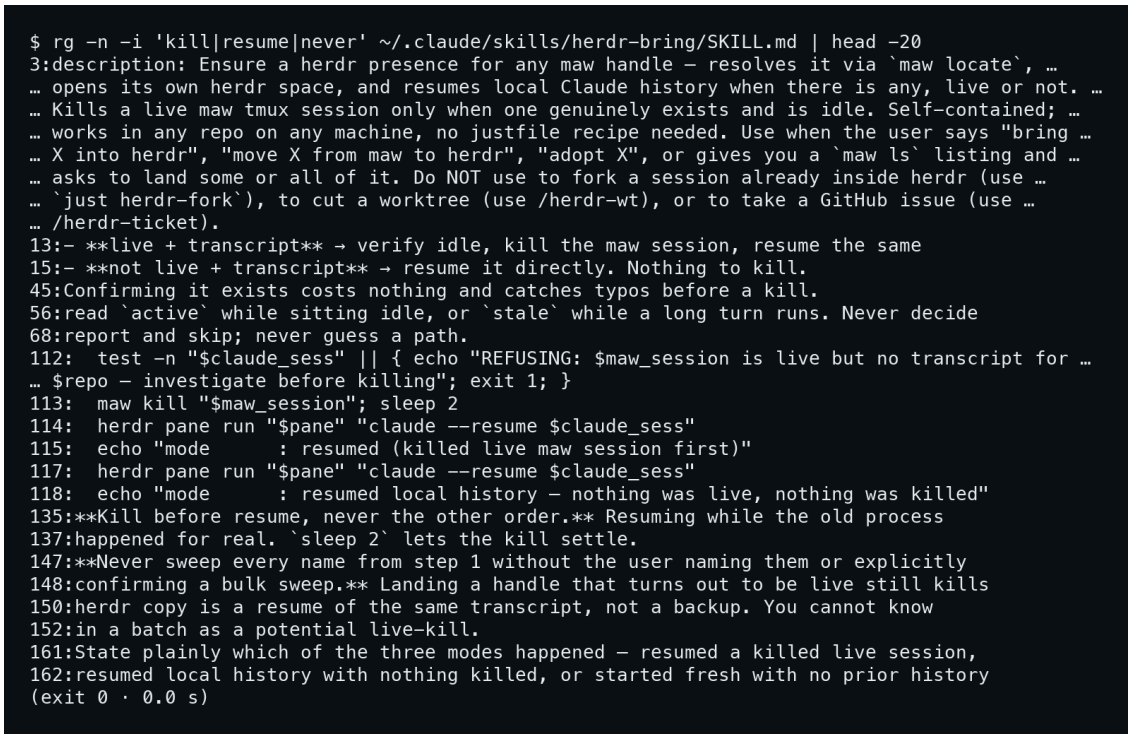
3.3 กฎของมัน

กฎสำคัญของ bring ซ่อนอยู่ในคำว่า kill, resume และ never ลอง `rg` สามคำนี้ในไฟล์เดียวกัน

ขั้นที่ 1 — ค้นกฎ

สั่ง `rg -n -i` ด้วยสามคำ แล้วตัดที่ 20 บรรทัดแรก จากผลลัพธ์ **ฆ่า tmux ของ maw เฉพาะตัวที่ live จริงและ idle** นอกนั้น resume ตรง ๆ

```
rg -n -i 'kill|resume|never' ~/.claude/skills/herdr-bring/SKILL.md \
  | head -20
```



รูปที่ 16: บรรทัดที่มีคำ kill resume never ใน SKILL.md ของ herdr-bring

อ่านตามบรรทัดได้สามโหมด บรรทัด 13 คือ live ที่มี transcript ให้ตรวจว่า idle, kill แล้ว resume บรรทัด 15 คือไม่ live แต่มี transcript ให้ resume ทันที ไม่มีอะไรให้ kill และมีโหมดที่สามคือไม่เคยมีประวัติ ก็เริ่มใหม่ (บรรทัด 161 ให้รายงานโหมดที่เกิดขึ้นเป็นภาษาตรง ๆ)

ขั้นที่ 2 — กฎที่กันของเสีย

สามบรรทัดในผลลัพธ์ตอบคำถามว่าทำไมสกิลระวังขนาดนั้น บรรทัด 112 ถ้า live แต่ไม่มี transcript ให้ `REFUSING` ไม่ kill บรรทัด 135 เขียนว่า **kill ก่อน resume ไม่เคยกลับลำดับ** เพราะ resume ตอนตัวเก่ายังรันอยู่ จะได้สองตัวเขียน transcript เดียวกัน และบรรทัด 147 ห้ามกวาดทุกชื่อจาก Step 1 ถ้าผู้ใช้ไม่ได้ระบุชื่อเอง

ที่สกิลเขียนเข้มก็เพราะ resume ของ herdr ไม่ใช่ backup ของ session เดิม มันคือ transcript เดียวกันที่ถูกเปิดอีกครั้ง

3.4 เรื่องจริง 3 พ.ย.: bring neo

สกิลเขียนมาเพื่อกรณีง่าย handle เดียว repo เดียว แต่ Neo ในวันที่ 3 พ.ย. 2026 ไม่ได้ง่าย เรื่องนี้มาจาก relic5 ของ session 51fdd0de (Laris Labs Oracle, pane wDW:p1) ไม่ได้รันซ้ำเพื่อหนังสือ เวลาเป็นเวลากรุงเทพ

ขั้นที่ 1 — สกิลยอมรับว่าตัดสินเองไม่ได้

22:06:40 Nat พิมพ์ `/herdr-bring bring neo` แล้วพิมพ์ซ้ำอีกครั้งตอน 22:09:59 agent ไปหา body ของ neo ทั้ง m5 และ white แล้วตอบตอน 22:10:13 ว่า `maw locate neo` ไม่คืน path และ Neo มีหลาย body อยู่แล้ว **เมื่อสกิลชนเรื่องที่ไม่ใช่กรณีง่าย agent ถามคน ไม่เดาเอง**

```
22:10:13  agent: "`/herdr-bring` can't settle this one by itself. …"
```

```
$ cat ../2026-11-08_herdr-bring-promote/captures/relic5-session-51fdd0de.txt
relic5, session 51fdd0de-67e7-498c-94a1-3a2e0afaf4f4 ("LarisCo host body in laris-team", m5, …
… herdr session laris-co pane wDW:p1)
Times are Bangkok (UTC+7), 3 Nov 2026. Agent replies shortened; tool calls quoted from their …
… own one-line descriptions.

22:06:40  Nat: /herdr-bring bring neo
22:09:59  Nat: /herdr-bring bring neo            (typed again)
22:10:06  tool: "Resolve neo and find every live neo body on m5 and white"
22:10:13  agent: "`/herdr-bring` can't settle this one by itself. `maw locate neo` returns no …
… repo path, and Neo already has several bodies."
22:10:16  AskUserQuestion "bring neo":
          "Neo already runs in 3 places: its Discord bot body on white …
          … (rd-team/wt/neo-oracle, pane wZ:p1, session laris-co),
           and on m5 a separate herdr session 'neo' with 5 agents (2 on neo-oracle main: …
           … w1Y:p1, w1Z:p2; 3 in herdr worktrees,
           none with the bot)."
          options: 1 Fold m5 'neo' session in (herdr-unpromote session neo into m5 laris-co …
          … ... same conversations)
                   2 Move bot body to m5
                   3 Open main neo here
          -> Nat chose 1 (agent at 22:10:58: "as you chose")
22:10:26  tool: herdr-unpromote --session neo --to laris-co --dry
22:10:33  agent: "Session `neo` holds more than Neo: 11 agents, including the relic5, …
… relic-extract, relic4-scan and messenger-archive agents."
22:10:45  tool: check which agents hold a bot token (names only) -> all token=0
22:10:50  agent: "None of them holds a bot token, so losing `--channels` on resume changes …
… nothing. Running the fold now."
22:10:50  tool: herdr-unpromote --session neo --to laris-co
22:11:20  agent: "`neo-oracle` main (`w1Y`) couldn't close because it has linked worktree …
… spaces, so the tool stopped there."
22:11:33  tool: herdr-move --workspace w1Z / w24 --from neo --to laris-co
22:11:43  tool: maw herdr ticket open /opt/Code/github.com/laris-co/neo-oracle --session …
… laris-co --session-id 08f33511-...
22:11:50  tool: herdr agent rename wFK:p2 neo-oracle ; wFK:p1 builder-neo
22:12:56  tool: back up session neo to …
… ~/.herdr/tombstones/sessions/2026-11-03/neo-session.tgz, herdr session stop neo, delete neo
          screen: "stopped session neo / deleted session neo / link removed / session neo gone"
          screen (agent list): wFK:p1 idle builder-neo 840b12d9 · wFK:p2 idle neo-oracle …
          … 08f33511 · wFM wFN wFP neo worktrees ·
  … 39 more lines in captures/24-relic5-neo-3nov.txt
(exit 0 · 0.0 s)
```

รูปที่ 17: บันทึก relic5 ของ session 51fdd0de ช่วง 22:06 ถึง 22:19 ของเรื่อง bring neo

ขั้นที่ 2 — สามตัวเลือก Nat เลือกข้อ 1

ตอน 22:10:16 agent เปิดคำถามสามข้อ แล้ว Nat เลือกข้อ 1 คือพับ session `neo` ของ m5 เข้ามาใน `laris-co` **agent ซ้อมด้วย `--dry` ก่อน** พอเห็นว่า `token=0` ทุกตัว ตอน 22:10:50 จึงสั่งพับจริง

ขั้นที่ 3 — herdr-move หยุดกลางทาง

ตอน 22:11:20 เครื่องมือหยุด เพราะ `neo-oracle` main ปิดไม่ได้ **space ที่มี worktree space เกาะอยู่ ปิดตรง ๆ ไม่ได้** agent ต้องทำมือต่อ resume สองตัวด้วย id ของมัน คือ neo-oracle main ที่ `08f33511` กับ builder-neo ที่ `840b12d9` แล้วตั้งชื่อ pane ใหม่ พอครบก็สำรอง session `neo` ไว้ที่ `~/.herdr/tombstones/sessions/2026-11-03/neo-session.tgz` แล้วหยุดและลบ session นั้นตอน 22:12:56

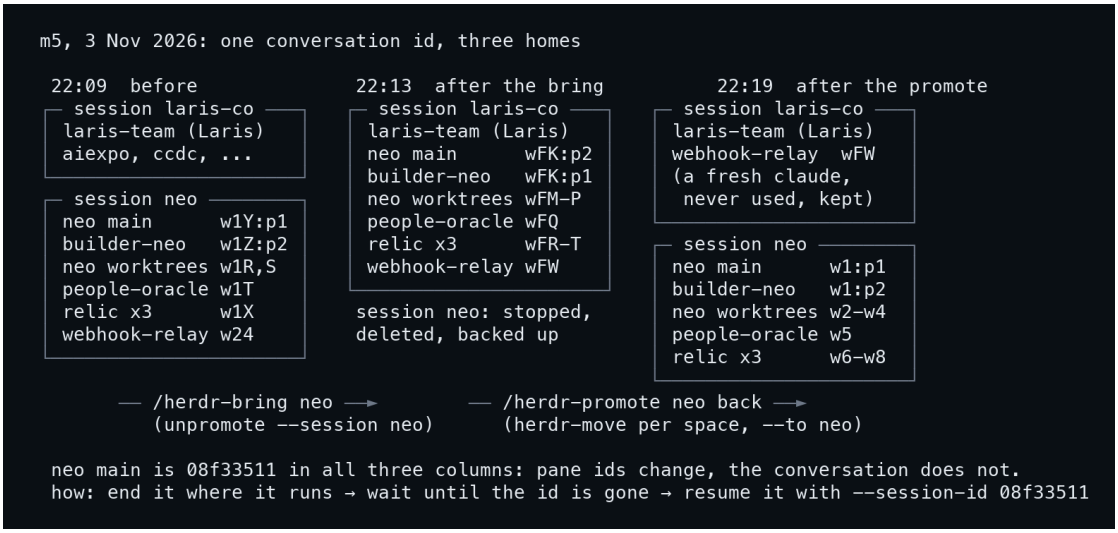
ขั้นที่ 4 — พับกลับด้วย promote

22:14:36 Nat สั่ง `/herdr-promote neo back session` แล้ว **agent ซ้อมด้วย `--dry` ก่อน** แล้ว `herdr-move` ทีละ space โดยเอา worktree ไปก่อน ตอน 22:17:54 ได้เจ็ดตัวกลับ `neo` แต่ neo main ชนปัญหาเดิมอีก agent ต้อง resume สองตัวด้วย id เดิม แล้วพบตอน 22:18:11 ว่าสองคำสั่งรายงาน pane เดียวกัน (`w1:p1`) คำสั่งที่สองอาจพิมพ์ลงช่องพิมพ์ของตัวแรก มันเลยล้างคำสั่งนั้นโดยไม่กดส่ง แล้วแยก pane ใหม่เปิดตัวที่สอง

```
22:19:05  agent: "Neo is back in its own herdr session, `neo`, …"
```

ขั้นที่ 5 — ดูภาพรวมสามช่วงเวลา

การ์ด D2 วางสามคอลัมน์เทียบกัน ก่อน (22:09) หลังพับ (22:13) และหลังพับกลับ (22:19) **pane id เปลี่ยนทุกคอลัมน์ แต่ conversation id `08f33511` ไม่เปลี่ยน** ท่าที่ได้ผลคือจบตัวเก่าที่ที่มันรัน แล้ว resume ด้วย `--session-id` บทถัดไปไปที่การลงชื่อบนงาน คือ /herdr-pr



รูปที่ 18: การ์ดแสดง session laris-co กับ neo ในสามช่วงเวลา ก่อนพับ หลังพับ และหลังพับกลับ

บทที่ 4: /herdr-pr — เซ็นชื่อบน PR

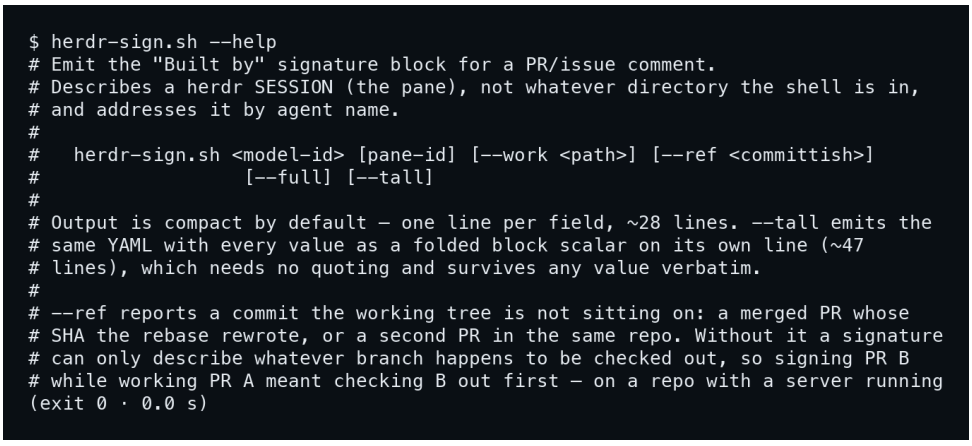
PR ที่เปิดจาก worktree หลายตัวเข้า repo เดียว reviewer มองแค่ diff ก็ไม่รู้ว่า session ไหนเขียน สกิล /herdr-pr แก้เรื่องนี้ด้วยการแปะบล็อก “Built by” ลงใน PR ว่าใครสร้าง ใช้โมเดลอะไร อยู่ worktree ไหน และคุยกลับถึง session นั้นได้ยังไง บทนี้ดูสคริปต์ที่ทำบล็อกนี้ แล้วดูของจริงสามชิ้น

4.1 สคริปต์และ flag ของมัน

ขั้นที่ 1 — ขอ help จากสคริปต์

สกิลนี้ทำงานผ่านสคริปต์ตัวเดียวคือ `herdr-sign.sh` พิมพ์ `--help` ก่อนก็จะเห็นว่ามันรับอะไรบ้าง **สคริปต์อธิบาย herdr session (pane) ไม่ใช่โฟลเดอร์ที่ shell ยืนอยู่**

```
herdr-sign.sh --help
```



รูปที่ 19: help ของ herdr-sign.sh แสดง usage และคำอธิบาย --ref กับ --tall

บรรทัด usage เขียนว่า

```
herdr-sign.sh <model-id> [pane-id] [--work <path>] [--ref <committish>]
              [--full] [--tall]
```

ต้องใส่ model id เสมอ ส่วน pane id ใส่หรือไม่ใส่ก็ได้ `--tall` พิมพ์ YAML ชุดเดิมแต่ทุกค่าเป็น folded block scalar บรรทัดละค่า ยาวราว 47 บรรทัด (แบบปกติราว 28 บรรทัด) ข้อดีคือไม่ต้องใส่เครื่องหมายคำพูด และค่าไม่เพี้ยน ส่วน `--ref` ไว้รายงาน

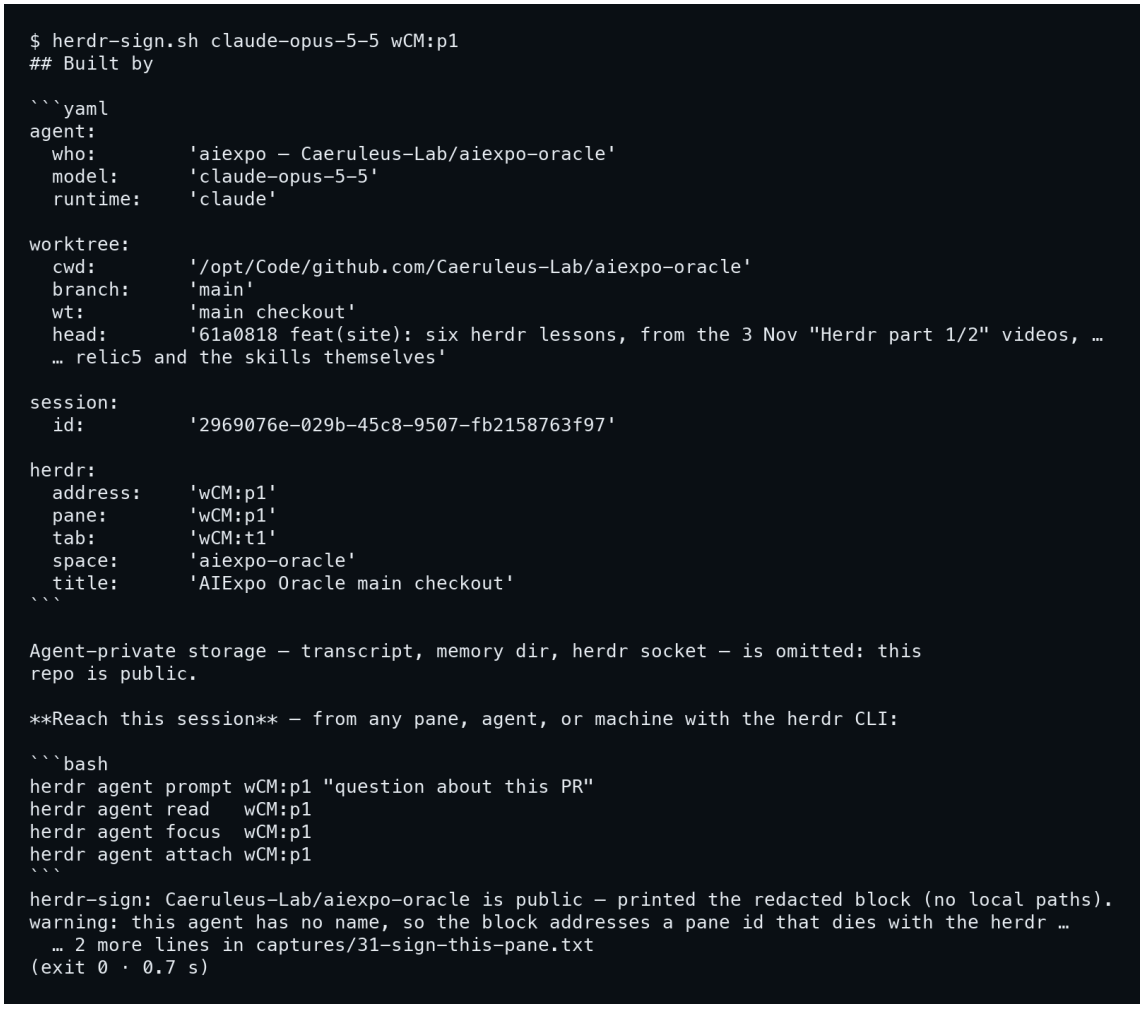
commit ที่ working tree ไม่ได้ยืนอยู่ เช่น PR ที่ merge แล้ว SHA ถูก rebase เขียนใหม่ หรือ PR ที่สองใน repo เดียวกัน ไม่ต้อง checkout อีกตัวมาก่อนเซ็น

4.2 เซ็น pane นี้เอง: repo สาธารณะ ช่องส่วนตัวหายไป

ขั้นที่ 1 — รันเซ็น pane ที่เขียนหนังสือเล่มนี้

ใส่ model id แล้วตามด้วย pane id ของเรา ตอนเขียนเล่มนี้ pane คือ `wCM:p1` **repo นี้เป็น public สคริปต์เลยพิมพ์บล็อกแบบ redacted**

```
herdr-sign.sh claude-opus-5-5 wCM:p1
```



รูปที่ 20: บล็อก Built by ของ pane wCM:p1 ไม่มี transcript และ socket พร้อมคำเตือนท้ายผลลัพธ์

ในบล็อกไม่มี `transcript` `memory` และ `socket` สคริปต์บอกเหตุผลเองว่าถูกตัดเพราะ repo นี้เป็น public ท้ายผลยังมีคำเตือนว่า agent นี้ไม่มีชื่อ address จึงเป็น pane id ที่

ตายไปพร้อม herdr server แก้ด้วย `herdr agent rename` ตามที่สคริปต์แนะนำแล้วเซ็นใหม่

4.3 บล็อกเต็มบน PR #130 (repo ส่วนตัว)

ขั้นที่ 1 — ดึงคอมเมนต์ที่เซ็นไว้เมื่อ 3 พ.ย. มาดู

PR #130 ของ laris-co/laris-team เป็น repo ที่บล็อกไม่ถูก redact agent ของ worktree เซ็นไว้ด้วย `herdr-sign.sh | gh pr comment 130` ลองดึงกลับมาด้วย gh แล้วกรองเอาส่วนที่ขึ้นต้นด้วย Built by **ใน repo ส่วนตัว บล็อกมีช่อง transcript, memory และ socket เพิ่มมา**

```
gh pr view 130 --repo laris-co/laris-team --comments \
  | rg -A32 'Built by' | head -40
```



รูปที่ 21: บล็อก Built by เต็มบน PR 130 มี cwd ของ worktree, transcript, memory และ socket

ลองเทียบกับบล็อกของ pane นี้ ช่องที่เพิ่มขึ้นมาคือ `transcript` `memory` และ `socket`

transcript คือไฟล์ที่เก็บเหตุผลของ session ใครสงสัยว่า agent ตัดสินใจแบบนี้ทำไม ก็เปิดไฟล์นี้อ่านได้ ช่อง `herdr.address` เป็น `demo-hello-world-laris` ซึ่งเป็นชื่อ agent ไม่ใช่ pane id แปลว่าตั้งชื่อไว้แล้ว คุยกลับได้แม้ pane id เปลี่ยน ส่วนบรรทัด head คือ `c689d8e demos/hello-world: smoke-test script for the /herdr-wt flow`

4.4 PR #7 ของ repo นี้ และเวลาของมัน

ขั้นที่ 1 — ดูคอมเมนต์ที่เซ็นบน PR #7

PR #7 ใน repo นี้ถูกเซ็นด้วย /herdr-pr โดย agent ฝั่งขวา (pane `wCM:p2`) เพราะ repo เป็น public คอมเมนต์เลยเป็นแบบ redacted เหมือนบล็อกของ pane นี้ **บล็อกบอกชัดว่า agent อยู่ worktree ไหน และ commit ไหนที่เป็นหัวของงาน**

```
gh pr view 7 --comments | rg -A30 'Built by' | head -36
```



รูปที่ 22: บล็อก Built by บน PR 7 ของ aiexpo-oracle ไม่มี transcript และ socket

ในบล็อกมี `branch: '5de0408'` ซึ่ง branch เป็น SHA ไม่ใช่ชื่อ branch ส่วน head คือ `5de0408 feat(rrr): rrr_notion.py mirrors one /rrr retro into the Notion Retros database` และ address ที่คุยกลับได้คือ `aiexpo-right`

ขั้นที่ 2 — ถาม gh เรื่องเวลา

อยากรู้ว่า PR นี้ใช้เวลานานแค่ไหน ถามเป็น JSON ตรง ๆ ก็ได้ **เวลาทั้งหมดที่ gh ตอบเป็น UTC (ตัว Z) ต้องบวก 7 ชั่วโมงเป็นเวลาไทย**

```
gh pr view 7 --json createdAt,mergedAt,headRefName,title
```

```
$ gh pr view 7 --json createdAt,mergedAt,headRefName,title
{"createdAt":"2026-11-07T03:24:46Z","headRefName":"rrr-notion-mirror-aiexpo-07nov-sat2026","me …
… rgedAt":"2026-11-07T03:25:26Z","number":7,"title":"rrr: mirror a Claude Code retro into the …
… Notion Retros database"}
(exit 0 · 0.7 s)
```

รูปที่ 23: JSON ของ PR 7 มี createdAt, mergedAt, headRefName และ title

เปิดตอน `2026-11-07T03:24:46Z` merge ตอน `2026-11-07T03:25:26Z` ห่างกัน 40 วินาที branch คือ `rrr-notion-mirror-aiexpo-07nov-sat2026`

4.5 กับดัก: รันใน pane ที่ทำงานนั้นจริง

3 พ.ย. 21:33 `/herdr-pr for demo` ถูกพิมพ์ใน pane ของ fan check ซึ่งตอบว่า “nothing to sign yet” **สกิลเซ็นให้ session ที่รันมัน** ไม่ใช่งานที่ชื่อบอก ลายเซ็นจริงบน PR #130 มาจาก agent ของ demo เอง จะเซ็นจากที่อื่นก็ส่ง pane id เป็นอาร์กิวเมนต์ที่สอง และใช้ `--ref` ถ้า PR ไม่ใช่ commit ที่ working tree ยืนอยู่

บทที่ 5: /herdr-dissolve — ปิดงานโดยไม่ทิ้งอะไร

งานที่เปิดด้วย /herdr-wt ต้องมีวันปิด และตอนปิดมักมีของค้างใน worktree บทนี้ดูว่า /herdr-dissolve ปิดยังไงโดยไม่ทำของหาย ตามหลัก Nothing is Deleted

5.1 โหมดทั้งห้า และการถามก่อนทำ

ขั้นที่ 1 — ดูว่าเรียกได้กี่แบบ

ใช้ `rg` หาชื่อโหมดใน SKILL.md สกิลเดียวรับตั้งแต่ worktree เดียวไปจนถึงทั้งเครื่อง **ต่างกันแค่ขอบเขตที่ส่งให้**

```
rg -n -e '--sweep' -e '--all-but-me' -e '--sync' -e 'self' \
  ~/.claude/skills/herdr-dissolve/SKILL.md | sed 1d | head -10
```

```
$ rg -n -e '--sweep' -e '--all-but-me' -e '--sync' -e 'self' …
… ~/.claude/skills/herdr-dissolve/SKILL.md | sed 1d | head -10
4:argument-hint: "[<wt|pane> ...] | --all-but-me | --sweep [--host <ssh>]... | --sync [--all] …
… | self"
24:| `/herdr-dissolve --all-but-me` | every pane and worktree of the CURRENT repo, except the …
… pane running this | "close all panes and wt, just only 1 here" |
25:| `/herdr-dissolve --sweep [--host H]` | every repo on the machine (`ghq list -p`) | …
… "clean up the sidebar", "too many ✓" |
26:| `/herdr-dissolve --sync [--all]` | checkouts that are only behind | "everything is ↓" |
27:| `/herdr-dissolve self` | the worktree you are running in | an agent finishing its own …
… task |
29:`--sweep` runs sync's plan too, as the old `/herdr-clean-up-sync` did. Every mode goes
56:repo on the machine** (1278 on m5, 2026-10-08). Only `--sweep` may run unscoped.
59:# named / --all-but-me: this repo only
63:# --sweep
68:For `--all-but-me`, also list the panes on the main checkout, since `fleet audit` covers
(exit 0 · 0.0 s)
```

รูปที่ 24: ผล rg ในไฟล์ SKILL.md ของ herdr-dissolve เห็นตารางโหมดและบรรทัดที่พูดถึง self

ตารางในสกิลสรุปไว้ห้าแบบ ได้แก่ ชื่อ wt หรือ pane (named), `--all-but-me`, `--sweep`, `--sync` และ `self`

`self` คือ agent ปิดงานตัวเอง ส่วน `--sweep` คือของเดิมชื่อ /herdr-clean-up-sync ที่ถูกรวมเข้ามา บรรทัดที่ 56 ของสกิลบอกว่า repo บน m5 มี 1278 ตัว (นับเมื่อ 2026-10-08) ก็เลยมีแค่ `--sweep` ที่อนุญาตให้รันแบบไม่จำกัด repo

ขั้นที่ 2 — หาจุดที่สกิลหยุดถาม

ทุกโหมดเดินสี่จังหวะเหมือนกัน คือ survey, plan, ask, act ลองหาคำว่า ask กับ confirm ในไฟล์เดียวกัน เพื่อดูว่าสกิลหยุดตรงไหน

```
rg -n -i 'ask|confirm|never close' \
  ~/.claude/skills/herdr-dissolve/SKILL.md | head -16
```



รูปที่ 25: ผล rg หาคำ ask และ confirm ใน SKILL.md เห็นหัวข้อ Show the plan, then ask

หัวข้อ `## 3. Show the plan, then ask` อยู่บรรทัดที่ 115 และ description ก็บอกว่าสกิลแสดงแผนและถามก่อนปิดหรือถอดอะไรทั้งนั้น พอถึงจังหวะนั้นมันเปิด AskUserQuestion แบบเลือกได้หลายข้อ หนึ่งตัวเลือกต่อหนึ่งกลุ่มที่มีแถว และไม่ปิด pane ที่ตัวเองรันอยู่ด้วย

> สกิลนี้ไม่ลบอะไรก่อนที่คนจะเห็นแผน

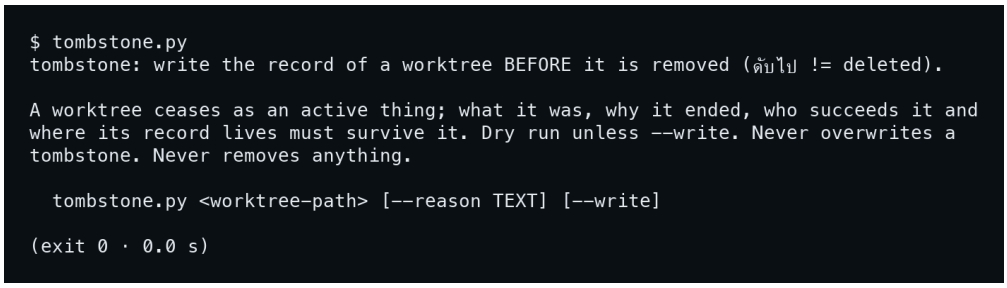
อีกจุดที่น่าจำคือบรรทัดที่ 94 ของสกิล ที่บอกว่า **เครื่องหมายติ๊กถูกในแถบข้างไม่ได้แปลว่างานเสร็จ** ติ๊กถูกแปลว่า agent จบ turn ของมัน ถ้า agent ไปจบที่คำถามค้างอยู่ สกิลก็ไม่นับว่าพร้อมปิด

5.2 tombstone.py: ซ้อมก่อน เขียนจริงทีหลัง

ขั้นที่ 1 — อ่าน help ของ tombstone.py

ก่อน worktree หายไป สกิลเรียก tombstone.py เขียนบันทึกไว้ก่อน ลองสั่งโดยไม่ใส่อาร์กิวเมนต์ เพื่ออ่านว่ามันสัญญาอะไรไว้

```
tombstone.py
```



รูปที่ 26: help ของ tombstone.py ที่บอกว่า dry run unless --write

ข้อความตรงนี้สำคัญสามประโยค

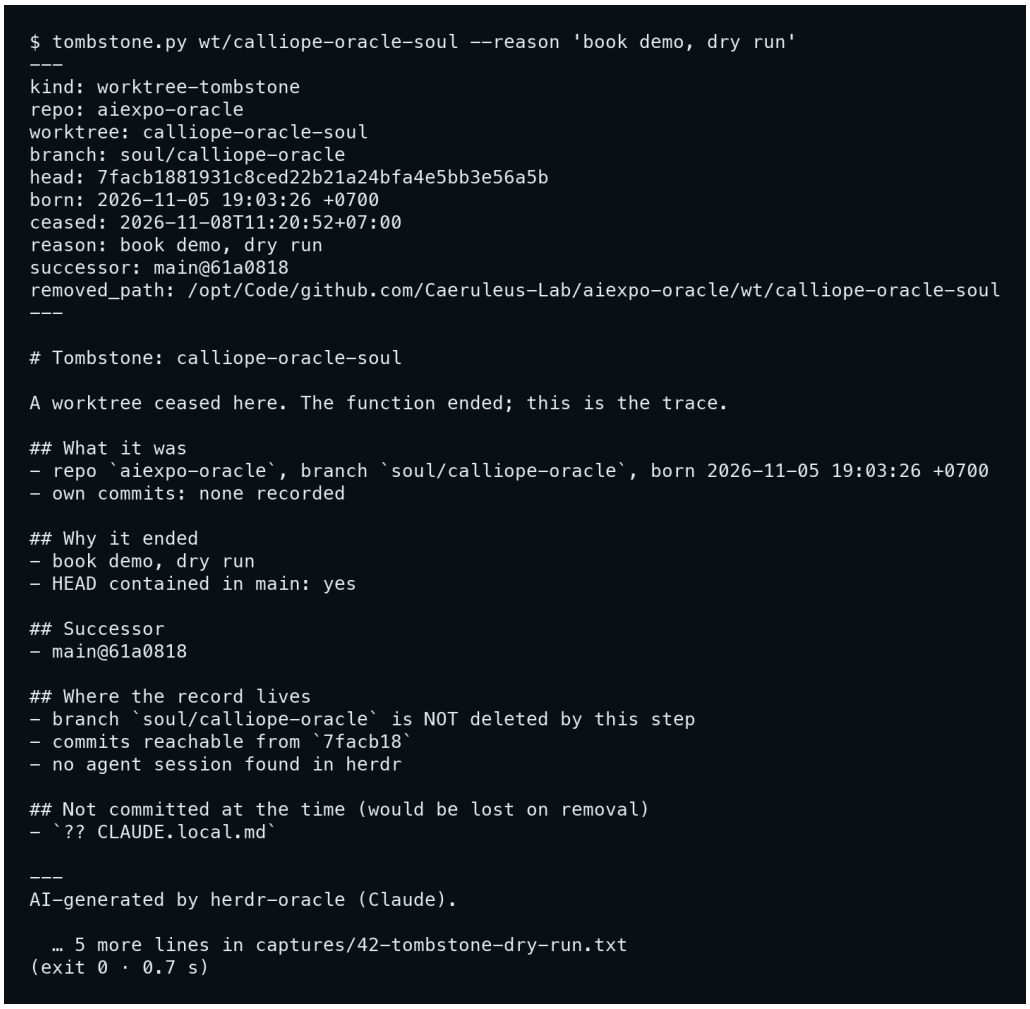
```
Dry run unless --write. Never overwrites a tombstone.
Never removes anything.
  tombstone.py <worktree-path> [--reason TEXT] [--write]
```

ไม่ใส่ `--write` คือแค่ซ้อม สคริปต์ไม่เขียนทับของเดิม และไม่ลบอะไรเลย การลบ worktree เป็นงานของ herdr ในจังหวะถัดไป ไม่ใช่ของสคริปต์นี้

ขั้นที่ 2 — ลองกับ worktree จริงของ repo นี้

ใน repo นี้มี worktree ชื่อ calliope-oracle-soul อยู่ในโฟลเดอร์ `wt/` เลยใช้มันเป็นตัวอย่าง สั่งแบบไม่ใส่ `--write` เพราะฉะนั้นไม่มีอะไรถูกเขียนและไม่มีอะไรถูกลบ

```
tombstone.py wt/calliope-oracle-soul --reason 'book demo, dry run'
```



รูปที่ 27: ผล dry run ของ tombstone.py บน wt/calliope-oracle-soul พร้อม BLOCKER ท้ายผล

ผลออกมาเป็น Markdown เต็มใบ มี frontmatter บอก branch (`soul/calliope-oracle`) วันเกิด `2026-11-05 19:03:26 +0700` เวลาที่ดับ ผู้สืบต่อ (`main@61a0818`) และ path ที่จะถูกถอด ส่วนท้ายสุดคือสามบรรทัดที่ควรอ่านก่อนอย่างอื่น

```
destination: …/ψ/memory/tombstones/2026-11-08_calliope-oracle-soul.md
BLOCKER: 1 uncommitted/untracked file(s) would be LOST on removal
dry run — add --write to save it
```

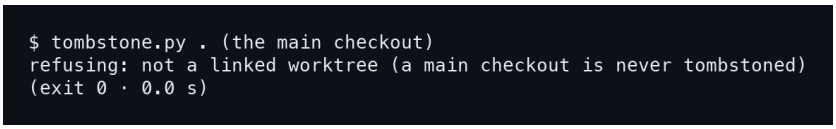
BLOCKER บอกว่ามีไฟล์ที่ยังไม่ commit ในกรณีนี้คือ `CLAUDE.local.md` ซึ่งจะหายถ้าถอด worktree นี่คือประโยชน์ของการซ้อม คือเห็นของที่จะหายก่อนลงมือ ส่วนหัวข้อ “Where the record lives” ก็ระบุว่า branch ไม่ถูกลบ และ commit ยังเข้าถึงได้จาก `7facb18`

5.3 ปฏิเสธ main checkout

ขั้นที่ 1 — สั่งกับ main checkout

main checkout ไม่ใช่ worktree ที่ควรถูกเก็บเป็นอนุสรณ์ ลองสั่งที่โฟลเดอร์ปัจจุบัน ซึ่งเป็น main ของ repo เพื่อดูว่าสคริปต์กันไว้ไหม

```
tombstone.py .
```



รูปที่ 28: tombstone.py ปฏิเสธ main checkout ด้วยข้อความ refusing

```
refusing: not a linked worktree (a main checkout is never tombstoned)
```

มันตอบสั้นๆ ว่าไม่ทำ และ **main checkout ไม่เคยถูก tombstone** สคริปต์รู้ได้เพราะ main เป็นรายการแรกของลิสต์ ไม่ใช่ linked worktree ส่วนรายละเอียดว่าตรวจยังไง ไปอ่านต่อในบทที่ 6

5.4 สอง tombstone จริง

ขั้นที่ 1 — worktree issue3 ของวันนี้

วันที่ 8 พ.ย. worktree `workshop-for-ai-aiexpo-issue3-30oct-fri2026` จบงานหลัง PR #7 merge herdr ล็อกมันไว้ตั้งแต่ 30 ต.ค. agent ที่ main checkout เลยต้องปลดล็อก แล้วถอดจากข้างนอก ก่อนถอดมีของค้างสามอย่าง สองอย่างแรก copy ไป main คือ og/ src กับ deck-shots ที่ยังไม่ track (PNG 17 ไฟล์ กับ deck_report.json) ส่วน books/ dist ขนาด 61 MB ที่ถูก ignore ก็ tar เก็บไว้ที่ `~/.herdr/tombstones` เปิดบันทึกที่ได้ออกมาอ่าน

```
cat ψ/memory/tombstones/\
2026-11-08_workshop-for-ai-aiexpo-issue3-30oct-fri2026.md
```

```
$ cat ψ/memory/tombstones/2026-11-08_workshop-for-ai-aiexpo-issue3-30oct-fri2026.md
---
kind: worktree-tombstone
repo: aiexpo-oracle
worktree: workshop-for-ai-aiexpo-issue3-30oct-fri2026
branch: rrr-notion-mirror-aiexpo-07nov-sat2026
head: 5de040848c16eade76135eb7e15547c977cb33ca
born: 2026-11-07 10:20:36 +0700
ceased: 2026-11-08T09:48:09+07:00
reason: merged PR #7 (f582893); right pane done. deck-shots copied to main …
… ψ/writing/slides/deck-shots-2026-10-30, og/src copied to main, books/dist tarred to …
… ~/.herdr/tombstones/aiexpo-oracle/2026-11-08_wt-issue3_books-dist.tgz
successor: https://github.com/Caeruleus-Lab/aiexpo-oracle/pull/7
removed_path: …
… /opt/Code/github.com/Caeruleus-Lab/aiexpo-oracle/wt/workshop-for-ai-aiexpo-issue3-30oct-fri2 …
… 026
---

# Tombstone: workshop-for-ai-aiexpo-issue3-30oct-fri2026

A worktree ceased here. The function ended; this is the trace.

## What it was
- repo `aiexpo-oracle`, branch `rrr-notion-mirror-aiexpo-07nov-sat2026`, born 2026-11-07 …
… 10:20:36 +0700
- own commits:
  - `5de0408 2026-11-07 feat(rrr): rrr_notion.py mirrors one /rrr retro into the Notion …
  … Retros database`

## Why it ended
- merged PR #7 (f582893); right pane done. deck-shots copied to main …
… ψ/writing/slides/deck-shots-2026-10-30, og/src copied to main, books/dist tarred to …
… ~/.herdr/tombstones/aiexpo-oracle/2026-11-08_wt-issue3_books-dist.tgz
- HEAD contained in main: yes

## Successor
- https://github.com/Caeruleus-Lab/aiexpo-oracle/pull/7

## Where the record lives
- branch `rrr-notion-mirror-aiexpo-07nov-sat2026` is NOT deleted by this step
  … 5 more lines in captures/43-tombstone-issue3.txt
(exit 0 · 0.0 s)
```

รูปที่ 29: tombstone ของ worktree issue3 ใน repo นี้ เห็น reason ที่ยาว และ successor เป็น PR #7

ช่อง reason คือที่เก็บเรื่องราว มันเขียนว่า merged PR #7 (f582893) พร้อมบอกว่าอะไรถูก copy และอะไรถูก tar ไปไว้ที่ไหน ช่อง successor ชี้ไปที่ PR #7 ส่วน own commits มีหนึ่งรายการ (`5de0408` rrr_notion.py) และบรรทัด “HEAD contained in main: yes” ยืนยันว่า commit นี้อยู่ใน main แล้ว branch `rrr-notion-mirror-aiexpo-07nov-sat2026` ยังอยู่ ไม่ถูกลบ

ขั้นที่ 2 — demo เมื่อ 3 พ.ย.

อีกใบมาจากงาน demo ใน laris-team (บทที่ 2) เก็บไว้ที่ `~/.herdr/tombstones/laris-team` ตามแบบเดียวกัน เปิดดูเทียบกัน

```
cat ~/.herdr/tombstones/laris-team/\
2026-11-03_demo-hello-world-laris-team-issue129-3nov-tue2026.md
```

```
$ cat ~/.herdr/tombstones/laris-team/2026-11-03_demo-hello-world-laris-team-issue129-3nov-tue2 …
… 026.md
---
kind: worktree-tombstone
repo: laris-team
worktree: demo-hello-world-laris-team-issue129-3nov-tue2026
branch: demo-hello-world-laris-team-issue129-3nov-tue2026
head: c689d8e2c2508c5c087f040be72730035abec5b4
born: 2026-11-03 21:29:29 +0700
ceased: 2026-11-03T22:04:34+07:00
reason: merged PR #130 (closes #129)
successor: https://github.com/laris-co/laris-team/pull/130
removed_path: …
… /opt/Code/github.com/laris-co/laris-team/wt/demo-hello-world-laris-team-issue129-3nov-tue2026
---

# Tombstone: demo-hello-world-laris-team-issue129-3nov-tue2026

A worktree ceased here. The function ended; this is the trace.

## What it was
- repo `laris-team`, branch `demo-hello-world-laris-team-issue129-3nov-tue2026`, born …
… 2026-11-03 21:29:29 +0700
- own commits:
  - `c689d8e 2026-11-03 demos/hello-world: smoke-test script for the /herdr-wt flow`

## Why it ended
- merged PR #130 (closes #129)
- HEAD contained in main: yes

## Successor
- https://github.com/laris-co/laris-team/pull/130

## Where the record lives
- branch `demo-hello-world-laris-team-issue129-3nov-tue2026` is NOT deleted by this step
- commits reachable from `c689d8e`
- claude session `0b82f9e8-b25a-4279-b1b9-b2b0ab4db321`: …
… `/Users/beta/.claude/projects/-opt-Code-github-com-laris-co-laris-team-wt-demo-hello-world-l …
… aris-team-issue129-3nov-tue2026/0b82f9e8-b25a-4279-b1b9-b2b0ab4db321.jsonl`

  … 2 more lines in captures/44-tombstone-demo129.txt
(exit 0 · 0.0 s)
```

รูปที่ 30: tombstone ของ demo-hello-world เมื่อ 3 พ.ย. เห็น born, ceased และ claude session

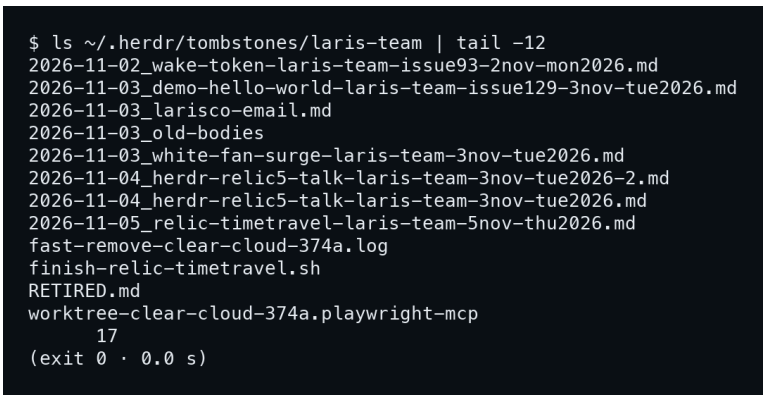
ใบนี้เกิด `2026-11-03 21:29:29 +0700` และดับ `2026-11-03T22:04:34+07:00` เหตุผลคือ “merged PR #130 (closes #129)” ตรงกับที่ relic5 บันทึกไว้ว่า merge ตอน 22:05 สิ่งที่ใบนี้มีและใบ issue3 ไม่มีคือบรรทัด claude session (`0b82f9e8-b25a-4279-b1b9-b2b0ab4db321`) พร้อม path ของไฟล์ jsonl **จึงย้อนกลับไปอ่านบทสนทนาของ agent ตัวนั้นได้** ส่วนใบ issue3 เขียนว่า “no agent session found in herdr”

5.5 ชั้นวาง tombstone

ขั้นที่ 1 — ดูชั้นวางของ laris-team

tombstone มีสองชั้นวาง ชั้นใน home อยู่ที่ `~/.herdr/tombstones/<repo>/` ส่วนชั้นของ repo อยู่ที่ `ψ/memory/tombstones/` ลอง `ls` ดูแค่ 12 ชื่อท้ายสุดของ laris-team

```
ls ~/.herdr/tombstones/laris-team | tail -12
```



รูปที่ 31: รายชื่อไฟล์ใน ~/.herdr/tombstones/laris-team เรียงตามวันที่

ชื่อไฟล์ขึ้นต้นด้วยวันที่ จึงเรียงตามเวลาอยู่ในตัว มีใบของ 2026-11-03 สี่ใบอยู่ใกล้กัน รวมถึง demo-hello-world ที่อ่านไปเมื่อกี้ และ white-fan-surge ซึ่งเป็น worktree ที่ถูกถอดตอน 21:37 **บนชั้นเดียวกันยังมีของที่ไม่ใช่ tombstone ปนอยู่** เช่น log, สคริปต์ finish-relic-timetravel.sh และ RETIRED.md

> Tombstone คือร่องรอย หน้าที่ของ worktree จบลง แต่เรื่องราวของมันยังอยู่

สรุปบทนี้ไว้ในประโยคเดียวนั้นพอ ส่วนในบทหน้าจะแกะไส้ในของสองสคริปต์ คือ herdr-sign.sh ที่ตัดสิน ว่าจะพิมพ์ฟิลด์ไหนตามความเป็นส่วนตัวของ repo และ tombstone.py ที่เพิ่งใช้ไป

บทที่ 6: แกะไส้ในสองสคริปต์

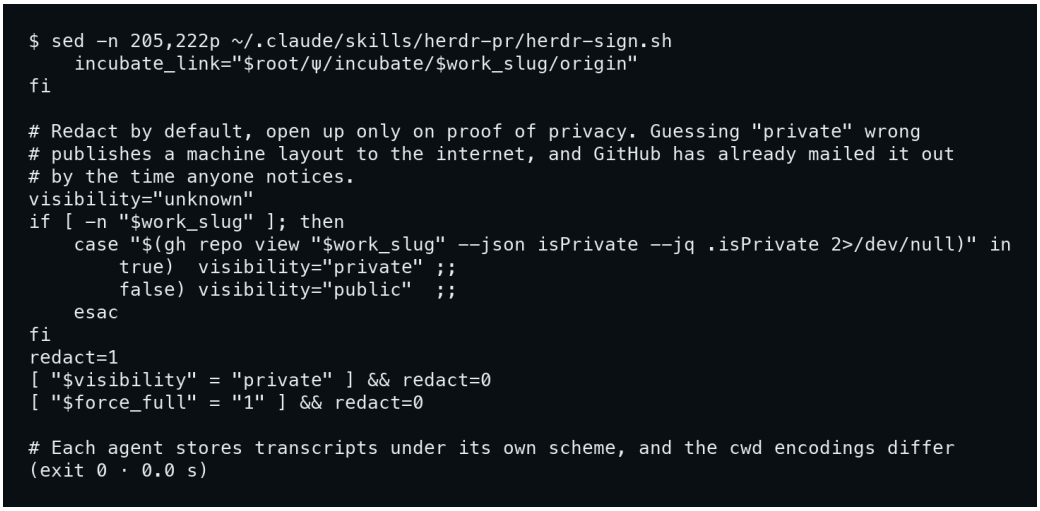
สี่บทที่ผ่านมาเราใช้สกิลจากข้างนอก พิมพ์คำสั่งแล้วดูผล บทนี้เปิดฝาดูข้างใน สองสคริปต์ที่ทำงานหนักที่สุดคือ `herdr-sign.sh` ของ /herdr-pr กับ `tombstone.py` ของ /herdr-dissolve เราจะอ่านทีละบรรทัดจากชิ้นที่ตัดมาจริง ไม่เดาและไม่เติม ถ้าอยากเห็นทุกบรรทัดของไฟล์ ให้เปิดภาคผนวก `09-code.md` ซึ่งเก็บโค้ดเต็มทั้งสี่สกิลไว้

6.1 herdr-sign.sh ตัดสินจากความเป็น public ว่าจะพิมพ์แค่ไหน

ขั้นที่ 1 — อ่านบล็อกที่ถาม GitHub ว่า repo เป็น private ไหม

เปิดไฟล์ช่วงบรรทัด 205 ถึง 222 ด้วย `sed -n` ช่วงนี้คือจุดที่สคริปต์ตัดสินว่าบล็อกที่ตัวเองกำลังจะพิมพ์ จะเปิดข้อมูลบนเครื่องหรือไม่

```
sed -n 205,222p ~/.claude/skills/herdr-pr/herdr-sign.sh
```



รูปที่ 32: บรรทัด 205 ถึง 222 ของ herdr-sign.sh ตัวแปร visibility และ redact

ตัวแปรเริ่มต้นคือ `visibility="unknown"` สคริปต์เรียก `gh repo view "$work_slug" --json isPrivate --jq .isPrivate` แล้ว `case` ดูคำตอบ มีสองทางเท่านั้นที่เปลี่ยนค่า คือ `true` ได้ `private` กับ `false` ได้ `public` ถ้า `gh` ล้มหรือไม่มี `$work_slug` ค่าก็ค้างที่ `unknown` **ค่าเริ่มต้นคือปิด ต้องมีหลักฐานว่าเป็น private ถึงจะเปิด** ตรงนี้คอมเมนต์ของสคริปต์เองก็บอกเหตุผลไว้ (เดาว่า private ผิด = เอาโครงเครื่องไปประกาศบนอินเทอร์เน็ต)

ขั้นที่ 2 — ดูสองบรรทัดที่ตั้ง redact

อ่านต่อสองบรรทัดท้าย ๆ ของชิ้นเดียวกัน

```
redact=1
[ "$visibility" = "private" ] && redact=0
[ "$force_full" = "1" ] && redact=0
```

`redact=1` ตั้งไว้ก่อนเสมอ แล้วมีแค่สองเงื่อนไขที่ลดเป็น 0 เงื่อนไขแรกคือ visibility เท่ากับ `private` พอดี `unknown` กับ `public` ไม่ผ่าน เงื่อนไขที่สองคือ `--full` ซึ่งตั้ง `force_full` ให้คนที่รู้ดีกว่าสคริปต์ สั่งทับได้เอง บทที่ 4 เราเห็นผลของสองบรรทัดนี้มาแล้ว repo นี้ public บล็อกของ pane นี้เลยออกมาแบบ redact

6.2 ฟิลด์ที่มีแต่ private repo เท่านั้นที่ได้

ขั้นที่ 1 — อ่านว่าฟิลด์ไหนถูกกั้นไว้หลัง redact

ไปที่บรรทัด 266 ถึง 285 ตรงนี้สคริปต์ประกอบสองก้อนข้อความ `sess_private` กับ `herdr_private` ทั้งสองเริ่มเป็นสตริงว่าง

```
sed -n 266,285p ~/.claude/skills/herdr-pr/herdr-sign.sh
```

```
$ sed -n 266,285p ~/.claude/skills/herdr-pr/herdr-sign.sh
# a line added to the full block silently never reached the public one.
# Groups answer the four questions a reviewer actually asks in order — who built
# it, from which tree, against which repo, and how do I reach them.

sess_private=""
herdr_private=""
if [ "$redact" = "0" ]; then
    sess_private="
  transcript: >-
    ${transcript/#$HOME/\~}
  memory: >-
    ${memory}"
    herdr_private="
  socket: >-
    ${HERDR_SOCKET_PATH:-~/.config/herdr/herdr.sock (default; not run from a pane)}"
fi

# The incubated group only exists when the PR's repo is not the session's own:
# a different repo, not just a different worktree of the same one.
incubated_group=""
(exit 0 · 0.0 s)
```

รูปที่ 33: บรรทัด 266 ถึง 285 ของ herdr-sign.sh ก้อน sess_private และ herdr_private

พอ `redact` เป็น 0 ก้อนแรกถึงได้ `transcript` กับ `memory` ก้อนที่สองได้ `socket` (ค่าจาก `HERDR_SOCKET_PATH` หรือค่า default) ถ้า `redact` ยังเป็น 1 สองก้อนนี้ว่างต่อไป บล็อกสาธารณะเลยไม่มีทั้งสามฟิลด์ **ฟิลด์ที่บอกว่าไฟล์อยู่ที่ไหนในเครื่อง มีเฉพาะบล็อก**

ของ repo ที่พิสูจน์แล้วว่าเป็นส่วนตัว สังเกตว่า transcript ถูกแปลง `$HOME` เป็น `~` ก่อนพิมพ์ด้วย

คอมเมนต์ด้านบนของชิ้นนี้ยังเล่าที่มาของโครงสร้างนี้ด้วย แต่ก่อนมีบล็อก full กับบล็อก public แยกกันสองก้อน บรรทัดที่เติมลง full เงียบ ๆ ไม่เคยไปถึง public ตอนนี้ทั้งสองบล็อกมาจากแม่แบบเดียว ต่างกันแค่สองตัวแปรนี้ว่างหรือไม่ว่าง อ่านโค้ดทั้งไฟล์ได้ที่ `09-code.md` ส่วน herdr-pr

6.3 tombstone.py หา worktree แล้วปฏิเสธ main checkout

ขั้นที่ 1 — อ่านวิธีหา worktree จาก porcelain

ฝั่ง /herdr-dissolve เปิด `tombstone.py` บรรทัด 18 ถึง 40 ซึ่งเป็นต้นของ `main()`

```
sed -n 18,40p ~/.claude/skills/herdr-dissolve/tombstone.py
```

```
$ sed -n 18,40p ~/.claude/skills/herdr-dissolve/tombstone.py
def main():
    args = [a for a in sys.argv[1:]]
    write = "--write" in args
    reason = ""
    if "--reason" in args:
        i = args.index("--reason"); reason = args[i + 1]; del args[i:i + 2]
    args = [a for a in args if a != "--write"]
    if not args: print(__doc__); sys.exit(2)
    path = os.path.realpath(args[0])

    wts, cur = [], {}
    for line in run(["git", "-C", path, "worktree", "list", "--porcelain"]).split("\n"):
        if not line:
            if cur: wts.append(cur); cur = {}
            continue
        k, _, v = line.partition(" "); cur[k] = v or True
    if cur: wts.append(cur)
    me = next((w for w in wts if os.path.realpath(w["worktree"]) == path), None)
    if not me or me is wts[0]:
        print("refusing: not a linked worktree (a main checkout is never tombstoned)"); …
        … sys.exit(2)
    main_path, main_sha = wts[0]["worktree"], wts[0]["HEAD"]
    repo = os.path.basename(main_path)
    branch = (me.get("branch") or "").replace("refs/heads/", "") or "(detached)"
(exit 0 · 0.0 s)
```

รูปที่ 34: บรรทัด 18 ถึง 40 ของ tombstone.py ลูปอ่าน git worktree list --porcelain

สคริปต์รับ path ตัวแรกแล้วทำ **`os.path.realpath` ก่อน** เพื่อให้ symlink ไม่หลอกการเทียบ จากนั้นรัน `git -C path worktree list --porcelain` ผลลัพธ์แยกเป็นก้อนด้วยบรรทัดว่าง ลูปเก็บแต่ละก้อนเป็น dict โดยตัดบรรทัดที่ช่องว่างแรก ได้ key เช่น `worktree`, `HEAD`, `branch` แล้วหา `me` คือก้อนที่ realpath ของ `worktree` ตรงกับ path ที่ได้รับ

ขั้นที่ 2 — ดูเงื่อนไขปฏิเสธ

บรรทัดที่สำคัญที่สุดของชิ้นนี้คือ

```
    if not me or me is wts[0]:
        print("refusing: not a linked worktree (a main checkout …")
```

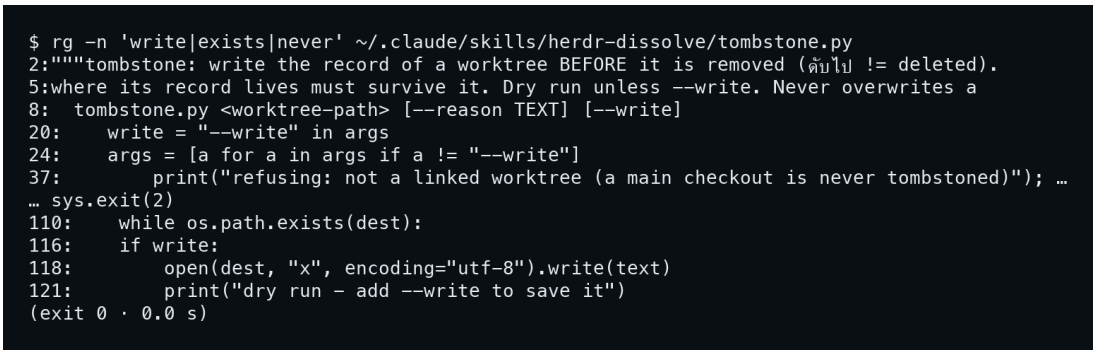
`git worktree list` พิมพ์ main checkout เป็นรายการแรกเสมอ สคริปต์เลยใช้ `me is wts[0]` เป็นตัวตัดสิน ไม่ต้องเดาจากชื่อโฟลเดอร์ และถ้าหา `me` ไม่เจอ (path ไม่ใช่ worktree เลย) ก็ปฏิเสธด้วยข้อความเดียวกัน **ทั้งสองกรณีออกด้วย `sys.exit(2)` ก่อนจะทำอะไรต่อ** หลังผ่านด่านนี้ถึงอ่าน `main_path`, `main_sha` ชื่อ repo จาก `basename` และชื่อ branch ที่ตัด `refs/heads/` ออก ถ้า detached ก็ขึ้น `(detached)` บทที่ 5 เรารันด่านนี้กับ main checkout จริงแล้วเห็นข้อความนี้ออกมา ส่วน exit 0 ที่เห็นในภาพนั้นเป็นของ wrapper ที่เรียกสคริปต์ ไม่ใช่ค่าที่ `sys.exit(2)` คืนมา

6.4 ไม่เขียนทับ ไม่ลบ เขียนเมื่อสั่ง --write เท่านั้น

ขั้นที่ 1 — ค้นคำที่เกี่ยวกับการเขียนทั้งไฟล์

ไฟล์ยาวเกินจะอ่านทีละบรรทัดในหน้าเดียว เราเลยใช้ `rg` ดึงเฉพาะบรรทัดที่มีคำว่า `write`, `exists` หรือ `never`

```
rg -n 'write|exists|never' ~/.claude/skills/herdr-dissolve/tombstone.py
```



รูปที่ 35: ผล rg ในแต่ละบรรทัดของ tombstone.py ที่เกี่ยวกับ write exists never

เรียงตามเลขบรรทัดแล้วเห็น **สามจุดที่คุมการเขียน** บรรทัด 5 ใน docstring บอกว่า `Dry run unless --write. Never overwrites` บรรทัด 20 กับ 24 เก็บ `write = "--write" in args` แล้วกรอง flag ออกจาก args บรรทัด 110 มี `while os.path.exists(dest):` ซึ่งหาชื่อไฟล์ใหม่เมื่อชื่อเดิมถูกใช้แล้ว

ขั้นที่ 2 — อ่านท้ายไฟล์ที่เขียนจริง

สองบรรทัดสุดท้ายในผลคือหัวใจ

```
118:        open(dest, "x", encoding="utf-8").write(text)
121:        print("dry run - add --write to save it")
```

`if write:` ที่บรรทัด 116 คือประตูเดียว ถ้าไม่ใส่ `--write` สคริปต์พิมพ์ข้อความ dry run แล้วจบ ถ้าใส่ ก็เปิดไฟล์ด้วยโหมด `"x"` ซึ่ง Python จะล้มทันทีถ้าไฟล์มีอยู่แล้ว **สองชั้นกันเขียนทับ คือลูป `os.path.exists` หาชื่อใหม่ กับโหมด `"x"` ที่ปฏิเสธไฟล์เดิม** rg ชุดแรกค้นแค่ `write|exists|never` เลยยังพิสูจน์ไม่ได้ว่าไม่มีคำสั่งลบ ขั้นต่อไปค้นคำพวกนั้นตรง ๆ

ขั้นที่ 3 — ค้นหาคำสั่งลบทั้งไฟล์

ค้นคำที่เกี่ยวกับการลบไฟล์ ได้แก่ `remove`, `rmtree`, `unlink`, `os.rm` และ `shutil`

```
rg -n 'remove|rmtree|unlink|os\.rm|shutil' \
  ~/.claude/skills/herdr-dissolve/tombstone.py
```

```
$ rg -n 'remove|rmtree|unlink|os\.rm|shutil' ~/.claude/skills/herdr-dissolve/tombstone.py
2:"""tombstone: write the record of a worktree BEFORE it is removed (ดับไป != deleted).
6:tombstone. Never removes anything.
88:        f"removed_path: {path}", "---", ""]
(exit 0 · 0.0 s)
```

รูปที่ 36: ผล rg หาคำสั่งลบใน tombstone.py ตรงแค่ docstring กับบรรทัด removed_path

มีแค่สามบรรทัดที่ตรง บรรทัด 2 กับ 6 เป็น docstring ที่เขียนว่า `Never removes anything` และบรรทัด 88 เป็นข้อความ `removed_path:` ที่เขียนลงบันทึก **ไม่มีคำสั่งลบจริงสักบรรทัด** สคริปต์นี้ทำหน้าที่จดบันทึก ส่วนการถอด worktree จริงเป็นงานของสกิลที่เรียกมัน หลังจากแสดงแผนและถามคนก่อน

ตอนนี้เราเห็นทั้งสองสคริปต์ว่าระวังเรื่องเดียวกัน คือถ้าไม่แน่ใจ ให้ทำน้อยที่สุด บทหน้ารวมสิ่งที่พลาดจริงตลอดงานนี้ พร้อมการ์ดสรุปใบเดียว

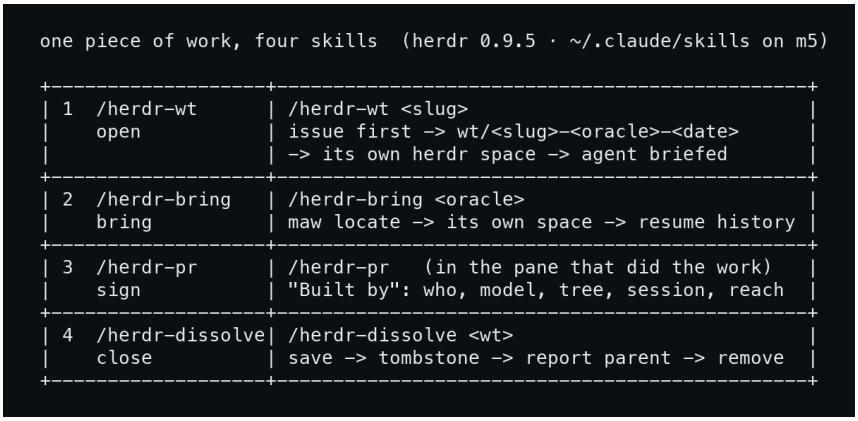
บทที่ 7: กับดักที่เจอจริง และการ์ดสรุป

บทนี้เริ่มด้วยการ์ดสรุปแผ่นเดียว แล้วไปดูทางที่พลาดกัน ทุกข้อเกิดขึ้นจริง ที่ 3 พ.ย. ตอนเดโม `demo-hello-world` กับ `bring neo` และที่ 8 พ.ย. ตอนปิด worktree ของ issue #3 แต่ละข้อเขียนเป็นสามท่อน: เกิดอะไร, สาเหตุ, วิธีแก้

การ์ดสรุปหนึ่งใบ

ขั้นที่ 1 — ดูสี่ skill ในตารางเดียว

เปิดการ์ดนี้เมื่อลืมว่าต้องพิมพ์อะไร แต่ละแถวคือหนึ่ง skill: คำสั่ง, ท่อนที่ มันทำ และเงื่อนไขสำคัญ **ข้อที่ต้องจำที่สุดคือ /herdr-pr ต้องพิมพ์ใน pane ที่ทำงานเอง** ซึ่งก็คือกับดักข้อแรกของบทนี้



รูปที่ 37: การ์ดสรุปสี่ skill: wt เปิด, bring ดึงมา, pr เซ็น, dissolve ปิด

ภาพนี้ให้เห็นลำดับ open, bring, sign, close ในหน้าเดียว

ขั้นที่ 2 — ที่เหลือถาม herdr --help

เรื่องอื่นนอกสี่ skill เช่น `herdr worktree`, `herdr pane`, `herdr agent`, `herdr machine` ให้ถาม help ของ herdr เอง **ทุกกลุ่มคำสั่งมี subcommand ของตัวเอง** และหน้านี้คือแผนที่ของทั้งหมด

พิมพ์ `herdr --help` แล้วบรรทัดแรกที่เห็นคือ:

```
herdr — terminal workspace manager for AI coding agents
```



รูปที่ 38: หน้า herdr --help แสดง usage คำสั่งหลักและ options

ท้ายหน้ามีบรรทัดบอก agent ว่าถ้างานคือควบคุม pane ให้รัน `herdr --skill` เพื่อพิมพ์ไฟล์ skill ออกมาอ่าน

กับดักหกข้อ

ขั้นที่ 1 — /herdr-pr พิมพ์ผิด pane

ที่ 3 พ.ย. เวลา 21:33 Nat พิมพ์ `/herdr-pr for demo` ลงใน pane ของงาน fan check บน white ไม่ใช่ pane ที่ทำเดโม pane นั้นตอบว่า “nothing to sign yet” **สาเหตุคือ / herdr-pr เซ็นจากสิ่งที่ pane ตัวเองทำ** ใน pane ที่ไม่ได้ทำ อะไรก็ไม่มีอะไรให้เซ็น

(บันทึก relic5 ของคืนนั้นอยู่ในบทที่ 2 หัวข้อ 2.5)

วิธีแก้คือให้ agent ที่ทำงานเซ็นเอง ซึ่งก็เกิดขึ้นจริง agent ของเดโมโพสต์ ลายเซ็นบน PR #130 ด้วยตัวเอง:

```
herdr-sign.sh | gh pr comment 130
```

ขั้นที่ 2 — /herdr-rename วิ่งใน worktree ที่สอง

เวลา 21:32 `/herdr-rename` ทำงานใน worktree ที่สอง (งาน fan check บน white) ไม่ใช่ worktree ของเดโม ชื่อ `worktree/calm-field-2991` เลยกลายเป็น `white-fan-surge-laris-team-3nov-tue2026` ส่วน branch ของเดโมยังใช้ชื่อเดิม **สิ่งที่บันทึกบอกได้มีแค่นี้ คือ rename แก้ branch ของ worktree ที่มันรันอยู่** ถ้าอยากรู้ว่า pane ไหนจะถูกแก้ ให้ดู `git branch --show-current` ใน pane นั้น ก่อนสั่ง

ขั้นที่ 3 — worktree ที่ล็อกและมี PR ยังไม่ merge จะไม่หายไป

หลัง `/herdr-dissolve` ตอน 21:35 worktree ของ fan check ถูกลบตอน 21:37 แต่ worktree ของเดโมยังอยู่ agent ของเดโมรัน dissolve แบบ self mode ซึ่งลบ worktree ที่ตัวเองรันอยู่ไม่ได้ ขั้นสุดท้ายเลยเป็นของ pane host และ pane host หยุดที่ **สองข้อ คือ PR ยังไม่ merge กับ worktree ถูกล็อกโดยตั้งใจ**

วิธีแก้คือ merge ก่อนแล้วค่อยปิด ตอน 22:01 Nat สั่ง “merge PR #130 and dissolve it” 22:05 merge เป็น `04b87c5` issue #129 ปิด เขียน tombstone แล้ว และลบ space wFA ออก นับจากเปิด issue ถึง merge ใช้เวลา 35 นาที

```
gh pr view 130 --repo laris-co/laris-team \
  --json number,mergedAt,mergeCommit,headRefName
```

```
$ gh pr view 130 --repo laris-co/laris-team --json number,mergedAt,mergeCommit,headRefName
{"headRefName":"demo-hello-world-laris-team-issue129-3nov-tue2026","mergeCommit":{"oid":"04b87 …
… c5c78b3a97f84f76277a13dc1a85a932826"},"mergedAt":"2026-11-03T15:04:27Z","number":130}
(exit 0 · 0.6 s)
```

รูปที่ 39: gh pr view 130 แสดง mergedAt 2026-11-03T15:04:27Z และ merge commit 04b87c5

ขั้นที่ 4 — bring ไม่เจอ repo path

เวลา 22:06:40 และ 22:09:59 Nat พิมพ์ `/herdr-bring bring neo` สองรอบ agent ตอบว่า `maw locate neo` ไม่คืน repo path และ Neo มีหลาย body อยู่แล้ว มัน เลยไม่เดา แต่ถามกลับ 3 ข้อให้ Nat เลือก **สาเหตุคือ handle เดียวชี้ได้หลาย body** วิธีแก้คือให้คนเลือก ส่วนวันนี้ (8 พ.ย.) `maw locate neo` คืน path ได้ แล้ว เรื่อง bring นี้เกิดคืนเดียวกับเดโม แต่ไม่ได้อยู่ในสายงานของ issue #129

ขั้นที่ 5 — herdr-unpromote หยุดบน space ที่มี linked worktree

หลัง Nat เลือกข้อ 1 (รวม session `neo` ของ m5 เข้า laris-co ด้วย `herdr-unpromote --session neo --to laris-co`) tool นี้ปิด neo-oracle main ไม่ได้ **สาเหตุคือ space นี้มี linked worktree space ผูกอยู่** มันเลยหยุดที่นั่น

(บันทึก relic5 ของเรื่องนี้อยู่ในบทที่ 3)

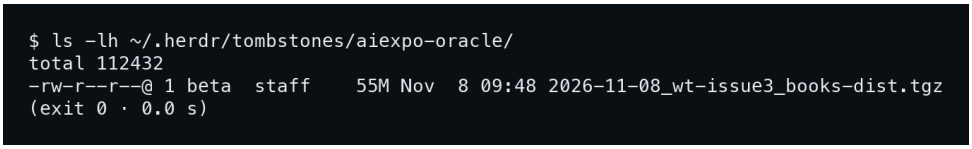
agent แก้ด้วยมือ ด้วยการ resume neo-oracle main (`08f33511`) กับ builder-neo (`840b12d9`) ทีละตัว ต่อมา `/herdr-promote neo back session` ย้าย Neo กลับออกมา ตอนนั้น herdr-move ติดข้อจำกัดเดิมอีกรอบที่ neo main agent จึงปิด space ด้วย `--group` ข้อนี้บอกว่า skill ที่หยุดแล้วบอกเหตุผล ดีกว่าที่ลบของที่ยังมีของผูกอยู่

ขั้นที่ 6 — โฟลเดอร์ที่ถูก ignore หนัก 61 MB ใน worktree

ตอนปิด worktree `workshop-for-ai-aiexpo-issue3-30oct-fri2026` ที่ 8 พ.ย. มี สองอย่างที่ git ไม่เห็น ไฟล์ untracked คือ deck-shots (PNG 17 ไฟล์กับ `deck_report.json`) และโฟลเดอร์ `books/dist` ที่ถูก ignore หนัก 61 MB **สาเหตุคือ `git status` ไม่แสดงของที่ ignore** ลบ worktree แล้วของพวกนี้หายเงียบ ๆ

วิธีแก้คือเก็บก่อนลบ deck-shots ถูกคัดลอกไป main ส่วน `books/dist` ถูก tar ไว้ ไฟล์ tarball ขนาด 55M:

```
ls -lh ~/.herdr/tombstones/aiexpo-oracle/
```



รูปที่ 40: ls -lh แสดง 2026-11-08_wt-issue3_books-dist.tgz ขนาด 55M

worktree นี้ถูก herdr ล็อกไว้ agent ของ main checkout จึงปลดล็อกแล้ว ลบจากข้างนอก จากนั้นปิด pane ขวา wCM:p2 ส่วน branch เก็บไว้ ## ต่อไป

บทต่อไปคือภาคผนวก โค้ดเต็มของสคริปต์ทั้งหมดที่เล่มนี้พูดถึง พิมพ์ครบทุก บรรทัดตามต้นฉบับ ไว้ให้เปิดเทียบกับที่อ่านมา

ภาคผนวก: โค้ดเต็มทั้ง 4 สกิล

ทุกไฟล์ด้านล่างพิมพ์ตามจริงทุกบรรทัด อ่านจาก `~/.claude/skills/` บนเครื่อง m5 วันที่ 2026-11-08 ไม่ย่อ ไม่แก้ให้สวย คัดลอกโฟลเดอร์สกิลไปไว้ที่ `~/.claude/skills/` ของคุณแล้วใช้ได้เลย

สกิล herdr ทั้ง 52 ตัวแบบเต็มอยู่ในบทเรียน herdr ทุกสกิล ฉบับเต็ม: https://aiexpo.buildwithoracle.com/2026-11-08_herdr-skills-full/

herdr-wt/SKILL.md

/herdr-wt: ตัด worktree เปิด issue สร้าง space ปลุก agent แล้ว brief · 861 บรรทัด

```
---
name: herdr-wt
description: Create a git worktree for a distinct piece of work in ANY
repo, placed at <repo>/wt/<slug>-<oracle>-<date> and opened as its own
herdr space (or, with --here, as a split pane in the current tab). The
SPACE always belongs to the repo you run it FROM (your oracle or team):
when the code lives in another repo, the code worktree goes there and
only the agent pane sits in it, so the sidebar row nests under you (Step
0 — missed five times before it became a guard). By default it first
checks the current repo and opens a GitHub issue carrying the FULL task
context, then cuts the worktree, starts an agent and briefs it from that
issue. Self-contained — works without any justfile recipe. Use when
starting work that deserves its own branch and checkout, a parallel
agent's workspace, or when the user says "make a worktree", "start work
on X", "spin up a branch for this", "new wt dir". Also the reference for
WHY the naming is slug-oracle-date and what breaks if that changes. Do
NOT use to move an existing session into herdr (use /herdr-bring), to
fork a session in place (`herdr` skill covers forking), or to spawn an
isolated OMX/Codex worker (use /isolate-worktree).
argument-hint: "<slug> [base-ref] [--here] [--no-issue] [--repo [org]] |
--list"
---

# /herdr-wt — worktree per piece of work

Works in any repo on any machine. Requires `herdr` on PATH and a git
repo; nothing else.

## Step 0 — whose space? Decide before anything (Nat, 2026-10-30)

Two repos can be in play, and they are not the same thing:
```

```
| | what it is | where it comes from |
|---|---|---|
| **home** | the repo you run `/herdr-wt` FROM: your oracle, or your
team's desk repo (`rd-team`, `laris-team`, …) | the cwd of YOUR pane
(`herdr pane get "$HERDR_PANE_ID"`), never `$PWD` — you may have `cd`'d
into the target |
| **target** | the repo the code and the PR live in | the task |

**The herdr space is always a worktree of HOME.** herdr groups the
sidebar by the SPACE's
`worktree.repo_key` and never by a pane's cwd, so a space created with
`herdr worktree create --cwd <target>` lands in the target's group, away
from the team that
owns the work. Home ≠ target → use *Under the oracle* below (code
worktree in the target, space
in home, agent pane split into the code worktree). Home = target → *Run
it* as written.

Missed five times before this became a check (relic5, 2026-10-30):

| date | what | landed under | should have been |
|---|---|---|---|
| 10-06 | floodboy-ops space `wQ` lost its worktree link | flat |
`floodboy-ios` |
| 10-11 | herdr-board moved spaces with `workspace create` (neo PR #174)
| flat | their repo |
| 10-17 | `larisco-email` worktree | a separate space | `larisco-oracle`
|
| 10-21 | fm-gw, laris-meshtastic#3 | `laris-meshtastic` | `neo-oracle` |
| 10-30 | reply-desk, messenger-archive#56, run by the rd-team desk |
`messenger-archive` | `rd-team` |

The 10-21 case added *Under the oracle*, but as prose 250 lines below the
block everyone
copies; the 10-30 run copied the block from inside the target and
repeated it. So the block
now checks it: *Run it* stops with exit 2 when home ≠ target, and both
recipes end by
printing the new space's `repo_name`, which must read home.

## `--list` — what already exists

`herdr worktree list` is the only surface that shows **closed**
worktrees. The sidebar
draws open spaces only, so a worktree you closed is invisible there while
still being on
disk, still registered in git, and one `herdr worktree open --path
<path>` from returning.
`open_workspace_id` is present when a space is open and absent when it is
closed — that
field, not the sidebar, is the state.

```bash
herdr worktree list | jq -r '.result.worktrees[]
```
```

```
  | [ (if .open_workspace_id then "OPEN " + .open_workspace_id else
"closed" end),
      (if .is_prunable then "PRUNABLE" else "-" end),
      .branch,
      (.path | sub("^" + (env.PWD | sub("/wt/.*$";"")); "·") | sub("^/
Users/[^/]+";"~")) ]
  | @tsv' \
  | sort | awk -F'\t' '{printf "%-10s %-9s %-46s %s\n",$1,$2,$3,$4}'
```

Scoped to ONE repo — the response's `source` block shows which repo it resolved from the
cwd. It will not show another oracle's worktrees; for fleet-wide use `wt-picker ls`.

`is_prunable` is the column that matters: it flags a worktree `git worktree prune` would
drop. Locked ones read `false`, which is the whole point of the lock below.

The flow (Nat, 2026-09-28)

`/herdr-wt <slug>` runs these five steps in order. Steps 1 and 5 are the ones people skip.

1. **Issue first** — check the current repo, open a GitHub issue that carries the full
   context of the task (section below). `--no-issue` skips it; so does a repo where you may
   not open issues.
2. **Worktree** — first *Step 0*: if the code lives in another repo than the one you run
   from, use *Under the oracle*, not *Run it*. Otherwise *Run it* below, with the issue number in the lock reason. New work goes
   in `ψ/lab/<slug>/` inside it (the default). `--repo [org]` makes a new repo instead;
   see *`--repo`* below.
3. **Record the workspace on the issue** — one comment: branch, path, herdr space and pane.
4. **Agent** — warm direnv, `pane run "claude"`, rename (see *Then put an agent in it*).
5. **Brief from the issue** — send the agent a short prompt that points at the issue.
   The issue is the brief; the prompt only says where it is and how to report back.

Why an issue and not just a long prompt: a prompt typed into a pane lives in that agent's
scrollback and dies with it. Nobody else can read it, review it, or find it next week. An
issue is durable, linkable, and readable by the human, the main session and any other
agent, and the eventual PR closes it with `Closes #N`. On 2026-09-28 two worktree agents

````
(presentation, proof-with-coding) were each briefed with ~60-line prompts
that existed only
in their panes; this step exists so the next brief is not lost the same
way.

## Step 1 — open the issue in the current repo

Check before opening anything:

```bash
gh repo view \-\-json nameWithOwner,hasIssuesEnabled,viewerPermission \
  -q '"\(.nameWithOwner) issues=\(.hasIssuesEnabled)
perm=\(.viewerPermission)"'
```

Open the issue only when all three hold:

- the cwd is a GitHub repo (`gh repo view` succeeds),
- `hasIssuesEnabled` is `true`,
- `viewerPermission` is `ADMIN`, `MAINTAIN` or `WRITE` — i.e. a repo we
own or are a member
  of. **Never open issues on a third-party repo** (fleet rule: fix
locally, document as
  knowledge). For those, say so and fall back to `--no-issue`.

The body must stand alone: an agent that reads only the issue should be
able to do the work.
Write it for a reader who has never seen the conversation.

```bash
REPO=$(gh repo view \-\-json nameWithOwner -q .nameWithOwner)
LABEL=$(gh label list \-\-limit 100 \-\-json name -q '.[].name' | rg -m1
'^oracle:' || true)
ISSUE_URL=$(gh issue create \-\-repo "$REPO" \-\-title "<slug>: <one-line
goal>" \
  ${LABEL:+\-\-label "$LABEL"} \-\-body-file - <<'EOF'
## Goal
<what done looks like, one paragraph>

## Context
<why this exists; decisions already made; what was tried; links to prior
issues/PRs/files>

## Sources
- <repo paths the agent must read first, with line ranges where useful>

## Deliverables
1. <file / artifact / PR>

## Rules
- <hard constraints: data it must not use, things it must not claim, no
push/merge, etc.>

## Done when
````
````

````
- [ ] <checkable criterion>

## Out of scope
- <what not to touch>

## Workspace
Filled in by a comment once the worktree exists.

\-\-\-
Opened by <oracle> (AI) via /herdr-wt · session <session-id>
EOF
)
ISSUE=${ISSUE_URL##*/}; echo "$ISSUE_URL"
```

Keep the `--body-file -` heredoc form: long bodies with backticks and `$`
survive it intact,
where `--body "..."` gets mangled by the shell. The trailing attribution
line matters because
the issue is authored by the human's GitHub account, not by the agent.

Then run *Run it* below with `ISSUE` set, and afterwards record the
workspace:

```bash
gh issue comment "$ISSUE" \-\-repo "$REPO" \-\-body "Workspace
- branch: \`$name\`
- path: \`wt/$name\`
- herdr space: \`$SPACE\` · pane: \`$PANE\`
- agent: \`$SLUG-$owner\`
- lab: \`${lab:-none (no ψ/ in this repo)}\`"
```

## Run it

Copy-paste, substituting the slug (and optionally a base ref). With an
issue, add `|#$ISSUE`
to the lock reason so the worktree can be traced back to it even after
the space is closed:

```bash
SLUG=herdr-bring; BASE=HEAD
repo=$(dirname "$(git rev-parse --path-format=absolute --git-common-
dir)")   # MAIN checkout, see below
# Step 0 guard: home = the repo of YOUR pane (not $PWD). Another repo →
"Under the oracle".
HOME_REPO=${HOME_REPO:-$(herdr pane get "$HERDR_PANE_ID" | python3 -c
"import json,sys;print(json.load(sys.stdin)['result']['pane']['cwd'])")}
home=$(dirname "$(git -C "$HOME_REPO" rev-parse --path-format=absolute --
git-common-dir)")
[ "$repo" = "$home" ] || { echo "STOP: code is in $repo, you run from
$home — use 'Under the oracle' so the space nests under $(basename
"$home")"; exit 2; }
day=$(TZ='Asia/Bangkok' date +%-d%b-%a%Y | tr '[:upper:]' '[:lower:]')
````

```
owner=$(basename "$repo" | sed 's/-oracle$//')
name="$SLUG-$owner${ISSUE:+-issue$ISSUE}-$day"   # register-free-cloud-
homelab-issue23-28sep-mon2026
dest="$repo/wt/$name"
test ! -e "$dest" || { echo "already exists: $dest"; exit 1; }
herdr worktree create \-\-cwd "$repo" \-\-branch "$name" \-\-base "$BASE"
\-\-path "$dest" \-\-no-focus \
  | python3 -c "import json,sys; print('space', json.load(sys.stdin)
['result']['workspace']['workspace_id'], '->', sys.argv[1])" "$name"
git -C "$repo" worktree lock \-\-reason "herdr|$(whoami)@$(hostname -s)|
$(date -Iseconds)|$SLUG${ISSUE:+|#$ISSUE}" "$dest"
[ -f "$dest/.envrc" ] || [ ! -f "$repo/.envrc" ] || cp -a "$repo/.envrc"
"$dest/.envrc"   # untracked: not in a new worktree
(cd "$dest" && maw token use "$(maw token resolve)")   # see below —
token AND direnv trust, in one step
lab=""                                  # lab folder: only in repos with
a ψ/ vault
if [ -d "$dest/ψ" ]; then
  lab="ψ/lab/$SLUG"
  mkdir -p "$dest/$lab"
  if git -C "$dest" check-ignore -q "$lab/x"; then
    printf '!%s/\n' "$lab" >> "$dest/.gitignore"
    git -C "$dest" check-ignore -q "$lab/x" \
      && { echo "$lab still ignored: a parent dir is excluded,
fix .gitignore by hand"; exit 1; }
  fi
fi
echo "$dest"
herdr workspace list | python3 -c "import json,sys;[print('space',
w['workspace_id'], 'nests under', w['worktree']['repo_name']) for w in
json.load(sys.stdin)['result']['workspaces'] if (w.get('worktree') or
{}).get('checkout_path')==sys.argv[1]]" "$dest"
```

`repo` is the main checkout, from `--git-common-dir`, never `--show-toplevel`. From
inside a worktree, `--show-toplevel` returns that worktree, so two things break at once:
`dest` nests as `<repo>/wt/<a>/wt/<b>` (the JSONDecodeError of 2026-09-15: `herdr worktree
create` answered with empty stdout), and `owner` becomes the worktree's folder name.
Measured 2026-10-08 from a herdr worktree: `--show-toplevel` gave owner `worktree-calm-meadow-eea9`, `--git-common-dir` gave `neo`. The `just herdr-wt` recipe has
had this fix since 2026-09-15; this block lagged it until 2026-10-08.

New work goes in `ψ/lab/<slug>/` (Nat, 2026-09-28), the default when there is no
`--repo`. Oracle repos gitignore `ψ/` and re-include lab folders one by one
(`!ψ/lab/codex-fleet/` in homelab), so the block appends the same line for the new folder
and stops if git still ignores it. That `.gitignore` line is committed on

````
the branch with
the work. Repos without `ψ/` skip this and work at the worktree root.

The lock is borrowed from `/incubate` (same `tool|who|when|why` field
order, so one parser
reads both). It is **not** security — it is a "do not auto-clean me"
flag, and it earns its
keep twice, both verified 2026-09-14:

- `git worktree prune` leaves it registered **even after the directory is
deleted**.
  Without a lock, prune is exactly what silently drops a worktree — and
prune is the
  command recovery docs tell people to run.
- `git worktree remove --force` refuses it outright:
  `fatal: cannot remove a locked working tree` (needs `-f -f`).

The lock reason is also the only per-worktree metadata that survives
`kill -9`; it lives at
`<repo>/.git/worktrees/<id>/locked`, where `<id>` is the folder name at
creation time
(a later `git worktree move` keeps it). `git -C <worktree> rev-parse --
git-dir` prints it.

Produces, for slug `herdr-bring` in `neo-oracle` on 14 Sep 2026:

| | |
|---|---|
| branch | `herdr-bring-neo-14sep-mon2026` |
| path | `<repo>/wt/herdr-bring-neo-14sep-mon2026` |
| herdr | its own space, `--no-focus` |

**In `laris-co/neo-oracle` only**, the same thing is wrapped as `just
herdr-wt <slug> [base]`.
No other repo has that recipe — verified across the full `ghq list`, 1 of
N repos — so
use the inline block everywhere else.

Then put an agent in it (the pane comes up as a bare shell):

```bash
herdr pane run <pane> 'echo "TOK=$CLAUDE_TOKEN_NAME"'   # warm direnv
FIRST — see below
herdr pane run <pane> "claude"          # NOT `herdr agent start` — see
below
herdr agent rename <pane> "$SLUG-$owner"   # agent names cap at 32 chars
```

That first throwaway command is not decoration — without it the agent can
still come up
on the wrong token even though you used `pane run`. See *Trap: `pane run`
is not enough on
a fresh worktree* below.
````

`herdr agent rename` run immediately after `pane run "claude"` can fail with
`{"error":{"code":"agent_not_found",...}}` — the claude process has not registered as an
agent yet (measured 2026-09-28, `w6K:p1`). Wait for it, then rename:

```bash
herdr pane wait-output <pane> \-\-match 'bypass permissions' \-\-timeout 40000   # MILLISECONDS
herdr agent rename <pane> "$SLUG-$owner"
```

`--timeout` is in **milliseconds**. The earlier `--timeout 40` meant 40 ms and timed out
every time, even with claude already up (measured 2026-09-28, `w6Q:p1` and `w6R:p1`; the
same match returned instantly once the value was right).

Brief the agent from the issue (step 5)

Send it through fleet (`fleet_send`, target = the pane id), not `herdr agent prompt`, which
prefix-matches agent names. Keep the prompt short; the issue carries everything:

```
Your task is GitHub issue #<N> in <owner/repo>: <issue URL>
Read it first: gh issue view <N> \-\-repo <owner/repo> \-\-comments
It is your full brief — sources, deliverables, rules, done criteria.
Report progress as comments on that issue (what is done, what is blocked, open questions).
Work only in this worktree, commit on this branch. Open a PR with "Closes #<N>" only if the issue says to.
```

Anything you learn later that changes the task goes on the issue as a comment first, then
into the pane with a one-line "see the new comment on #N". The pane is never the only copy.

Do not use `herdr agent start` here, despite it being the obvious command. Two
measured reasons, both 2026-09-17 in `mycelium-oracle`:

- **It bypasses direnv.** `herdr agent start` spawns claude from the *herdr server's*
  environment, not through the pane's shell, so the `.envrc` you just allowed never
  runs. The agent inherits whatever token the server happens to hold. Measured in
  `wt/mycelium-vm-server-17sep-thu2026`, where the repo is assigned `dd2`:

  | how the agent was started | `CLAUDE_TOKEN_NAME` in its process |

```
  |---|---|
  | `herdr agent start --kind claude` | `pb` ✗ |
  | `herdr pane run <pane> "claude"` | `dd2` ✓ |

  `herdr pane run` goes through the shell, direnv fires, the right token
is inherited.
- **Its `<name>` argument caps at 32 characters** and the full worktree
name often
  exceeds it — `mycelium-reincarnate-17sep-thu2026` is 34:

  ```
  {"error":{"code":"invalid_agent_name","message":"agent name must start
with a
  lowercase letter and contain only lowercase letters, digits, '-' or '_'
(1-32
  characters)"}}
  ```

  It **starts the agent anyway and leaves it unnamed** — the error lands
after the
  side effect. Name the agent `<slug>-<owner>`; the workspace label
already carries
  the full `<slug>-<owner>-<date>`.

`--here` — new worktree, opened as a split in THIS tab (Nat,
2026-10-08)

Use it when the human wants a second parallel agent beside them, in the
same pane layout,
but **not** in the same checkout. Two agents in one worktree share the
branch and the git
index, so one `git add` stages the other's half-done edits. `--here`
gives the second
agent its own branch and folder while the pane sits next to yours.

Everything else in the flow is unchanged: issue first, lock, `.envrc`,
token, agent, and
brief. Only *Run it* changes, in two ways:

1. **`git worktree add`, not `herdr worktree create`.** The herdr command
always opens a
   new space and has no flag to skip that.
2. **`pane split --cwd <dest>`**, so the new pane starts in the new
worktree inside the
   current tab.

```bash
SLUG=<slug>; BASE=HEAD
main=$(dirname "$(git rev-parse --path-format=absolute --git-common-
dir)")   # the MAIN checkout
day=$(TZ='Asia/Bangkok' date +%-d%b-%a%Y | tr '[:upper:]' '[:lower:]')
owner=$(basename "$main" | sed 's/-oracle$//')
name="$SLUG-$owner${ISSUE:+-issue$ISSUE}-$day"
dest="$main/wt/$name"
```
```

```
test ! -e "$dest" || { echo "already exists: $dest"; exit 1; }
git -C "$main" worktree add -b "$name" "$dest" "$BASE"
git -C "$main" worktree lock \-\-reason "herdr|$(whoami)@$(hostname -s)|
$(date -Iseconds)|$SLUG${ISSUE:+|#$ISSUE}" "$dest"
[ -f "$dest/.envrc" ] || [ ! -f "$main/.envrc" ] || cp -a "$main/.envrc"
"$dest/.envrc"
(cd "$dest" && maw token use "$(maw token resolve)")
DIR=right   # use DIR=down when the tab already has two panes
PANE=$(herdr pane split \-\-current \-\-direction "$DIR" \-\-ratio 0.5 \-
\-\-cwd "$dest" \-\-no-focus \
  | python3 -c "import json,sys;print(json.load(sys.stdin)['result']
['pane']['pane_id'])")
herdr pane rename "$PANE" "$name" >/dev/null
echo "$PANE -> $dest"
```

Then run the lab-folder block from *Run it* against `$dest`, and continue with *Then put an
agent in it* using `$PANE`. Use `--direction down` when the tab already has two panes.

- **Use `--git-common-dir`, not `--show-toplevel`.** From inside a worktree, the toplevel is
  that worktree, so `$toplevel/wt/...` would nest the new worktree inside the current one.
  Herdr's own spaces live in `~/.herdr/worktrees/<repo>/…`, where this matters every time.
- **The sidebar shows no new row.** The pane belongs to the current space, whose row
  still shows the current branch. Check the new branch with the pane's prompt,
  `herdr pane get $PANE` (`cwd`), or `git -C "$main" worktree list`.
- **Splitting a pane is not `--here`.** Right-click → split, or `pane split` without
  `--cwd`, opens the SAME worktree. A fresh `claude` there is a separate session, but it
  shares the files.

Tested 2026-10-08 on a scratch repo: `w2J:p4` opened in tab `w2J:t1` with cwd
`.../wt/demo-here-8oct-thu2026`, and the prompt showed the new branch.

`--here` with no slug: the agent names it, then a popup (Nat, 2026-10-08)

The same act-then-veto flow as `/herdr-rename`. Do not open with a question.

1. **Pick the slug yourself.** Base it on what the human just said they want the new
   pane for, then on the conversation. Draft 3 slugs, best first: meaningful and
   memorable (`kong-cutover`, `fts-stopword-purge`), accurate before clever. Run the

````
   collision check on all three.
2. **Cut the worktree with the best pick right away.** From the `--here`
block above,
   run only the lines up to and including `git worktree add` (setting
`SLUG`, `main`,
   `day`, `owner`, `name`, `dest`, then the `test ! -e` check). Stop
before
   `git worktree lock`.
3. **Popup, once.** AskUserQuestion: `Created <name>. Keep it?`. Option 1
is
   `Keep <slug> (Recommended)`, options 2–3 are the runner-ups with a
one-line reason
   each, and the built-in Other takes a typed name.
4. **If the human picks another name, move the whole thing, not just the
branch.** No
   agent and no pane exist yet, so the folder can still move safely:

   ```bash
   SLUG=<chosen-slug>; new="$SLUG-$owner${ISSUE:+-issue$ISSUE}-$day"
   git -C "$main" branch -m "$name" "$new"
   git -C "$main" worktree move "$dest" "$main/wt/$new"
   name=$new; dest="$main/wt/$new"
   ```

   Set `SLUG` to the final pick too. The lock reason in step 5 is built
from `$SLUG`;
   on 2026-10-08 a test run kept the first pick there and had to re-lock.

   Tested 2026-10-08 on a scratch repo: `rc=0`, and `git worktree list`
showed the new
   folder on the new branch.
5. **Then the rest of the `--here` block, from `git worktree lock` on,**
with the FINAL
   slug: lock, `.envrc`, token, `pane split --cwd`,
   label. Lock **after** the popup, because `git worktree move` refuses a
locked worktree.

   `git worktree move` keeps the admin dir under its FIRST name: after
moving
   `here-demo-…` to `here-flow-trial-…`, the lock file was still
   `.git/worktrees/here-demo-here-test-8oct-thu2026/locked`. Find it with
   `cat "$(git -C "$dest" rev-parse --git-dir)/locked"`, never by
building the path from
   the final name. Measured 2026-10-08, git 2.54.

Name, folder, branch, label and lock reason all agree, because nothing
was created
under the old name except the folder you just moved.

**No evidence at all** (the human said nothing about the task): fall back
to a herdr-style
placeholder and skip the popup, since there is nothing to choose between:

```bash
````
````

```
SLUG=$(printf '%s-%s-%s' "$(shuf -n1 -e amber brisk calm quiet silver)" \
  "$(shuf -n1 -e dune fern meadow river valley)" "$(openssl rand -hex
2)")   # e.g. quiet-dune-8846
```

- **Skip the issue** for a placeholder (implied `--no-issue`). An issue needs a goal.
  With an AI-picked slug, open the issue after the popup, under the final name.
- **Name a placeholder later with `/herdr-rename` in the NEW pane**, not in yours. It
  renames `git branch --show-current` of the pane it runs in, so running it from the
  original pane renames the wrong branch.

What a later rename changes and what it leaves behind. Measured 2026-10-08 on a scratch
repo, git 2.54:

thing	after `git branch -m`
branch (prompt, `git worktree list`)	new name, rc=0
commits, lock state	kept
folder `wt/<placeholder>…`	**unchanged** (moving it kills the agent's cwd)
lock reason	**unchanged**, still names the placeholder slug
pane label	follows: `/herdr-rename` step 3 renames it when it equals the old branch (tested, `w2J:p6`)

`git branch -m` also works from a different worktree of the same repo (tested: renamed
from the main checkout, and the other worktree's prompt followed). Git updates every
worktree's HEAD.

Working on ANOTHER repo the oracle cares for (Nat, 2026-10-10)

The oracle sits in its own repo, but the code lives elsewhere: herdr-orca-oracle building a herdr
feature in the fork `Soul-Brews-Studio/herdr` (issue herdr-orca-oracle#8). Run *Run it* with
`repo=<that repo's main checkout>` instead of the cwd. Three things differ, all met for real:

what	in this case	why
sidebar row	**must nest under the ORACLE** (Nat, 2026-10-21) — see *Under the oracle* below. A plain `herdr worktree create --cwd <target>` nests it under the target repo's group instead	herdr groups by the SPACE's `worktree.repo_key`, never by a pane's cwd. So the space must be an oracle worktree; the agent's pane can still sit in the target repo's worktree
the issue	open it in the **oracle's** repo when the target has issues off (the fork: `hasIssuesEnabled=false`) or is third-party	

(`viewerPermission=READ`, e.g. upstream `herdrdev/herdr`) | the brief must be durable and ours; never on a third-party repo. Note the target repo in the issue's Context and the Workspace comment |
`gh repo view`	run it as `gh repo view <owner/repo>`	in a fork with an `upstream` remote, bare `gh repo view` answered for **upstream** (READ)
`.envrc` / token	the target repo has no `.envrc` and `maw token resolve` there says "no token assignment found": copy the **oracle's** `.envrc` and run `maw token use "$(cd <oracle> && maw token resolve)"` in the worktree	otherwise the agent starts on the herdr server's token
keep `.envrc` out of git	`echo .envrc >> "$(git -C "$repo" rev-parse --path-format=absolute --git-common-dir)/info/exclude"`	WITHOUT `--path-format=absolute`, `--git-common-dir` printed `.git`, relative to the CALLER's cwd, and the line landed in the oracle's own exclude file (2026-10-10)
report back	the brief ends with `maw hey <oracle main checkout path> "<result + PR url>"`	the worker is a body of the oracle even in another repo

Under the oracle — space from the oracle, pane in the target (Nat, 2026-10-21)

The fm-gw run (laris-meshtastic#3) first opened its space with `herdr worktree create --cwd laris-meshtastic`,
so the sidebar showed it under `laris-meshtastic`. Nat: "it should be here", under `neo-oracle`. Fixed live,
and this is the recipe from now on. It works because worktree identity lives on the SPACE (see the trap below)
and herdr never regroups by a pane's cwd:

```bash
# 1. the code worktree in the TARGET repo — plain git, no herdr space
git -C "$target" worktree add -b "$name" "$target/wt/$name" "$BASE"
git -C "$target" worktree lock \-\-reason "herdr|…|$SLUG|#$ISSUE" "$target/wt/$name"
# 2. the SPACE: a worktree of the ORACLE repo, same slug  →  nests under the oracle
WS=$(herdr worktree create \-\-cwd "$oracle" \-\-branch "$SLUG-$owner-$day" \-\-base origin/main \
       \-\-path "$oracle/wt/$SLUG-$owner-$day" \-\-no-focus | jq -r .result.workspace.workspace_id)
ln -s "$target/wt/$name" "$oracle/wt/$SLUG-$owner-$day/<repo>-$SLUG"
# one hop to the code
echo "/<repo>-$SLUG" >> "$(git -C "$oracle" rev-parse --path-format=absolute --git-common-dir)/info/exclude"
# 3. the agent pane: split INSIDE that space, cwd = the target worktree
SHELL=$(herdr pane list | jq -r \-\-arg w "$WS" '.result.panes[]|select(.workspace_id==$w).pane_id' | head -1)
PANE=$(herdr pane split "$SHELL" \-\-direction right \-\-cwd "$target/wt/$name" \-\-no-focus | jq -r .result.pane.pane_id)
```

Check before the agent starts — the row must name HOME:

```bash
herdr workspace list | jq -r \-\-arg w "$WS" '.result.workspaces[]|
select(.workspace_id==$w)|"\(.label) nests under \(.worktree.repo_name)"'
```

A team desk is an oracle here: the rd-team desk working in `messenger-archive` (#56, 2026-10-30)
made space `wBC` from `rd-team` (`reply-desk-rd-team-issue56-30oct-fri2026`), linked the code
worktree as `messenger-archive-reply-desk/`, and split the agent pane into it. A long-running
helper for the same work (`just desk`) goes in the same space as a third pane, not in a space
of its own in the target.

Then `.envrc` + `maw token use` in BOTH worktrees, warm direnv, `pane run "claude"` in `$PANE`, brief. The
oracle space's own first pane stays as the agent's run pane. Commits, the branch and the PR all happen in the
target repo; the oracle worktree only carries notes (`ψ/lab/<slug>/`) if any.

Repairing one that already landed under the target. If its agent has not started work
(not briefed yet), do not move it: `/exit` it, `herdr workspace close` the wrong space (the
locked code worktree stays), and run the recipe above — done this way on 2026-10-30 (`wBB` →
`wBC`), no live pane moved. If the agent is mid-work (done 2026-10-21, agent kept working mid-build):
create the oracle space as in step 2, then `herdr pane move <agent pane> --tab <its tab> --target-pane
<its shell> --split right`, close the leftover pane in the old space (the space closes itself), and tell the
agent its new pane id — `pane move` renumbers it (`wV:p1` → `wW:p2`).

Starting claude in a folder it has never seen shows **"Is this a project you trust?"** and
blocks `wait-output --match 'bypass permissions'` (timed out at 40 s, `wBF:p1`). After the
throwaway `echo TOK=` command, start claude, wait for `trust this folder|bypass permissions`,
and if it is the trust prompt send `herdr pane send-keys <pane> Down Enter` — only for a repo
we own. If the first `echo TOK=` prints an empty value, run `cd . && echo TOK=…` once more
(direnv evaluated the shell before `maw token use`; a fresh prompt picks it up).

`--repo [org]` — a new repo instead of a lab folder

Without `--repo`, work lands in `ψ/lab/<slug>/` inside the worktree. With it, the work
gets its own repo. Everything else is the normal flow: issue here,

````
worktree under
`<repo>/wt/`, agent, brief. **The main session does not run `/incubate`**
(Nat,
2026-09-28); the agent in the worktree does, because the issue tells it
to. The issue opens
in the current repo, because the new repo's name needs the issue number
first.

Reuse check, in the main session:

```bash
oracle=homekeeper   # the oracle's own name (CLAUDE.md "I am"), not the
repo folder (homelab)
[ -n "$ORG" ] || ORG=$(maw locate "$oracle" \-\-json 2>/dev/null | jq -
er .org) \
  || { echo "oracle '$oracle' not in the maw registry: pass --repo
<org>"; exit 1; }
names=$(gh repo list "$ORG" \-\-limit 1000 \-\-json name -q '.[].name')
echo "$names" | rg "^$oracle-$SLUG-issue[0-9]+$"   # exact hit: reuse
that repo
echo "$names" | rg -F \-\- "$SLUG"                  # loose hits: older
names; show them, the human picks
```

- **Exact hit**: the agent will run `/incubate $ORG/<that-name>`. The old
name keeps its old
  issue number, which is when the repo was born. A new issue is still
opened (every piece
  of work gets its own brief) and links to the repo.
- **Loose hits only**: list them and ask. Do not pick for the human.
- **No hit**: the agent will run `/incubate $ORG/$oracle-$SLUG-
issue$ISSUE`.
  `/incubate` creates the private repo when it does not exist and clones
it with `ghq`.

Put that exact `/incubate` command in the issue as the first deliverable,
with "add this
issue's URL to the new repo's README" as the second. Then do *Run it*
exactly as usual,
minus the lab block (set `lab=""`, skip the `if`). The worktree sits
under the current repo
with the others, and `/incubate` run from it links the clone under the
worktree's
`ψ/incubate/`.

Do **not** open a bare space on the main checkout with `herdr workspace
create --cwd "$repo"`.
Tried 2026-09-28 (space `w6Q`, closed): it lands in the sidebar as a
separate `main` entry,
not in the repo's worktree tree, and its agent works in the main
checkout.

The agent's name is `$SLUG-$owner`, or just `$SLUG` when that is over 32
characters
````

(`local-black-white-machine-homelab` is 33).

Check GitHub, not `ghq list`: ghq keeps stale entries. On 2026-09-28
`laris-co/homekeeper-oracle` was in ghq and in the maw registry but not
on GitHub; the
folder is an old clone of `laris-co/homelab`. `maw locate` resolves
oracles only
(`register-free-cloud` gives `not found in 313 registered`), so it
supplies the org, not
the reuse answer.

Before first use on a new machine — one prerequisite

`wt/` is already ignored **fleet-wide** via `~/.config/git/ignore` (added
2026-09-14).
Git reads that path by XDG default when `core.excludesFile` is unset, so
no per-repo
`.gitignore` edit is needed — one line covers every repo on the machine.
Verify on a new
machine before trusting it:

```bash
git check-ignore -v wt/x || printf '/wt/\n' >> ~/.config/git/ignore
```

The leading slash anchors it to each repo's root, so it does not shadow
nested `wt/`
directories such as incubate's `ψ/incubate/<slug>/wt/`.

`dig.py` must fold `wt` or every worktree reports as its own phantom
project in
`/dig`. Check and patch the global skill file:

```bash
rg -n 're\.sub' ~/.claude/skills/dig/scripts/dig.py | rg 'wt'
# want: re.sub(r'-(wt|agents)-.*$', '', base)
# NOT:  re.sub(r'-wt-\d+.*$', '', base)
```

The original required a **digit** right after `wt-`, so it matched `wt-1-
white` but not
`wt-herdr-14sep-mon2026`. Session dirs then fall through to
`clean.split('-')[-1]` and
report as a phantom repo — exactly why `homelab-agents-1-gpu` showed up
as repo `gpu`.
Applied on m5 2026-09-14; **white still needs it**.

Why the name is shaped this way

Decided against the obvious alternatives, with evidence. Do not "fix"
them back without
re-running the same checks.

Date goes LAST. maw's own day repos are date-first, so date-first

looks consistent.
Two reasons it isn't, for worktrees:

1. *Date-first does not sort chronologically.* maw writes single-digit days unpadded
   (`1sep-tue2026` … `9sep-tue2026`), so `9sep` sorts **after** `10sep`.
Verify:
   `printf '%s\n' 9sep-tue2026-a 10sep-wed2026-b | sort`
2. *Trailing it groups by topic* — every `herdr-*` worktree lands together instead of
   scattering across months.

Not an inconsistency with maw: its date-first names are **day repos** where the date is
the identity. A worktree is **task-scoped** — the task is the identity, date is metadata.

Slug FIRST, then the oracle (changed 2026-09-20). The oracle still belongs in the
name — a branch, a PR and `git worktree list` are all flat, and there `herdr-14sep` alone
does not say whose it is. But it must not lead, because herdr's sidebar **groups by repo
now** and truncates from the right: under a `neo-oracle` node every row began `neo-` and
the prefix spent ~4 of ~9 visible characters repeating what the parent already said.

Measured over 38 live workspaces, collisions *within a repo group* by visible width:

chars	`<oracle>-<slug>-<date>`	`<slug>-<oracle>-<date>`
9	5	2
14	3	**0**
18	1	**0**

The old order produced the exact ambiguity the prefix existed to prevent —
`neo-herdr…` for both `neo-herdr-14sep` and `neo-herdr-remote-16sep`, `neo-arra-…` for
both arra worktrees. Slug-first resolves every pair by 14 characters.

`neo-oracle` → `neo`; a non-oracle repo keeps its basename (`homelab` → `…-homelab-…`).

The earlier rationale said the sidebar "lists spaces FLAT". That was true on 2026-09-14
and is not true now; it is a three-level tree (Local → repo → worktrees). If it ever goes
flat again, the owner is still present, just later in the string.

Issue number between oracle and date (`-issue23-`, Nat 2026-09-28). The branch then

names its own brief: `register-free-cloud-homelab-issue23-28sep-mon2026`
points at #23
without opening anything. Written `issue23`, not `i23`, so it reads
without a legend.
No issue (`--no-issue`), no segment.

A new repo (`--repo`) is `<oracle>-<slug>-issue<N>`, with the oracle
first and no date
(Nat, 2026-09-28): `homekeeper-register-free-cloud-issue23`. The GitHub
org list and
`ghq list` are flat and alphabetical, with no parent node to group by, so
the oracle prefix is
what keeps one oracle's repos together. A repo outlives the day it was
made, so the date stays
on the worktree and branch, and the issue number already says where it
came from. The
`-issue<N>` tail also ends the slug, so `^homekeeper-register-free-cloud-
issue[0-9]+$` finds
that feature and never `register-free-cloud-api`. `--repo` therefore
needs an issue; it does
not combine with `--no-issue`.

The first run (#25, 2026-09-28) used an earlier draft,
`<oracle>-<slug>-<oracle>-issue<N>-<day>` (69 characters). Nat dropped
the repeated oracle and
the date the same day.

Location `<repo>/wt/`, not `~/.herdr/worktrees/`. herdr's
`worktrees.directory` is a
single global root it appends `<repo>/<branch>` to — it cannot express
"relative to the
current repo". `--path` is the only lever. `wt` also matches the
convention already
dominant on disk (`homelab-wt-1-white`, `wt-2-cftunnel`, `wt-
statusline`).

Always `maw token use` the new worktree

A worktree is a **new path**, so direnv's allow-list has never seen it —
even
though the parent repo is trusted and `.envrc` is the identical file.
Until you
allow it, direnv refuses to load and every pane opened there starts
without
the repo's environment.

`direnv allow "$dest"` fixes only half of that, so **call `maw token use`
instead** —
it does both jobs in one step. Measured 2026-09-17 on
`wt/mycelium-vm-server-17sep-thu2026`:

| | rewrites `.envrc` to the assigned token | direnv-allows the new path
|
|---|---|---|

````
| `direnv allow "$dest"` | no | yes |
| `maw token use <name>` | yes | yes |

Proof for both halves. Trust: with the allow-file deleted, `maw token use
dd2`
recreated `~/.local/share/direnv/allow/dc56099d…` on its own. Token: `maw
token use pb`
rewrote **both** lines of the worktree `.envrc` (`CLAUDE_TOKEN_NAME` and
the
`pass show claude/token-<name>` path, md5 `812ca807…` → `bc2ed1aa…`), and
`maw token use dd2` restored it byte-for-byte.

Why the token half matters: the `.envrc` a worktree starts with is a
**copy**, so it
names whatever token the source checkout named at copy time. `direnv
allow` faithfully
trusts a stale answer. `maw token resolve` reads the assignment for the
cwd's oracle —
it resolves correctly from inside a worktree, verified — so the repo-
agnostic form is:

```bash
(cd "$dest" && maw token use "$(maw token resolve)")
```

Skipping this is not cosmetic. Agents in an unallowed worktree run
without
`CLAUDE_CODE_OAUTH_TOKEN`, `CLAUDE_TOKEN_NAME`, `ORACLE_URL` and
`ORACLE_PROJECT` — a
different environment than the shell that created them, which surfaces
later as an auth
failure nobody can reproduce. Measured 2026-09-15 on `wt/neo-herdr-14sep-
mon2026`, the
pane showed:

```
direnv: error …/.envrc is blocked. Run `direnv allow` to approve its
content
```

while the *loaded* RC was a different repo's entirely.

It is a trust decision, so read the file before approving it — `.envrc`
here
shells out to `pass show` for a real token.

### Verify the process, not the statusline

```bash
direnv exec "$dest" bash -c 'echo "$CLAUDE_TOKEN_NAME $ORACLE_PROJECT"'
```

That proves what a *shell* at that path gets. It does **not** prove what
a running
````

agent got — those differ whenever the agent was started outside the shell. The
statusline is no help either: its `🔐<name>` marker reads `.envrc`, so it happily
displayed `🔐dd2` for an agent whose process actually held `pb`. The only check that
settles it:

```bash
ps -Eww -p <agent-pid> | tr ' ' '\n' | rg '^CLAUDE_TOKEN_NAME='
```

Find the pid by cwd:

```bash
pgrep -f 'claude \-\-dangerously' | while read p; do
  c=$(lsof -a -p "$p" -d cwd -Fn 2>/dev/null | rg '^n/' | sed 's/^n//' | head -1)
  [ "$c" = "$dest" ] && echo "$p"
done
```

Two details in that snippet are load-bearing, both measured 2026-09-21:

- **`pgrep -f claude` is too wide.** An agent's MCP servers inherit its cwd, so the plain
  pattern returned 5 pids all reporting the worktree directory — `uv`/`serena`, a
  `python`, a `bun` plugin server, and the real `claude`. They all carried the same
  (wrong) `CLAUDE_TOKEN_NAME`, so the output looked like corroboration rather than one
  fact repeated four times. Match the claude invocation, then confirm with
  `ps -o pid,lstart,command -p <pid>`.
- **`rg '^n'` must be `rg '^n/'` and end in `head -1`.** `lsof -Fn` emits other `n`-prefixed
  fields; without the anchor and the head, `$c` can pick up a non-path line and the
  comparison silently never matches.

Recovery is cheap when the agent is fresh: `herdr agent prompt <pane> '/exit'`, then
`herdr pane run <pane> "claude"`.

Trap: `pane run` is not enough on a fresh worktree

The table above says `pane run` gets the right token. That holds only if the pane's shell
reached a **prompt after** `maw token use` allowed the path. On a worktree you just created
it usually has not, and the agent comes up on the server's token anyway.

Measured 2026-09-21, `wt/maw-cli-neo-21sep-mon2026` (repo assigned

`dd2`):

`.envrc`	`dd2`
statusline	`🔐dd2`
agent process	**`pb`** ✗

The sequence is the cause. `herdr worktree create` opens the space, and its pane shell
starts **immediately** — before `maw token use` runs. direnv evaluates once, at that
prompt, finds the path untrusted, and refuses. Allowing the path afterwards does not
reach back into a shell that is already sitting at a prompt, so `pane run "claude"`
executes in a shell holding no repo environment and claude inherits the herdr server's.

Fire one throwaway command first — any command, its output does not matter. It forces a
fresh prompt, direnv evaluates against the now-trusted path, and the shell picks the
token up:

```bash
herdr pane run <pane> 'echo "TOK=$CLAUDE_TOKEN_NAME"'
# expect in the pane: direnv: loading .../.envrc   then   TOK=<assigned>
herdr pane run <pane> "claude"
```

If you skip it and find the agent on the wrong token, recovery is the same as elsewhere:
`herdr agent prompt <pane> '/exit'`, one throwaway command, then `pane run "claude"`.

Cleanup

Unlock first — the lock blocks removal on purpose:

```bash
git -C <repo> worktree unlock wt/<name>
herdr worktree remove \-\-workspace <wsid> \-\-force
```

Removes the checkout AND deregisters from `git worktree list` in one step. Check first:

```bash
git -C wt/<name> status \-\-short                  # dirty?
git -C wt/<name> rev-list \-\-count main..HEAD     # unmerged commits?
```

If the worktree was ever **moved**, herdr's record keeps the OLD path —

it deletes the
space but leaves the real checkout behind. Finish with git:

```bash
git -C <repo> worktree remove wt/<name>
```

Trap: never `git worktree move` a worktree with a live agent in it

A running agent pins its **cwd and Claude project dir at startup**. git and herdr both
relocate cleanly — git metadata updates, herdr's pane cwd follows — but the agent inside
cannot follow. It points at a vanished path and silently falls back to the main repo.

The tell is quiet: `pwd` inside the agent returns the **main repo**, and its status line
reads `main@<sha>` instead of the worktree branch.

A healthy worktree agent shows path, branch, and 🌳:

```
📁 laris-co/neo-oracle/wt/neo-herdr-14sep-mon2026
   neo-herdr-14sep-mon2026@31d3ceb 🌳
```

Rule: move while empty, or restart the agent after. Restart is cheap — `/exit` (two
`ctrl+c` did NOT exit it), then `herdr pane run <pane> "claude"` again; the pane cwd is
already right, and going through the shell is what re-applies direnv.

Trap: worktree identity lives on the SPACE, not the pane

From herdr's source, `src/workspace.rs:196` — `worktree_space` is a field on `Workspace`.
`pane.rs` and `tab.rs` contain **zero** `worktree` references.

Moving a worktree's pane into another space's tab strips its worktree identity: it loses
the indented nesting and inherits the destination's branch label. `herdr worktree list`
then shows `open_workspace_id: -`. Recover:

```bash
herdr worktree open \-\-cwd <repo> \-\-path <worktree-path> \-\-label <name> \-\-no-focus
```

Adjacency and worktree identity are mutually exclusive — a model limit, not a missing flag.

Putting worktree agents side by side and back (`/herdr-room`). A room

````
takes them OUT of their
worktree spaces, so the nesting disappears while they share a page. That
is expected. On
`herdr-room out`, check that they nest again:

```bash
herdr workspace list | python3 -c "import json,sys;
[print(w['workspace_id'], w.get('label'), 'WT' if w.get('worktree') else
'FLAT') for w in json.load(sys.stdin)['result']['workspaces']]"
```

Any `FLAT` row that should be a worktree is repaired in place, with no
pane move and the agent
untouched, by the `worktree open` above (`already_open: true` means it
attached to the existing
space). Measured 2026-10-06: herdr-room before its fix broke 5 floodboy
worktree agents out flat. 4 were
repaired this way. The 5th was registered to its old space (kept alive by
a helper pane), so it was moved back
there with `pane move --tab <old tab> --target-pane <pane> --split
right`.

## Trap: don't branch in a shared checkout

`git checkout -b` in a repo other agents are working in moves **all of
them** onto your
branch. That is what this skill exists to avoid — use a worktree for
branch work,
including your own PR branches.

## Boundary with `/incubate --wt`

Both cut worktrees; they do not merge and neither calls the other. The
split is
**ownership of the repo**, and it decides where the checkout may legally
live.

| | `/incubate --wt` | `/herdr-wt` |
|---|---|---|
| repo | someone else's, cloned via `ghq` | an oracle repo you own |
| worktree path | `$XDG_STATE_HOME/incubate/worktrees` (outside) |
`<repo>/wt/` (inside) |
| branch | `incubate/<slug>` | `<slug>-<oracle>[-issue<N>]-<date>` |
| lock | always, at creation | always, at creation (adopted from
incubate) |
| opens a herdr space | no | yes |

**Why incubate must stay outside and we may stay inside** — from
incubate's own notes:
`ghq list` decides "is this a repository" by `stat`-ing `<dir>/.git`, and
a linked
worktree's `.git` is a *file*, not a directory. A worktree under the ghq
root therefore
registers as a phantom repo. Incubate clones *into* the ghq root, so it
````

```
has no choice but
to place bodies elsewhere. Our `wt/` sits inside an oracle repo that is
already a single
ghq entry, and it is gitignored, so nothing enumerates it. Same
constraint, two correct
answers — do not "unify" them onto one path.

**Picking one**: if you would open a herdr space for it, it is `/herdr-
wt`. If it is a
third-party repo you are contributing to, it is `/incubate`.

## Boundary with `/herdr-ticket`

`/herdr-ticket` starts from an issue that **already exists** and cuts a
`wt/<oracle>-<slug>-<date>` branch off `origin/<default>`. `/herdr-wt`
**creates** the issue
from the work you are about to start. If the issue exists, use `/herdr-
ticket`; if it does
not, `/herdr-wt` writes it.

## Related

- `/herdr-bring` — land an existing maw handle into a herdr space
(project skill in neo-oracle)
- `herdr` (global skill) — the CLI reference: panes, agents, forking,
prompting
- `/isolate-worktree` — different tool: spawns an isolated OMX/Codex
worker
```

herdr-bring/SKILL.md

/herdr-bring: หา oracle ด้วย maw locate แล้วเปิด space ของมันใน herdr · 270 บรรทัด

```
---
name: herdr-bring
description: Ensure a herdr presence for any maw handle — resolves it via
`maw locate`, opens its own herdr space, and resumes local Claude history
when there is any, live or not. Kills a live maw tmux session only when
one genuinely exists and is idle. Self-contained; works in any repo on
any machine, no justfile recipe needed. Use when the user says "bring X
into herdr", "move X from maw to herdr", "adopt X", or gives you a `maw
ls` listing and asks to land some or all of it. Do NOT use to fork a
session already inside herdr (use `just herdr-fork`), to cut a worktree
(use /herdr-wt), or to take a GitHub issue (use /herdr-ticket).
argument-hint: "<maw-handle> [label] | (no args to list candidates)"
---

# /herdr-bring — land a maw handle into herdr
```

````
Puts a Claude Code process into its own herdr space for the repo a maw
handle
resolves to, on **this pane's own herdr server**. A live maw tmux session
is the
exception case, not the requirement — three landing modes:

- **live + transcript** → verify idle, kill the maw session, resume the
same
  session id. Full history survives the move.
- **not live + transcript** → resume it directly. Nothing to kill.
- **neither** → start fresh.

"Bring" undersells it: this *ensures a herdr presence for this repo,
using
whatever local history exists*. A handle that only ever existed as a
registry
entry still lands, as a fresh start.

## Before anything

```bash
test "${HERDR_ENV:-}" = 1 || { echo "not inside a herdr pane — stop";
exit 1; }
```

If that fails, say so and stop. Do not drive a herdr session from outside
one.

## Socket targeting

herdr sockets are per-server and ids do not collide across them. This
targets
the **calling pane's own socket** (`$HERDR_SOCKET_PATH`, which herdr
exports into
every pane it manages). An earlier version hardcoded a named session and
broke
the moment that server was not running while the calling pane lived in
`default`. To target a different server on purpose, pass `--session
<name>` to
each `herdr` call below, or export `HERDR_SOCKET_PATH` first.

Note the documented precedence: `--session` > `HERDR_SOCKET_PATH` >
`HERDR_SESSION`
> default. A pane's own `HERDR_SOCKET_PATH` silently wins over
`HERDR_SESSION`.

## Step 1 — list candidates

Show what exists before touching anything, even when the user named a
target.
Confirming it exists costs nothing and catches typos before a kill.

```bash
maw ls \-\-json | python3 -c "
````
````

```
import json,sys
for s in json.load(sys.stdin)['sessions']:
    print(f\"{s['session']:30} {s['status']:8} panes={s['panes']}
agents={s.get('agents',0)}\")
"
```

`status` is **tmux pane recency**, not whether a turn is running. A
session can
read `active` while sitting idle, or `stale` while a long turn runs.
Never decide
safety from this field — step 3's spinner check is the real gate.

Step 2 — resolve

```bash
maw locate <handle> \-\-json
```

`maw locate` exits 1 on an ambiguous match while still printing valid
JSON with
a `candidates` list, so capture it with `|| true` before reading. If the
result
has `candidates`, print them and stop — do not pick one. If `repoPath` is
empty,
report and skip; never guess a path.

Step 3 — bring one

```bash
HANDLE=neo
LABEL="$HANDLE"          # or an explicit label; oracle repos read best
as <name>-oracle

info=$(maw locate "$HANDLE" \-\-json 2>/dev/null) || true
test -n "$info" || { echo "REFUSING: maw locate $HANDLE returned
nothing"; exit 1; }
python3 -c "import json,sys; sys.exit(0 if 'candidates' in
json.load(sys.stdin) else 1)" <<<"$info" && {
  echo "AMBIGUOUS — re-run with an exact name:"
  python3 -c "
import json,sys
for c in json.load(sys.stdin)['candidates']: print(f\"  {c['name']:20}
({c['kind']}) -> {c['action']}\")" <<<"$info"
  exit 1; }

repo=$(python3 -c "import json,sys;
print(json.load(sys.stdin).get('repoPath',''))" <<<"$info")
maw_session=$(python3 -c "import json,sys;
print(json.load(sys.stdin).get('session') or '')" <<<"$info")
test -n "$repo" || { echo "REFUSING: no repoPath for $HANDLE"; exit 1; }
echo "resolved  : $HANDLE -> $repo"

live=""
```

```
if [ -n "$maw_session" ] && tmux has-session -t "$maw_session" 2>/dev/
null; then
  live=1
  # Busy check: Claude's own spinner line — a glyph, a gerund, an
ellipsis, then "(<duration>".
  if tmux capture-pane -t "$maw_session" -p 2>/dev/null | tail -20 | rg -
q '[✻✢✶◐●] .+…\s*\([0-9]'; then
    echo "REFUSING: $maw_session looks mid-turn — bring it only when
idle"; exit 1
  fi
  echo "maw session: $maw_session (LIVE)"
else
  echo "maw session: ${maw_session:-<none>} (not live — checking local
history)"
fi

# Newest transcript for that repo. Encoded path: leading / becomes -,
then / and . become -.
enc=$(echo "$repo" | sed 's|^/|-|; s|[/.]|-|g')
claude_sess=$(ls -t "$HOME/.claude/projects/$enc"/*.jsonl 2>/dev/null |
head -1 \
  | xargs -n1 basename 2>/dev/null | sed 's/\.jsonl$//')

pane=$(herdr workspace create \-\-cwd "$repo" \-\-label "$LABEL" \-\-no-
focus \
  | python3 -c "import json,sys; print(json.load(sys.stdin)['result']
['root_pane']['pane_id'])")
echo "new pane  : $pane"

if [ -n "$live" ]; then
  test -n "$claude_sess" || { echo "REFUSING: $maw_session is live but no
transcript for $repo — investigate before killing"; exit 1; }
  maw kill "$maw_session"; sleep 2
  herdr pane run "$pane" "claude --resume $claude_sess"
  echo "mode      : resumed (killed live maw session first)"
elif [ -n "$claude_sess" ]; then
  herdr pane run "$pane" "claude --resume $claude_sess"
  echo "mode      : resumed local history — nothing was live, nothing was
killed"
else
  herdr pane run "$pane" "claude"; claude_sess="(new)"
  echo "mode      : fresh start — no maw session and no local transcript"
fi

for i in $(seq 8); do
  sleep 5
  agent=$(herdr pane get "$pane" 2>/dev/null \
    | python3 -c "import json,sys; print(json.load(sys.stdin)['result']
['pane'].get('agent','none'))" 2>/dev/null || echo none)
  [ "$agent" = "claude" ] && break
done
[ "$agent" = "claude" ] || { echo "!! claude did not come up in $pane
within 40s — herdr pane read $pane --source visible --lines 20"; exit
1; }
```

````
herdr agent rename "$pane" "$LABEL" >/dev/null 2>&1 || true
echo "OK — $claude_sess is live in $pane, named $LABEL"
```

**Kill before resume, never the other order.** Resuming while the old process
still lives puts two writers on one transcript and they interleave — this
happened for real. `sleep 2` lets the kill settle.

The 40s poll is not padding: direnv on a cold pane can take that long by itself.
If it exceeds it, read the pane rather than assuming failure.

## Step 4 — several

Run the same block per handle, printing a `── name ──` header for each. One
failure must not abort the rest; a busy session is skipped and reported.

**Never sweep every name from step 1 without the user naming them or explicitly
confirming a bulk sweep.** Landing a handle that turns out to be live still kills
its maw side — real, and only somewhat reversible: the tmux pane is gone and the
herdr copy is a resume of the same transcript, not a backup. You cannot know
which mode a name falls into until the script resolves it, so treat every name
in a batch as a potential live-kill.

## After landing

```bash
herdr workspace list
herdr agent list
```

State plainly which of the three modes happened — resumed a killed live session,
resumed local history with nothing killed, or started fresh with no prior history
at all. These are different outcomes. Never paraphrase a refusal as success, and
never call a fresh start "brought".

## Conflict = exit 2 = ASK, never decide

Two guards must run **before** the space is created:

| guard | catches |
|---|---|
| cwd already open | a second space for the same repo |
| label already taken | two *different* repos colliding on one name |
````
````

```
Both print `CONFLICT`, list options, and exit **2** — a distinct code
meaning "a human
decides, then re-run". Exit 1 is a real error; exit 2 is not.

**On exit 2, present the options with `AskUserQuestion` and wait.** Do
not pick one, do
not retry with a guessed label, do not set the override yourself:

- **focus** — use the existing space, create nothing
- **relabel** — re-run with a different label
- **rename** — fix the *existing* space's wrong name first, then re-run
- **anyway** — `HERDR_BRING_FORCE=1`, only when the user explicitly says
so

The label guard exists because of a real miss on 2026-09-14: space `w1R`
sat in
`laris-co/pulse` but was **named** `pulse-oracle`. Landing the genuine
`laris-co/pulse-oracle` passed the cwd check (different repo —
correctly), created the
space, and only then failed at `herdr agent rename` with
`agent_name_taken`, leaving a
live unnamed space behind. A guard that fires after the side effect is
not a guard.

What that implies: a wrong label on one space silently blocks the *right*
repo from ever
taking its own name. When the existing space is the mislabelled one,
prefer **rename**
over **relabel**.

## Moving a body to ANOTHER herdr session (herdr to herdr, not tmux)

A herdr pane cannot cross sessions (`pane move` works inside one
session). Moving a body means
starting the same conversation in the target session, ending the old one,
and never having both alive.
Done 2026-10-05 for 5 live bodies (claude x4, codex x1); /herdr-info
caught the one duplicate.

1. **Read the body.** Scheduled tasks or background work? (they block `/
exit` with a dialog; see traps) Idle? (`herdr --session S agent list` ->
status; never move a `working` body, its
   turn dies.) Its pid and exact flags (`herdr --session S pane process-
info --pane P`, then
   `ps -o command -p PID`), conversation id (`agent_session.value`), cwd.
2. **New space in the target session at that cwd.** `herdr --session T
workspace create --cwd D --label L`,
   or for a worktree that should nest under its repo's main space
   `herdr --session T worktree open --workspace <main space> --path D`.
3. **Arm it.** `herdr --session T pane run NEW 'while kill -0 PID 2>/dev/
null; do sleep 1; done; claude <same flags> --resume ID'`
   (codex: `codex resume ID`, same bypass flags). The wait loop is what
guarantees one copy.
```

````
4. **End the old one.** `herdr --session S agent prompt OLD "/exit"`
(codex: `/quit`).
5. **Check.** `/herdr-info <oracle>` -> `warnings: none`; re-apply the
agent name (it resets on resume);
   close the old space only when EVERY pane in EVERY tab is a bare `zsh`:
atlas lived as a second tab inside
   another space, and closing that space would have killed it.

| trap | avoid |
|---|---|
| `/exit` stops at "You have N unsent feedback draft(s). Enter to review
& send, Esc to discard and exit": the process stays alive, the armed pane
keeps waiting (atlas, 2026-10-05) | the human chooses; never press either
for them (Enter sends feedback outward, Esc discards a draft) |
| `/exit` stops at "Background work is running … 1. Exit and stop tasks ·
2. Move to background and exit · 3. Stay" (homelab, 2026-10-05: a /loop
every 15 min running /cpu-ram-doctor and /cpu-kill) | never pick 2: the
old process keeps running, so the resumed copy becomes a second writer.
Pick 1 only with the owner's OK, then re-create the same scheduled task
in the new copy; to back out, pick 3 (Stay) and close the armed pane |
| a body that holds a live Discord connection (`--channels
plugin:discord`; `DISCORD_STATE_DIR` and `DISCORD_BOT_TOKEN` come from
the folder's `.envrc`) | start the new pane in the same folder so direnv
gives the same env; the old process must be gone first (one connection
per bot) |
| pane ids repeat across sessions (`w8:p2` existed in two) | always `--
session`; address by folder or agent name |
| `maw herdr join` / `break` act on the tab of the pane they run in |
they cannot do this move; use the steps above, or `/herdr-room` for rooms
inside one session |

## Traps

- **The agent name is cleared when the pane's occupant is replaced.**
Measured
  2026-09-14: after a resume re-initialised Claude, the workspace *label*
  survived and the agent *name* went to `None`. The rename at the end can
  therefore come undone a minute later — re-apply with
  `herdr agent rename <pane> <label>` and address panes by `pane_id`, not
name.
- **A plain workspace has NO `worktree` object.** When checking "is this
repo
  already open?", match on each **pane's `cwd`**, not on
  `workspace.worktree.checkout_path`. Only worktree-backed spaces carry a
  `worktree` object at all; a space made by `herdr workspace create` has
none,
  so checkout_path matching silently returns nothing and you create the
  duplicate you were guarding against. Hit for real on nexus-oracle,
2026-09-14.

  ```bash
  herdr pane list | python3 -c "
  import json,sys
  repo=sys.argv[1]
````
````

```
  print([p['workspace_id'] for p in json.load(sys.stdin)['result']
['panes'] if p.get('cwd')==repo])" "$repo"
  ```

- **`maw ls`'s active/idle/stale is not a busy signal** — see step 1.
- **Errors are JSON on stderr with exit 1** for server-side failures, and
some
  paths still exit 0 with an error body. Parse the body, not `$?`.
- **The transcript lookup takes the newest `.jsonl` for the repo path.**
A repo
  with a fork or a worktree copy running concurrently may not give you
the one
  in the named maw pane. Cross-check with `maw peek <handle>`.
- **The spinner regex is tuned to this fleet's status line.** If it ever
refuses
  an obviously idle session, or waves through a working one, say so — the
  pattern needs updating, not a silent override.

What this does NOT do

- Does not touch a session already inside herdr (use `just herdr-fork`);
to move a herdr body to another
  herdr session see § Moving a body to ANOTHER herdr session.
- Does not create a worktree — the space checks out the repo's own
working
  directory. For a worktree, `/herdr-wt`.
- Does not restart or fix a stuck maw session. If the busy check refuses
one
  whose agent actually crashed mid-turn, investigate with `maw peek`
first.

Related

- `/herdr-wt` — worktree per task, `wt/<oracle>-<slug>-<date>`
- `/herdr-ticket` — a GitHub issue in its own worktree and space
- `/herdr-incubate` — another oracle's topic from inside the current repo
- `herdr` — the CLI reference
- `/herdr-room` — pull live agents into one room and back out, inside one
session
- `/herdr-info` — where every body of an oracle (or a space) runs; run it
after any move
```

## herdr-pr/SKILL.md

/herdr-pr: เซ็น PR issue หรือคอมเมนต์ด้วยบล็อก Built by · 268 บรรทัด

```
---
name: herdr-pr
description: Sign a PR, issue, or comment with a "Built by" block — which
oracle and model built it, from which worktree and branch, the session
transcript holding the reasoning, and the herdr address a human or
```
```

````
another agent can talk back to. Use when opening or commenting on a PR
from a herdr pane, when the user says "sign your name", "sign the PR",
"who built this", "how do I reach that session", or when several
worktrees push to one repo and a reviewer needs to know which session
produced the diff. Do NOT use to create a worktree (/herdr-wt), to move a
session into herdr (/herdr-bring), or as a substitute for a real PR
description.
argument-hint: "[pr-number]"
---

# /herdr-pr — sign a PR with a reachable address

A PR body says what changed. This says who to ask when it's wrong.

## CLI

```bash
# 1. name the agent — the name is the address (do this once)
herdr agent rename "$HERDR_PANE_ID" <name>

# 2. print the block (pass your OWN exact model id; the script can't read
it)
~/.claude/skills/herdr-pr/herdr-sign.sh claude-opus-5

# 2b. PR in a repo that is NOT this pane's cwd — an incubated repo, a
worker's
#     checkout: name it, or the block signs the oracle's branch on
someone
#     else's PR
~/.claude/skills/herdr-pr/herdr-sign.sh claude-opus-5 \-\-work "$(ghq
root)/github.com/<org>/<repo>"

# 2c. taller block — one value per line, no quoting rules at all
~/.claude/skills/herdr-pr/herdr-sign.sh claude-opus-5 \-\-work <path> \-
\-tall

# 2d. a commit the working tree is NOT on — a merged PR whose SHA the
rebase
#     rewrote, or a second PR in the same repo
~/.claude/skills/herdr-pr/herdr-sign.sh claude-opus-5 \-\-work <path> \-
\-ref <sha|branch>

# 3. post it
~/.claude/skills/herdr-pr/herdr-sign.sh claude-opus-5 \-\-work <path> |
gh pr comment <N> \-\-body-file -

# 4. anyone reaches the session
herdr agent prompt <name> "question"    # ask; answers with full context
herdr agent read   <name>               # what it printed back
herdr agent focus  <name>               # jump a terminal there
```

Second arg signs another pane: `herdr-sign.sh gpt-5-codex w24:p5` — for a
lead
````

````
signing a worker's PR.

## Agents

claude, omp (Oh My Pi) and codex all store transcripts differently. The
script
handles all three; the encodings differ in ways that look like typos:

| agent | transcript | cwd key |
|---|---|---|
| claude | `~/.claude/projects/<key>/<uuid>.jsonl` | `/` **and** `.` → `-
` |
| omp | `~/.omp/agent/sessions/<key>/<ts>_<uuid>.jsonl` | only `/` → `-`,
wrapped in `--…--` |
| codex | `~/.codex/sessions/YYYY/MM/DD/rollout-<ts>-<uuid>.jsonl` | none
— found by uuid |

herdr reports no session id for omp, so the newest transcript in the
cwd's dir
wins and the id is read back off the filename. Only claude has a memory
dir.

Repo wrapper, when it exists: `just herdr-sign <model-id>`.

## Why the name

| target | `herdr agent get` |
|---|---|
| pane id `w2F:p1` | works, but reassigned after a herdr restart |
| agent name | works once claimed, re-claimable |
| workspace label (what the sidebar shows) | `agent_not_found` |

Unnamed → the script signs a pane id and warns on stderr. A stale pane id
points
reviewers at the wrong agent; that's worse than pointing nowhere.

## Traps

1. **Session, not shell** — paths come from the pane's cwd, never `$PWD`.
Check: `cd /tmp && herdr-sign.sh x | head -8` must show the pane's repo.
2. **Memory keys off the MAIN checkout**, not the worktree.
3. **Parse herdr's JSON with `\x1f`, not tab** — bash collapses runs of
whitespace IFS chars, so an agent with no name or no session id (every
omp agent) shifts every later field one column left. That misfire is
silent: the block still prints, with a cwd in the `session` line.

4. **`--work` is the PR's repo, not `$PWD`** — same discipline as trap 1,
from
   the other side: pass the path explicitly, never let the shell's cwd
decide.
5. **One checkout cannot be on two branches** — `--work` gets the repo
right but
   still reads whichever branch is checked out, so signing PR B while
working
````

   PR A silently cites A's branch and head. Use `--ref`; it never touches the
   tree, which matters when a server is running out of it.
6. **A rebase merge rewrites the SHA** — a signature posted before the merge
   cites a commit that only exists on the un-merged branch. Re-sign merged PRs
   with `--ref <landed-sha>` so the block names what is actually on main.
7. **A flag that only reaches one stanza is a no-op in the other** — `--ref`
   fed the `incubated:` pair, which is suppressed when the PR's repo *is* the
   session's checkout, so same-repo `--ref` validated the sha and then printed
   the tree's branch anyway. Traps 5 and 6 are both same-repo, so the flag did
   nothing in exactly the cases it exists for, and the block still looked
   right. Test every flag in both shapes: with `--work` and without.
8. **`$HERDR_PANE_ID` goes stale after a pane move.** A shell keeps the pane id
   it started with. After `/herdr-room` in/out or `pane move`, the agent has a new
   id, and the script fails with `no herdr agent for pane 'wF:p1'`. Pass the live
   id as the 2nd argument, found by name:
   `herdr agent list | jq -r '.result.agents[] | select(.name=="<you>") | .pane_id'`.
   On 2026-10-07 three floodboy agents hit this in one round:
   gods-eyes wF:p1 → w15:p4, water-api wH:p1 → w15:p6, flood-news wE:p1 → w16:p1.
   Fixed in the script the same morning: when the id came from the env and
   resolves to nothing, the script finds the one live pane whose foreground
   processes include an ancestor of itself, signs that pane, and says so on stderr.
   An explicit pane id that doesn't resolve still fails; nothing is guessed.
9. **A worktree session plus `--work <main checkout>` mislabels a same-repo PR as
   `incubated:`.** The script decides "another repo" by comparing toplevel *paths*,
   not git common dirs, so a linked worktree of the same repo counts as different.
   The `incubated:` stanza then carries the PR's ref, and `worktree:` shows the
   session's own branch. The commit is right; the label is wrong. For a PR in the
   session's own repo, pass `--work <your own worktree path>` with `--ref`. The PR
   commit then lands in `worktree:` with no `incubated:` stanza (checked on
   2026-10-07 for a same-checkout `--ref`). gods-eyes signed 17 floodboy PRs that

````
   night with the mislabel. **Fixed in the script the same morning:** it
compares
   `git rev-parse --git-common-dir`. Another worktree of the same repo
now puts the
   PR's branch and head (or `--ref`) in `worktree:` and adds a `work:`
line for the
   PR checkout. `incubated:` appears only for a genuinely different repo.
Tested:
   same repo other worktree, with and without `--ref`; a different repo;
the same
   checkout with `--ref`; a stale env pane; a wrong explicit pane.

After editing the script, `ls` both printed paths. A block whose paths
404 reads
authoritative and sends people nowhere.

## Two repos, two stanzas

An oracle working an **incubated** repo has its pane cwd in the vault and
the
diff in `$(ghq root)/...`. The session facts and the code facts then come
from
different checkouts, and the default block reports only the session's —
so a
reviewer reads the oracle's branch and HEAD as if they were the PR's.

`--work <path>` adds a second stanza for the repo the PR is in, and picks
up the
`.claude/INCUBATED_BY` breadcrumb `/incubate` leaves there (shown here in
`--tall` form, one value per line):

```yaml
agent:
  who: >-
    neo — laris-co/neo-oracle
  model: >-
    claude-opus-5
  runtime: >-
    claude

worktree:
  cwd: >-
    /opt/Code/github.com/laris-co/neo-oracle/wt/<space>
  branch: >-
    <space>
  wt: >-
    worktree of /opt/Code/github.com/laris-co/neo-oracle
  head: >-
    31d3ceb5 Merge pull request #31 …

incubated:          # only when \-\-work names a different repo
  repo: >-
    nat-build-with-oracle/idea-11sep-fri2026-cc-chat-ui
  branch: >-
````
````

```
    feat/server-side-repository-preferences
  head: >-
    97548ce feat: share repository preferences…
  work: >-
    /opt/Code/github.com/nat-build-with-oracle/<repo>
  link: >-
    …/ψ/incubate/<owner>/<repo>/origin
  by: >-
    neo on 2026-09-16

session:
  id: >-
    8f3ac9b3-…
  transcript: >-          # full block only
    …
  memory: >-              # full block only
    …

herdr:
  address: >-
    neo-digger
  pane: >-
    w2N:p1
  tab: >-
    w2N:t1
  space: >-
    neo-digger-16sep-wed2026
  title: >-
    Find this app v
  socket: >-              # full block only
    …
```

`--compact` folds every value back beside its key — 25 lines instead of 41 on a
four-group block, at the cost of re-quoting (see below):

```yaml
agent:
  who:        'neo — laris-co/neo-oracle'
  model:      'claude-opus-5'

worktree:
  cwd:        '/opt/Code/github.com/laris-co/neo-oracle/wt/<space>'
  head:       '31d3ceb5 Merge pull request #31 from …'
```

Valid YAML either way, so a reviewer reads it and a script parses it:

```bash
gh api repos/<org>/<repo>/issues/comments/<id> \-\-jq .body \
  | sed -n '/```yaml/,/```$/p' | sed '1d;$d' | yq -r '.herdr.address'
```

```
**Values are folded block scalars (`>-`) on their own line, never bare.**
Inside
one, `#` starts no comment and `:` separates no key, so a git subject
survives
verbatim. Bare, `head: 31d3ceb5 Merge pull request #31 from x` parses as
`31d3ceb5 Merge pull request` — silent truncation that still looks
correct.
Omitted fields are absent keys, so `yq` returns `null` rather than an
empty
string.

Compact — the default — is the one place those quoting rules come back:
folding
the value onto the key line re-exposes `#` and `: `, so the transform
single-quotes every value and doubles any apostrophe inside it. Verified
on
`it's a 'quoted' value`, on a `#31` subject, and on a `feat: …` subject.
Reach
for `--tall` when a value is doing something stranger than those.

Four groups, in the order a reviewer asks: **who** built it, from **which
tree**,
**which session** holds the reasoning, and **how to reach it** — plus a
fifth,
`incubated`, naming **which repo** the diff is in, only when `--work`
points at a
repo that is not the session's own checkout.

`work` is the ghq clone, `link` the vault symlink `/incubate` created —
two ways
into the same checkout, and a reader on the fleet needs one of them to
`cd`
there. A repo slug alone says which code to read, not where it is.

Both print in both blocks. `link` was public-omitted for one revision, on
the
grounds that its path carries the private oracle repo name — but the
public block
also prints the oracle `cwd` and its worktree parent, which carry the
same name,
so withholding one path while printing two others bought nothing and only
made
the rule hard to predict.

Merging the two into one `branch`/`head` pair is the bug this replaces: a
lone
branch name gives the reader nothing to attach it to, so they assume it
is the
PR's.

## Visibility decides how much leaks

The script reads `gh repo view <slug> --json isPrivate` for the **work**
repo and
```

```
prints the redacted block unless the repo is proven private:

| repo | block |
|---|---|
| private | everything |
| public | everything except the agent's own storage: transcript, memory
dir, herdr socket |
| unknown (gh failed) | same as public, with a stderr warning |

The line is **code vs. content**. Paths to code — oracle cwd, worktree,
work
checkout, vault symlink — identify which tree produced the diff, and a
reviewer
who can already read the repo gains nothing dangerous from them. Paths to
content — the transcript of everything this session reasoned about, the
memory
dir, the socket that accepts commands — stay out of public view.

Treating unknown as public is deliberate: guessing wrong toward "private"
publishes the transcript path to the internet, and GitHub has mailed it
out
before anyone notices. `--full` forces the complete block once you have
checked.

Even redacted, the block still answers the question the signature exists
for:
which oracle, which model, which commit, and an address that answers
back.

## Don't sign

Commits (use `Co-Authored-By`), third-party repos you do not own, or PRs
nobody
will review. Public repos you DO own are fine — they get the redacted
block
automatically.
```

herdr-pr/herdr-sign.sh

สคริปต์ที่พิมพ์บล็อก Built by (ตัดข้อมูลส่วนตัวเมื่อ repo เป็น public) · 394 บรรทัด

```
#!/usr/bin/env bash
# Emit the "Built by" signature block for a PR/issue comment.
# Describes a herdr SESSION (the pane), not whatever directory the shell
is in,
# and addresses it by agent name.
#
#   herdr-sign.sh <model-id> [pane-id] [--work <path>] [--ref
<committish>]
#                 [--full] [--tall]
#
```

```
# Output is compact by default — one line per field, ~28 lines. --tall
emits the
# same YAML with every value as a folded block scalar on its own line
(~47
# lines), which needs no quoting and survives any value verbatim.
#
# --ref reports a commit the working tree is not sitting on: a merged PR
whose
# SHA the rebase rewrote, or a second PR in the same repo. Without it a
signature
# can only describe whatever branch happens to be checked out, so signing
PR B
# while working PR A meant checking B out first — on a repo with a server
running
# from it.
#
# --work names the repo the PR actually lives in. An oracle working an
INCUBATED
# repo has its pane cwd in the oracle vault and the diff in $(ghq
root)/..., so
# session facts and code facts come from two different checkouts. Without
--work
# the block reports the ORACLE's branch and HEAD on someone else's PR: it
looks
# authoritative and points at an unrelated commit.
#
# Visibility decides how much local layout is printed. A public repo gets
a
# redacted block — no absolute paths, no transcript, no memory dir, no
socket, no
# private oracle slug — because those map a machine reviewers cannot
reach anyway.
# Unknown visibility redacts too; --full overrides once you have checked.
set -uo pipefail

MODEL=""; pane=""; work=""; ref=""; force_full=0; compact=1;
positional=()
while [ $# -gt 0 ]; do
    case "$1" in
        --work|-w) work="${2:-}"; shift 2 ;;
        --ref)     ref="${2:-}"; shift 2 ;;
        --full)    force_full=1; shift ;;
        --compact) compact=1; shift ;;   # default; kept so scripts that
pass it still work
        --tall)    compact=0; shift ;;   # one value per line, as folded
block scalars
        --help|-h) sed -n '2,16p' "$0"; exit 0 ;;
        *)         positional+=("$1"); shift ;;
    esac
done
MODEL="${positional[0]:-<your exact model id>}"
pane="${positional[1]:-${HERDR_PANE_ID:-}}"

agent_facts() {
```

```
    herdr agent get "$1" 2>/dev/null | python3 -c '
import json, sys
try:
    a = json.load(sys.stdin)["result"]["agent"]
except Exception:
    sys.exit(1)
print("\x1f".join([
    a.get("name") or "",
    a.get("pane_id") or "",
    a.get("tab_id") or "",
    a.get("workspace_id") or "",
    a.get("agent") or "",
    a.get("terminal_title_stripped") or "",
    (a.get("agent_session") or {}).get("value", ""),
    a.get("cwd") or "",
]))' 2>/dev/null
}
facts=$(agent_facts "$pane")

# HERDR_PANE_ID is fixed when the shell starts. After /herdr-room or
`pane move`
# the agent lives at a new pane id while the env still names the old one,
and
# three agents failed here in one round (2026-10-07). When the id came
from the
# env and resolves to nothing, find the live pane whose foreground
processes
# include an ancestor of this script. That is the session actually
running it.
# Exactly one match, or keep failing: guessing a pane signs someone
else's work.
if [ -z "$facts" ] && [ -z "${positional[1]:-}" ]; then
    live=$(python3 - "$$" <<'PY' 2>/dev/null
import json, subprocess, sys
def run(*a):
    return subprocess.run(a, capture_output=True, text=True).stdout
anc, pid = set(), int(sys.argv[1])
while pid > 1 and pid not in anc:
    anc.add(pid)
    out = run("ps", "-o", "ppid=", "-p", str(pid)).strip()
    pid = int(out) if out.isdigit() else 0
hits = []
for a in json.loads(run("herdr", "agent", "list"))["result"]["agents"]:
    try:
        pi = json.loads(run("herdr", "pane", "process-info", "--pane",
a["pane_id"]))["result"]["process_info"]
    except Exception:
        continue
    pids = {p.get("pid") for p in pi.get("foreground_processes") or []}
    pids.add(pi.get("foreground_process_group_id"))
    if pids & anc:
        hits.append(a["pane_id"])
print(hits[0] if len(hits) == 1 else "")
PY
```

```
)
    if [ -n "$live" ]; then
        echo "herdr-sign: HERDR_PANE_ID '${pane:-<unset>}' is stale (the
pane moved); signing the live pane ${live}, found from this process's
ancestors." >&2
        pane="$live"
        facts=$(agent_facts "$pane")
    fi
fi

# \x1f, not tab: bash collapses runs of whitespace IFS chars, so an agent
with no
# session id or no name silently shifts every later field one column
left.
IFS=$'\x1f' read -r name pane_id tab wsid agent title sess pane_cwd
<<<"$facts"

# No pane means no session to point at. Emitting a block of "unknown"
would be the
# exact failure this block exists to prevent: authoritative-looking,
reaching nobody.
if [ -z "${pane_id:-}" ]; then
    echo "herdr-sign: no herdr agent for pane '${pane:-<unset>}'." >&2
    echo "  Run this from inside a herdr pane (HERDR_PANE_ID set), or
pass one:" >&2
    echo "    herdr-sign.sh <model-id> <pane-id>     # herdr agent list"
>&2
    exit 1
fi

# The block describes the session, so every path is resolved from the
PANE's cwd.
# Signing $PWD would describe whatever directory the shell wandered into.
root="${pane_cwd:-$PWD}"

# A named agent is addressable by name; an unnamed one only by pane id,
which does
# not survive a herdr restart. Claim a name before signing.
addr="${name:-${pane_id:-<pane>}}"
label=$(herdr workspace get "${wsid:-x}" 2>/dev/null |
    python3 -c 'import json,sys; print(json.load(sys.stdin)["result"]
["workspace"].get("label",""))' 2>/dev/null)

repo_root=$(git -C "$root" rev-parse --show-toplevel 2>/dev/null || echo
"$root")
common=$(git -C "$root" rev-parse --git-common-dir 2>/dev/null)
case "$common" in
    /*) main=$(cd "$(dirname "$common")" && pwd) ;;
    *)  main=$(cd "$repo_root/$(dirname "${common:-.git}")" 2>/dev/null
&& pwd || echo "$repo_root") ;;
esac
[ "$main" = "$repo_root" ] && where="main checkout" || where="worktree of
${main}"
```

```
# Two passes: BSD sed has no lazy quantifier, so strip .git before
matching org/repo.
slug=$(git -C "$root" remote get-url origin 2>/dev/null | sed -E 's|
\.git$||; s|.*[:/]([^/]+/[^/]+)$|\1|')
oracle=$(basename "$main"); oracle=${oracle%-oracle}

# The work repo: where the diff is. Defaults to the session's own
checkout, so an
# oracle signing its own PR behaves exactly as before.
work_root="$root"
[ -n "$work" ] && work_root=$(git -C "$work" rev-parse --show-toplevel
2>/dev/null || echo "$work")
work_slug=$(git -C "$work_root" remote get-url origin 2>/dev/null | sed -
E 's|\.git$||; s|.*[:/]([^/]+/[^/]+)$|\1|')
work_branch=$(git -C "$work_root" branch --show-current 2>/dev/null)
work_head=$(git -C "$work_root" log --oneline -1 2>/dev/null)
if [ -n "$ref" ]; then
    # Fail loudly: a typo'd ref would otherwise silently fall back to the
    # checkout and sign the wrong commit under a confident-looking block.
    if ! work_head=$(git -C "$work_root" log --oneline -1 "$ref" 2>/dev/
null); then
        echo "herdr-sign: --ref '$ref' does not resolve in $work_root"
>&2
        exit 1
    fi
    work_branch="$ref"
fi

# Same repo, other checkout: a session in a linked worktree signing a PR
whose
# --work is the main checkout (or another worktree) of the SAME repo.
Comparing
# toplevel paths called that "another repo" and printed the PR under
# `incubated:` (17 floodboy PRs, 2026-10-07). Compare git common dirs
instead:
# one repo has exactly one, however many worktrees it has.
common_dir() { (cd "$1" 2>/dev/null && cd "$(git rev-parse --git-common-
dir 2>/dev/null)" 2>/dev/null && pwd -P); }
same_repo=0
if [ "$work_root" != "$root" ]; then
    a_common=$(common_dir "$root"); b_common=$(common_dir "$work_root")
    [ -n "$a_common" ] && [ "$a_common" = "$b_common" ] && same_repo=1
fi

# The session tree's own branch and head. --ref names the commit the
signature
# CITES, and when the PR's repo is the session's own checkout there is no
# incubated stanza to carry it — so it has to land here or the flag is a
silent
# no-op in exactly the same-repo case traps 5 and 6 tell people to use it
for.
# The same holds for another worktree of the same repo: --work then names
the
# checkout the PR is in, so its branch and head (or --ref) are what get
```

```
cited.
tree_branch=$(git -C "$root" branch --show-current 2>/dev/null)
tree_head=$(git -C "$root" log --oneline -1 2>/dev/null)
if { [ -n "$ref" ] && [ "$work_root" = "$root" ]; } || [ "$same_repo" =
1 ]; then
    tree_branch="$work_branch"
    tree_head="$work_head"
fi
work_line=""
[ "$same_repo" = 1 ] && work_line="
  work: >-
    ${work_root}"

# /incubate drops this breadcrumb into every repo it clones. It is the
only record
# tying a foreign checkout back to the oracle working it.
incubated=""
if [ -f "$work_root/.claude/INCUBATED_BY" ]; then
    when=$(sed -n 's/^date: //p' "$work_root/.claude/INCUBATED_BY" | head
-1)
    incubated="incubated by ${oracle}${when:+ on }${when}"
fi

# Where the work repo actually is. Naming the repo without its checkout
tells a
# reader which code to look at but not how to reach it — and for an
incubated
# repo the ghq clone and the vault symlink are two different ways in, so
both
# are printed. The symlink is the oracle-side handle /incubate created.
work_wt=""
work_common=$(git -C "$work_root" rev-parse --git-common-dir 2>/dev/null)
case "$work_common" in
    /*) work_main=$(cd "$(dirname "$work_common")" && pwd) ;;
    *)  work_main=$(cd "$work_root/$(dirname "${work_common:-.git}")" 2>/
dev/null && pwd || echo "$work_root") ;;
esac
[ "$work_main" != "$work_root" ] && work_wt="worktree of ${work_main}"

incubate_link=""
if [ -n "$work_slug" ] && [ -L "$root/ψ/incubate/$work_slug/origin" ];
then
    incubate_link="$root/ψ/incubate/$work_slug/origin"
fi

# Redact by default, open up only on proof of privacy. Guessing "private"
wrong
# publishes a machine layout to the internet, and GitHub has already
mailed it out
# by the time anyone notices.
visibility="unknown"
if [ -n "$work_slug" ]; then
    case "$(gh repo view "$work_slug" --json isPrivate --jq .isPrivate
2>/dev/null)" in
```

```
        true)  visibility="private" ;;
        false) visibility="public"  ;;
    esac
fi
redact=1
[ "$visibility" = "private" ] && redact=0
[ "$force_full" = "1" ] && redact=0

# Each agent stores transcripts under its own scheme, and the cwd
encodings differ
# in ways that look like typos but aren't. Claude replaces "/" AND "."
with "-";
# omp replaces only "/" and wraps the whole thing in "--". Guessing wrong
prints a
# path that does not exist, which is the one thing this block must never
do.
claude_key() { local p=${1//\//-}; echo "${p//./-}"; }            # /a/
b.c -> -a-b-c
omp_key()    { local p=${1#/}; echo "--${p//\//-}--"; }           # /a/
b.c -> --a-b.c--
memory="~/.claude/projects/$(claude_key "$main")/memory"

case "$agent" in
    claude)
        transcript="${CODEX_COMPANION_TRANSCRIPT_PATH:-$HOME/.claude/
projects/$(claude_key "$root")/${sess}.jsonl}"
        ;;
    omp)
        # herdr reports no session id for omp, so take the newest
transcript in the
        # cwd's session dir and read the id back off its filename.
        dir="$HOME/.omp/agent/sessions/$(omp_key "$root")"
        transcript=$(ls -t "$dir"/*.jsonl 2>/dev/null | head -1)
        [ -n "$transcript" ] && sess=$(basename "$transcript" .jsonl)
        memory="- (omp: session dir is $dir)"
        ;;
    codex)
        # Date-partitioned rollouts; herdr gives the uuid, the date is in
the name.
        transcript=$(find "$HOME/.codex/sessions" -name "rollout-*${sess}
*.jsonl" 2>/dev/null | head -1)
        memory="- (codex: rollouts only)"
        ;;
esac
transcript="${transcript:-unknown}"
sess="${sess:-unknown}"

# Valid YAML, so the block is machine-readable with `yq` and not just
aligned
# text. Every value is a folded block scalar (`>-`) on its own line:
inside one,
# "#" starts no comment and ":" separates no key, so a git subject like
# "feat: share… #31" survives verbatim. Quoted one-liners needed escaping
rules
```

```
# per value and truncated silently at the first " #" when one was missed.

# The vault symlink: the oracle-side handle on this checkout. Prints in
both
# blocks — the public block already carries cwd and the worktree parent,
which
# hold the same private slug, so withholding this one path bought
nothing.
link_line=""
[ -n "$incubate_link" ] && link_line="
  link: >-
    ${incubate_link}"

# One composed block, not two heredocs. Two drifted every time a field
changed:
# a line added to the full block silently never reached the public one.
# Groups answer the four questions a reviewer actually asks in order —
who built
# it, from which tree, against which repo, and how do I reach them.

sess_private=""
herdr_private=""
if [ "$redact" = "0" ]; then
    sess_private="
  transcript: >-
    ${transcript/#$HOME/\~}
  memory: >-
    ${memory}"
    herdr_private="
  socket: >-
    ${HERDR_SOCKET_PATH:-~/.config/herdr/herdr.sock (default; not run
from a pane)}"
fi

# The incubated group only exists when the PR's repo is not the session's
own:
# a different repo, not just a different worktree of the same one.
incubated_group=""
if [ "$work_root" != "$root" ] && [ "$same_repo" = 0 ]; then
    incubated_group="
incubated:
  repo: >-
    ${work_slug:-$work_root}
  branch: >-
    ${work_branch:--}
  head: >-
    ${work_head:--}
  work: >-
    ${work_root}${work_wt:+
  wt: >-
    ${work_wt}}${link_line}${incubated:+
  by: >-
    ${incubated#incubated by }}
"
```

```
fi

block=$(cat <<SIG
## Built by

\`\`\`yaml
agent:
  who: >-
    ${oracle} — ${slug}
  model: >-
    ${MODEL}
  runtime: >-
    ${agent:--}

worktree:
  cwd: >-
    ${root}
  branch: >-
    ${tree_branch}
  wt: >-
    ${where}
  head: >-
    ${tree_head}${work_line}
${incubated_group}
session:
  id: >-
    ${sess}${sess_private}

herdr:
  address: >-
    ${addr}
  pane: >-
    ${pane_id:--}
  tab: >-
    ${tab:--}
  space: >-
    ${label:-${wsid:--}}
  title: >-
    ${title:--}${herdr_private}
\`\`\`
${redact:+}
$([ "$redact" = "1" ] && printf '%s' "Agent-private storage — transcript,
memory dir, herdr socket — is omitted: this
repo is ${visibility}.")

**Reach this session** — from any pane, agent, or machine with the herdr
CLI:

\`\`\`bash
herdr agent prompt ${addr} "question about this PR"
herdr agent read   ${addr}
herdr agent focus  ${addr}
herdr agent attach ${addr}
\`\`\`
```

```
SIG
)

# Compact (the default) folds "key: >-" + its indented value back onto
one line.
# The value must be re-quoted on the way: inline YAML reads " #" as a
comment and
# ": " as a key separator, which is exactly what the block-scalar form
avoids, so
# this transform is the only place those rules come back. --tall skips
it.
if [ "$compact" = "1" ]; then
    block=$(printf '%s\n' "$block" | awk '
        match($0, /^[ ]+[A-Za-z_]+: >-$/) {
            key = $0; sub(/: >-$/, ":", key)
            indent = key; sub(/[^ ].*$/, "", indent)
            name = key; sub(/^[ ]+/, "", name)
            if ((getline value) > 0) {
                sub(/^[ ]+/, "", value)
                gsub(/\x27/, "\x27\x27", value)
                printf "%s%-11s \x27%s\x27\n", indent, name, value
                next
            }
            print key
            next
        }
        { print }
    ')
fi
printf '%s\n' "$block"


if [ "$redact" = "1" ]; then
    echo "herdr-sign: ${work_slug:-this repo} is ${visibility} — printed
the redacted block (no local paths)." >&2
    [ "$visibility" = "unknown" ] && echo "  gh could not read
visibility. Use --full only after confirming the repo is private." >&2
else
    [ -f "${transcript/#\~/$HOME}" ] || echo "warning: transcript path
does not exist: $transcript" >&2
fi
if [ -n "$work" ] && [ -z "$work_slug" ]; then
    echo "warning: --work ${work} has no origin remote; the repo line
falls back to a local path." >&2
fi
if [ -z "${name:-}" ]; then
    echo "warning: this agent has no name, so the block addresses a pane
id that dies with the herdr server." >&2
    echo "  herdr agent rename ${pane_id:-\$HERDR_PANE_ID} ${label:-
<name>}" >&2
fi
```

herdr-dissolve/SKILL.md

/herdr-dissolve: เก็บงาน เขียน tombstone แล้วปิด worktree · 295 บรรทัด

```
---
name: herdr-dissolve
description: The reverse of /herdr-wt, and the one skill for winding
herdr down — from a single worktree up to the whole machine. A worktree
whose agent did work gets saved first (its own agent commits, pushes,
opens a signed PR and reports to its PARENT, the repo's main-checkout
agent); a worktree with nothing in it, a merged or pushed one, or a space
on a missing folder is removed straight away through herdr so the sidebar
and git agree; idle agents on the main checkout are quit and their panes
closed; checkouts that are only behind are fast-forwarded. Always shows
the plan and asks before anything is closed or removed, and never closes
the pane running it. Modes — `<wt|pane ...>` (named), `--all-but-me`
(this repo down to one pane), `--sweep` (whole machine, was /herdr-clean-
up-sync), `--sync`, `self`. Use when the user says "dissolve", "wind down
this worktree", "finish and close", "commit push pr and close", "done
with this wt", "close all panes and wt, only me", "clean up herdr",
"clean up the sidebar", "sync", "too many ✓", "close the done ones",
"everything is ↓", /herdr-dissolve or /herdr-clean-up-sync. Do NOT use to
check whether ONE session is safe to close (use /herdr-done), to create
worktrees (/herdr-wt), or to move sessions between machines (/herdr-
bring).
argument-hint: "[<wt|pane> ...] | --all-but-me | --sweep [--host
<ssh>]... | --sync [--all] | self"
---

# /herdr-dissolve: wind herdr down, saving work first

`/herdr-wt` makes a branch, a locked worktree, a space and an agent. This
undoes it in
the right order, at any scale. **The agent that did the work saves it and
reports first.**
Only then is anything removed. Nothing is lost: work lands in a PR, the
summary reaches
the parent, the transcript stays in `~/.claude/projects`, and a tombstone
records what
ended and why.

Merged 2026-10-29 from `/herdr-dissolve` (save, then remove) and `/herdr-
clean-up-sync`
(sweep and sync). Nat: "merge … to one". The old texts are in
`SKILL.md.bak-20261029-premerge`
here and `../herdr-clean-up-sync/SKILL.md.bak-20261029-merged-into-
dissolve`.

## Modes

| call | scope | typical ask |
|---|---|---|
| `/herdr-dissolve <wt\|pane> [...]` | the named worktrees | "done with
this wt" |
```

`/herdr-dissolve --all-but-me`	every pane and worktree of the CURRENT repo, except the pane running this	"close all panes and wt, just only 1 here"
`/herdr-dissolve --sweep [--host H]`	every repo on the machine (`ghq list -p`)	"clean up the sidebar", "too many ✓"
`/herdr-dissolve --sync [--all]`	checkouts that are only behind	"everything is ↓"
`/herdr-dissolve self`	the worktree you are running in	an agent finishing its own task

`--sweep` runs sync's plan too, as the old `/herdr-clean-up-sync` did. Every mode goes through the same four steps: **survey, plan, ask, act**.

The engine: `fleet`

Everything runs on `fleet` (`tools/fleet-tui/worktrees.ts` in laris-co/neo-oracle, installed as `fleet` by `just fleet-install`). Every command prints a plan until `--go`.

```
fleet audit  [\-\-host H] [\-\-repo X] [\-\-pick wt1,wt2] [\-\-idle-agents] [\-\-idle-shells] [\-\-min-age N] [\-\-no-fetch] [\-\-json]
fleet clean  [\-\-host H] [\-\-repo X] [\-\-pick wt1,wt2] [\-\-tier A,B,C,D] [\-\-idle-agents] [\-\-idle-shells] [\-\-pr] [\-\-go]
fleet sync   [\-\-host H] [\-\-all] [\-\-go]
fleet send <pane> "<text>"      fleet watch <pane>      fleet tail <pane>
fleet close <pane>              fleet snapshot \-\-commit
```

No `--host` = this machine. `--host white.local` runs the same code there over ssh; it needs bun, git, gh and ghq, nothing from fleet.

`fleet <cmd> --help` is not help. Unknown flags are ignored, so `fleet clean --help` ran a full-machine `clean` plan (2026-10-08, stopped at repo 445/1278). Without `--go` it changes nothing, but never type a flag to "see the usage". `fleet` with no arguments prints the usage.

1. Survey (read-only)

Scope it to the mode. **With no `--repo` and no `--pick`, `audit` and `clean` walk every repo on the machine** (1278 on m5, 2026-10-08). Only `--sweep` may run unscoped.

```bash
# named / \-\-all-but-me: this repo only
repo=$(dirname "$(git rev-parse --path-format=absolute --git-common-dir)")
```

```
slug=$(git -C "$repo" remote get-url origin | sed -E 's#.*[:/]([^/]+/
[^/.]+)(\.git)?$#\1#')
fleet audit \-\-repo "$slug" \-\-idle-agents \-\-idle-shells \-\-min-age
0 \-\-no-fetch \-\-json > "$(mktemp -t dissolve).json"
# \-\-sweep
fleet audit [\-\-host H] \-\-idle-agents \-\-idle-shells \-\-min-age 0 \-
\-json > "$(mktemp -t cleanup-<host>).json"
fleet sync [\-\-host H] \-\-json > "$(mktemp -t sync-<host>).json"
```

For `--all-but-me`, also list the panes on the main checkout, since `fleet audit` covers
worktrees only:

```bash
me=$HERDR_PANE_ID
herdr pane list | jq -r \-\-arg r "$repo" \-\-arg me "$me"
'.result.panes[]
  | select(((.foreground_cwd // .cwd) == $r) and .pane_id != $me)
  | "\(.pane_id)\t\(.agent // "shell")"'
```

Rows come back in tiers:

tier	means	removing it loses
A	worktree folder gone, or a herdr space on a missing folder	nothing
B	merged (in the default branch, or a merged PR whose head IS this HEAD), clean	nothing
C	pushed, clean, not merged	nothing; the branch stays
D	uncommitted work or local-only commits	the work, unless saved first
K	an agent is **working/blocked** in it, or not picked	kept, never touched

For every row with an agent, read what it last said:

```bash
herdr pane read <pane> \-\-source recent-unwrapped \-\-lines 40 | tail
-12
```

✓ is not done. ✓ means the agent finished its turn, not its task. An agent that ended
on a question ("A or B?"), a plan waiting for approval, an AskUserQuestion popup, or "want
me to merge?" is **waiting on the human**: list it, never offer it, and never type into
its pane (keystrokes answer the popup).

2. Decide each row

| row | action |

````
|---|---|
| A | prune, nothing to save |
| B / C, no agent | remove (step 4b) |
| B / C, idle ✓ agent that reads finished | quit it, remove (step 4b) |
| D with an idle agent | **save first** (step 4a): its own agent commits,
PRs, reports, then remove |
| D with no agent | rescue to a draft PR (step 4c), only if asked |
| clean worktree with zero own commits | remove; nothing to save, no PR |
| idle agent on the MAIN checkout (`--all-but-me`) | `/exit`, then `fleet
close <pane>` (step 4d). Never removes the checkout |
| bare shell on the main checkout | `fleet close <pane>` |
| working / blocked / waiting agent | K: report it, touch nothing |
| the pane running this skill | always kept |
| behind only (`would ff`) | `fleet sync --go` (step 4e) |

## 3. Show the plan, then ask

One table per machine, grouped:

- **Save then remove**: D rows with an agent, and what is dirty in each.
- **Safe now**: A, plus B/C and empty worktrees with no agent. Nothing is
lost.
- **Done agents**: B/C with an idle ✓ agent whose last output reads
finished. Quitting keeps
  the transcript; `/herdr-bring` or `claude --resume` revives it.
- **Main-checkout panes** (`--all-but-me`): pane id, agent or shell,
idle.
- **Waiting on you**: listed, not offered.
- **Unsaved, no agent (D)**: listed; rescue is offered separately.
- **Behind (sync)**: `would ff` rows, and SKIP reasons (modified,
diverged, no upstream).

Then AskUserQuestion (multiSelect), one option per group that has rows.
Put the row
COUNT and the repos in each description (`removes 23 worktrees in 11
repos`): a label alone
hides that `--sweep` is machine-wide. For a long "Done agents" list, let
the human drop
names in "Other".

## 4. Act on exactly what was picked

### 4a. Save first (D rows with an agent)

**Who is the parent:** the agent in the repo's main checkout, where `/
herdr-wt` ran.

```bash
repo=$(git -C "$wt" rev-parse \-\-path-format=absolute \-\-git-common-dir
| xargs dirname)
herdr pane list | jq -r \-\-arg r "$repo" '.result.panes[]
  | select((.foreground_cwd // .cwd) == $r and .agent) | .pane_id' | head
-1
```
````

````
Usually that is you. If no agent runs there, the parent report becomes a
file in main's
vault, `$repo/ψ/inbox/handoff/<date>_dissolve_<wt>.md`, which the lead
reads to the human.

Brief each agent with `fleet send <pane> "<brief>"`, never `herdr agent
prompt` (it
prefix-matches names). The brief asks it to:

- Commit everything that is work, on this worktree's branch. Never
`.envrc`, `.codex/`,
  `.env*`, shell snapshots or credentials. Hooks off: `git -c
core.hooksPath=/dev/null commit`.
- Push the branch and open a PR signed with `/herdr-pr`. If nothing
changed, say so and skip the PR.
- Tell the parent with `fleet send <parent-pane> "…"`, at most 6 lines:
PR url, what was
  done, what is verified, what is left open, anything the parent must
pick up.
- Finish with the single line `DISSOLVE-READY <pr-url|no-changes>`.

Brief them all, then `fleet watch` them all. When each goes idle, `fleet
tail <pane>` and
confirm the `DISSOLVE-READY` line. If one fails, don't stop the others.
It is now a B/C
row: go to 4b.

### 4b. Tombstone, then remove (A/B/C, done agents, saved D)

Removing a worktree ends its function, not its record. Before any `--go`
that removes a
B/C worktree, write its tombstone:

```bash
python3 ~/.claude/skills/herdr-dissolve/tombstone.py <worktree-path>
# dry run
python3 ~/.claude/skills/herdr-dissolve/tombstone.py <worktree-path> \-\-
write \-\-reason "merged PR #12"
```

It records what the worktree was (branch, born, own commits), why it
ended, the successor
(PR url or `main@sha`) and where the record lives (branch kept, session
ids with transcript
paths). It writes `<main checkout>/ψ/memory/tombstones/
<date>_<label>.md`, or
`~/.herdr/tombstones/<repo>/` when the repo has no ψ. It never overwrites
and never removes.

Read its `BLOCKER:` lines first:

- uncommitted or untracked files: not in any commit, so removal loses
them. Save them (4a) or rescue (4c).
````

```
- HEAD not in main and no merged PR: not merged, so not a ceasing
worktree.
- LOCKED: `/herdr-wt` and `wtui` lock every worktree they cut. Unlock
deliberately
  (`git -C <repo> worktree unlock <path>`), do not force.

`/herdr-tree` shows the phase: `cease` rows are ready, `stand!` rows
(main moved on, or
commits landed after the PR merged) are not.

**Why:** a removed thing with no death date and no successor is a record
that lost its
chain (2026-08-22: 33 of 39 archived skills had neither). The tombstone
is the chain.

```bash
fleet clean [\-\-host H] \-\-repo X \-\-pick <wt1,wt2,...> \-\-idle-
agents \-\-tier B,C,D \-\-go   # named
fleet clean [\-\-host H] \-\-idle-shells \-\-go
# sweep: safe now
```

`--pick` switches off the age rule for those rows only. It still re-
checks just before
acting: a working agent, new edits or new local commits turn the row into
a SKIP. It quits
an idle agent with ctrl-c (transcript kept), and removes the worktree and
closes the space
through `herdr worktree remove`, so the sidebar agrees with git. **The
branch stays** until
its PR merges.

Run it from a pane **outside** the worktree. The **whole space** goes,
every pane in every
tab.

4c. Rescue D with no agent (only if asked)

```bash
fleet clean [\-\-host H] \-\-pick <wt,...> \-\-tier D \-\-pr \-\-go
```

Commits to `rescue/<wt>`, pushes, opens a signed draft PR, removes the
worktree. Guards:
`.codex/`, `.env*`, shell snapshots and credentials are never committed;
anything key-like
in the staged diff stops it before pushing; repo hooks never run; a file
over 10 MB keeps
its worktree; a rescue over 50 MB total is refused.

4d. Main-checkout panes (`--all-but-me`)

```bash
herdr agent prompt <pane> '/exit'          # an idle agent; its
```
```

````
transcript stays
fleet close <pane>                          # or: herdr pane close <pane>
```

Never the pane running this skill. Never `rm` or remove the main
checkout. If a pane was
the last one in its space, the space closes with it.

### 4e. Sync

```bash
fleet sync [\-\-host H] \-\-go
```

It only fast-forwards. `SKIP diverged ↑N` means someone has local commits
there, which is
a human's call. It never merges, rebases or resets.

## 5. Record and report

```bash
fleet snapshot \-\-commit        # the fleet history sees the change
(local machine)
```

Report: before and after counts of worktrees and panes, PRs opened (with
the parent's
acknowledgement), spaces closed, agents quit (pane id and session id so
they can be
revived), fast-forwards (`↓N → sha`), tombstones written, and every SKIP
with its reason.
Name what is left for the human: waiting agents, D rows, diverged
checkouts.

## Self mode: `/herdr-dissolve self` (inside the worktree)

Do 4a yourself: commit, push, signed PR, report to the parent. You cannot
remove the
worktree you run in, so hand that step off, then stop working:

```bash
fleet send <parent-pane> "DISSOLVE-READY <pr-url> — please: /herdr-
dissolve $(basename "$PWD")"
```

## Guards and traps

1. **Never dissolve a working agent, and never one waiting on the
human.** Read its last message.
2. **Fresh files count.** `.envrc`, `.DS_Store` and `._*` are junk to
fleet
   (`worktrees.ts:49,71`): a modified `.envrc` reads `dirty: 0` and goes
with the worktree.
   Any other uncommitted file makes it `SKIP uncommitted`. Gitignored
````
````

```
data under 50 MB
   (`IGNORED_CAP`) is deleted too, so copy anything you want out of
`.tmp/` first; over
   50 MB the worktree is kept. Do not bypass that check.
3. **Public repo:** before any PR or `--pr`, list what is about to be
committed
   (`git -C <wt> status --porcelain -uall | head -40`). A 2026-09-24 run
by hand pushed
   Codex session logs into public PRs; closed PRs keep their commits on
GitHub, and only
   GitHub Support can purge them.
4. **A space can hold an agent in a second tab or pane.** 2026-10-05:
atlas had been moved
   into another space as its own tab; closing that space would have
killed it. Before
   closing a space, list ALL its panes in ALL tabs (`herdr pane list --
workspace W`, then
   `herdr pane process-info --pane P`): close it only when every one is a
bare shell or is
   in the plan.
5. **Removing a worktree with `git` alone leaves a herdr space on a
missing folder.**
   `fleet clean` goes through `herdr worktree remove`; tier A closes
earlier orphans.
6. **Other owners.** On m5, `arra-oracle-v4` and `odoo-yd` belong to
other agents' work.
   `--sweep` uses `--skip arra-oracle-v4,odoo-yd` unless the user names
them.
7. **Idle shells on main checkouts** are not worktrees. `clean` never
removes a main
   checkout. Outside `--all-but-me`, closing those spaces is a separate
   `herdr workspace close <id>` the user must ask for.
8. **"local-only commits" may be MAIN's, not the worktree's.** A worktree
cut from a local
   `main` that is ahead of `origin` reports main's unpushed commits as
its own, so a clean,
   empty worktree reads as D. Check `git -C <repo> log --oneline origin/
main..main` first:
   if those are the same commits, push main (the human's call) or treat
the worktree by
   `git -C <wt> log main..HEAD` instead. Measured 2026-10-29 in aiexpo-
oracle: main
   `[ahead 5]`, and both empty worktrees showed `5 local-only commits`.
9. **Survey files go to `mktemp`, not `.tmp/`.** Not every repo ignores
`.tmp/`
   (aiexpo-oracle does not), and a survey file there shows up as
untracked.
10. **Nothing merges here.** PRs wait for a human. Removing a worktree
neither merges nor
   deletes its branch.

## Related

- `/herdr-wt`: creates what this dissolves
```

```
- `/herdr-issue`, `/herdr-ticket`: issue first, then a worktree; dissolve
when the PR is up
- `/herdr-done`: is ONE session safe to close (read-only)
- `/herdr-pr`: the signature on the PR
- `/herdr-tree`: the phase of each worktree (`cease`, `stand!`)
```

herdr-dissolve/tombstone.py

สคริปต์ที่เขียน tombstone ก่อนลบ worktree (dry run ถ้าไม่ใส่ --write) · 123 บรรทัด

```
#!/usr/bin/env python3
"""tombstone: write the record of a worktree BEFORE it is removed (ดับไป !
= deleted).

A worktree ceases as an active thing; what it was, why it ended, who
succeeds it and
where its record lives must survive it. Dry run unless --write. Never
overwrites a
tombstone. Never removes anything.

  tombstone.py <worktree-path> [--reason TEXT] [--write]
"""
import json, subprocess, sys, os, glob, datetime

def run(cmd, cwd=None):
    try:
        return subprocess.run(cmd, cwd=cwd, capture_output=True,
text=True, timeout=15).stdout.strip()
    except Exception:
        return ""

def main():
    args = [a for a in sys.argv[1:]]
    write = "--write" in args
    reason = ""
    if "--reason" in args:
        i = args.index("--reason"); reason = args[i + 1]; del args[i:i +
2]
    args = [a for a in args if a != "--write"]
    if not args: print(__doc__); sys.exit(2)
    path = os.path.realpath(args[0])

    wts, cur = [], {}
    for line in run(["git", "-C", path, "worktree", "list", "--
porcelain"]).split("\n"):
        if not line:
            if cur: wts.append(cur); cur = {}
            continue
        k, _, v = line.partition(" "); cur[k] = v or True
    if cur: wts.append(cur)
    me = next((w for w in wts if os.path.realpath(w["worktree"]) ==
```

```
path), None)
    if not me or me is wts[0]:
        print("refusing: not a linked worktree (a main checkout is never
tombstoned)"); sys.exit(2)
    main_path, main_sha = wts[0]["worktree"], wts[0]["HEAD"]
    repo = os.path.basename(main_path)
    branch = (me.get("branch") or "").replace("refs/heads/", "") or
"(detached)"
    head = me["HEAD"]
    now = datetime.datetime.now().astimezone()

    # herdr: label + agent sessions
    label, sessions = os.path.basename(path), []
    try:
        for w in json.loads(run(["herdr", "workspace", "list"]))
["result"]["workspaces"]:
            cp = (w.get("worktree") or {}).get("checkout_path", "")
            if cp and os.path.realpath(cp) == path:   # realpath("") is
the cwd: unbound spaces matched
                label = w.get("label", label); wid = w["workspace_id"]
                for p in json.loads(run(["herdr", "pane", "list"]))
["result"]["panes"]:
                    if p.get("workspace_id") == wid and
p.get("agent_session"):
                        sid = p["agent_session"]["value"]
                        tr = glob.glob(os.path.expanduser(f"~/.claude/
projects*/*/{sid}.jsonl"))
                        sessions.append((p.get("agent"), sid, tr[0] if tr
else "(transcript not found)"))
    except Exception:
        pass

    # born: oldest reflog entry of the branch; own commits since then
    rl = run(["git", "-C", path, "reflog", "show", "--date=iso", "--
format=%H|%gd|%gs", branch if branch != "(detached)" else
"HEAD"]).split("\n")
    born_sha, born = (rl[-1].split("|")[0], rl[-1].split("|")[1]) if rl
and rl[-1] else (head, "?")
    if "{" in born: born = born[born.index("{") + 1:born.rindex("}")]
    commits = run(["git", "-C", path, "log", "--format=%h %cs %s",
f"{born_sha}..{head}"]).split("\n")
    commits = [c for c in commits if c][:20]
    in_main = subprocess.run(["git", "-C", path, "merge-base", "--is-
ancestor", head, main_sha],
                             capture_output=True).returncode == 0

    # successor: merged PR whose head is this HEAD
    pr = None
    try:
        for p in json.loads(run(["gh", "pr", "list", "--state", "merged",
"--limit", "100", "--json",
"number,headRefName,headRefOid,mergedAt,url,mergeCommit"],
cwd=main_path)):
            if p["headRefName"] == branch and p["headRefOid"] == head: pr
```

```
= p
    except Exception:
        pass

    dirty = [l for l in run(["git", "-C", path, "status", "--porcelain",
"-uall"]).split("\n") if l]
    blockers = []
    if dirty: blockers.append(f"{len(dirty)} uncommitted/untracked
file(s) would be LOST on removal")
    if not (pr or in_main): blockers.append("HEAD is not in main and no
merged PR matches it: not merged")
    if me.get("locked"): blockers.append(f"worktree is LOCKED
({me['locked'] if me['locked'] is not True else 'no reason'})")

    fm = [
        "---", "kind: worktree-tombstone", f"repo: {repo}", f"worktree:
{label}", f"branch: {branch}",
        f"head: {head}", f"born: {born}", f"ceased:
{now.isoformat(timespec='seconds')}",
        f"reason: {reason or ('merged PR #%d' % pr['number'] if pr else
'commits are in main')}",
        f"successor: {pr['url'] if pr else 'main@' + main_sha[:7]}",
        f"removed_path: {path}", "---", ""]
    body = [f"# Tombstone: {label}", "",
            "A worktree ceased here. The function ended; this is the
trace.", "",
            "## What it was", f"- repo `{repo}`, branch `{branch}`, born
{born}",
            "- own commits:" + ("" if commits else " none recorded")]
    body += [f"  - `{c}`" for c in commits]
    body += ["", "## Why it ended",
             f"- {reason or ('PR #%d merged %s' % (pr['number'],
pr['mergedAt'][:10]) if pr else 'all commits are in main (no PR)')}",
             f"- HEAD contained in main: {'yes' if in_main else 'no'}",
"", "## Successor",
             f"- {pr['url'] if pr else 'main@' + main_sha[:7]}", "", "##
Where the record lives",
             f"- branch `{branch}` is NOT deleted by this step",
             f"- commits reachable from `{head[:7]}`"]
    body += [f"- {a} session `{sid}`: `{tr}`" for a, sid, tr in sessions]
or ["- no agent session found in herdr"]
    if dirty:
        body += ["", "## Not committed at the time (would be lost on
removal)"] + [f"- `{l}`" for l in dirty[:40]]
    body += ["", "---", "AI-generated by herdr-oracle (Claude)."]
    text = "\n".join(fm + body) + "\n"

    vault = os.path.join(main_path, "ψ", "memory", "tombstones")
    dest_dir = vault if os.path.isdir(os.path.join(main_path, "ψ")) else
os.path.expanduser(f"~/.herdr/tombstones/{repo}")
    dest = os.path.join(dest_dir, f"{now.strftime('%Y-%m-%d')}
_{label}.md")
    n = 2
    while os.path.exists(dest):
```

```
        dest = os.path.join(dest_dir, f"{now.strftime('%Y-%m-%d')}
_{label}-{n}.md"); n += 1

    print(text)
    print(f"destination: {dest}")
    for b in blockers: print(f"BLOCKER: {b}")
    if write:
        os.makedirs(dest_dir, exist_ok=True)
        open(dest, "x", encoding="utf-8").write(text)
        print("written." + ("  Blockers above still apply to removal." if
blockers else ""))
    else:
        print("dry run - add --write to save it")

main()
```